
Universidade Catolica de Santos

Iniciacao Cientifica - Estudante Pesquisador

Estudo de eficiencia computacional de operacoes de Transformers

Relatorio tecnico, protocolo experimental e caderno de pesquisa

Estado da pesquisa em 18/09/2026

A validacao conceitual ampliada aprovou 18 comparacoes. As baterias sintetica v3, fisica v3, OPT v1, as extensoes OPT-2.7B/OPT-6.7B e a bateria complementar de lacunas foram concluidas e analisadas. O resultado mais recente mostra que pruning percentual denso e INT8 fake nos OPT nao reduzem VRAM nem sustentam speedup, enquanto caminhos fisicos maiores podem acelerar apenas quando a qualidade tambem precisa ser auditada.

Estudante	Lucas Cerqueira Galvao
Orientacao	Prof. Me. Walter Silva Oliveira
Grupo	LPCIAD
Repositorio	GitHub
Versao documental	18 de setembro de 2026

Nota sobre a fonte documental

Este PDF é compilado a partir de fontes LaTeX versionadas. Os arquivos Markdown remanescentes no repositório são apenas índices de navegação para o GitHub; CSV, JSON, logs, imagens e a transcrição original são evidências brutas. O plano em Word é preservado como documento institucional submetido, enquanto seu conteúdo acadêmico também aparece em LaTeX neste volume.

Sumário

Nota sobre a fonte documental	i
I Relatório executivo	1
1 Visão geral da pesquisa	2
1.1 Problema e recorte	2
1.2 Objetivo	2
1.3 Estratégia de validação	2
1.4 Estado comprovado	3
2 Plano de trabalho consolidado	5
2.1 Identificação	5
2.2 Resumo	5
2.3 Objetivo geral	6
2.4 Objetivos específicos	6
2.5 Contrato experimental	6
2.6 Infraestrutura	6
3 Estado atual e próximos passos	7
3.1 Evidências concluídas	7
3.2 Infraestrutura implementada para a etapa v3	7
3.3 Resultado da bateria sintética v3	8
3.4 Resultado da bateria física v3	9
3.5 Resultado dos modelos OPT pre-treinados	11
3.6 Extensão de escala OPT-2.7B e OPT-6.7B	13
3.7 Complemento de lacunas experimentais	16
3.8 Respostas atuais as perguntas de pesquisa	17
3.9 Critérios pre-registrados	18
3.10 Ordem restante de execução	18
3.11 Riscos que permanecem abertos	19
II Fundamentação e validação técnica	21
4 Base teórica e validação científica do modelo	22
4.1 Objetivo do documento	22
4.2 Princípio metodológico	22
4.3 Matriz de rastreabilidade	23

4.4	Glossario operacional	25
4.4.1	Rede neural	25
4.4.2	Deep learning	25
4.4.3	Transformer	25
4.4.4	Self-attention	26
4.4.5	Scaled dot-product attention	26
4.4.6	Inferencia	27
4.4.7	Projecao densa	27
4.4.8	KV cache	27
4.4.9	Pruning	28
4.4.10	Sparsity	28
4.4.11	Quantizacao	29
4.4.12	Latencia	29
4.4.13	Throughput	30
4.4.14	Memoria	30
4.4.15	FLOPs	30
4.4.16	MSE	31
4.4.17	MAE	31
4.4.18	R2	31
4.4.19	Similaridade de cosseno	32
4.5	criterios para chamar uma implementacao de Transformer	32
4.6	Validacao de ferramentas e algoritmos	33
4.7	Procedimento inicial de trabalho	35
4.8	criterios de aceite desta fase	35
4.9	Referencias iniciais	35
4.10	Nota metodológica	36
4.11	1. Rede neural	36
4.12	2. Deep learning	37
4.13	3. Transformer	37
4.14	4. Bloco Transformer simplificado	38
4.15	5. Projecção linear densa	38
4.16	6. Q, K e V	39
4.17	7. Scaled dot-product attention	39
4.18	8. Self-attention	40
4.19	9. Multi-head attention	40
4.20	10. Informação posicional	41
4.21	11. Inferência	41
4.22	12. KV cache	42
4.23	13. Pruning	42
4.24	14. Sparsity	42
4.25	15. Quantização	43

4.26	16. Latência	43
4.27	17. Throughput	44
4.28	18. Memória	44
4.29	Conclusão geral	45
5	Validacao arquitetural da NN usada no estudo	46
5.1	Resultado atual	46
5.2	Criterio de rastreabilidade	47
5.3	Matriz Vaswani -> codigo -> teste	47
5.4	O que esta comprovado	50
5.5	O que nao esta comprovado	50
5.6	Como a verificacao e feita	51
5.7	Controle negativo	51
5.8	Regra para o texto final	51
6	Explicacao narrativa da metodologia de validacao	52
6.1	Capitulo 1 - Rede neural e deep learning	53
6.1.1	Ideia inicial	53
6.1.2	Definicao cientifica	53
6.1.3	Por que isso importa no nosso trabalho	53
6.1.4	Como implementamos	54
6.1.5	Como testamos	54
6.1.6	Com o que comparamos	54
6.1.7	O que podemos concluir	54
6.1.8	O que ainda nao podemos concluir	54
6.2	Capitulo 2 - Transformer	55
6.2.1	Ideia inicial	55
6.2.2	Definicao cientifica	55
6.2.3	Por que isso importa no nosso trabalho	55
6.2.4	Como implementamos	55
6.2.5	Como testamos	55
6.2.6	Com o que comparamos	56
6.2.7	O que podemos concluir	56
6.2.8	O que ainda nao podemos concluir	57
6.2.9	Ponte direta entre Vaswani e os nossos scripts	57
6.3	Capitulo 3 - Bloco Transformer simplificado	59
6.3.1	Ideia inicial	59
6.3.2	Definicao cientifica	59
6.3.3	Por que isso importa no nosso trabalho	59
6.3.4	Como implementamos	59
6.3.5	Como testamos	60

6.3.6	Com o que comparamos	60
6.3.7	O que podemos concluir	60
6.3.8	O que ainda nao podemos concluir	60
6.4	Capitulo 4 - Self-attention	60
6.4.1	Ideia inicial	60
6.4.2	Definicao cientifica	61
6.4.3	Por que isso importa no nosso trabalho	61
6.4.4	Como implementamos	61
6.4.5	Como testamos	61
6.4.6	Com o que comparamos	61
6.4.7	O que podemos concluir	61
6.4.8	O que ainda nao podemos concluir	62
6.5	Capitulo 5 - Scaled dot-product attention	62
6.5.1	Ideia inicial	62
6.5.2	Definicao cientifica	62
6.5.3	Por que isso importa no nosso trabalho	62
6.5.4	Como implementamos	62
6.5.5	Como testamos	63
6.5.6	Com o que comparamos	63
6.5.7	O que podemos concluir	63
6.5.8	O que ainda nao podemos concluir	63
6.6	Capitulo 6 - Multi-head attention	63
6.6.1	Ideia inicial	64
6.6.2	Definicao cientifica	64
6.6.3	Por que isso importa no nosso trabalho	64
6.6.4	Como implementamos	64
6.6.5	Como testamos	64
6.6.6	Com o que comparamos	65
6.6.7	O que podemos concluir	65
6.6.8	O que ainda nao podemos concluir	65
6.7	Capitulo 7 - Projecao densa	65
6.7.1	Ideia inicial	65
6.7.2	Definicao cientifica	65
6.7.3	Por que isso importa no nosso trabalho	66
6.7.4	Como implementamos	66
6.7.5	Como testamos	66
6.7.6	Com o que comparamos	66
6.7.7	O que podemos concluir	66
6.7.8	O que ainda nao podemos concluir	66
6.8	Capitulo 8 - Informacao posicional	66
6.8.1	Ideia inicial	67

6.8.2	Definicao cientifica	67
6.8.3	Por que isso importa no nosso trabalho	67
6.8.4	Como implementamos	67
6.8.5	Como testamos	67
6.8.6	Com o que comparamos	67
6.8.7	O que podemos concluir	68
6.8.8	O que ainda nao podemos concluir	68
6.9	Capitulo 9 - Inferencia	68
6.9.1	Ideia inicial	68
6.9.2	Definicao cientifica	68
6.9.3	Por que isso importa no nosso trabalho	68
6.9.4	Como implementamos	68
6.9.5	Como testamos	68
6.9.6	Com o que comparamos	69
6.9.7	O que podemos concluir	69
6.9.8	O que ainda nao podemos concluir	69
6.10	Capitulo 10 - KV cache	69
6.10.1	Ideia inicial	69
6.10.2	Definicao cientifica	69
6.10.3	Por que isso importa no nosso trabalho	69
6.10.4	Como implementamos	70
6.10.5	Como testamos	70
6.10.6	Com o que comparamos	70
6.10.7	O que podemos concluir	70
6.10.8	O que ainda nao podemos concluir	70
6.11	Capitulo 11 - Pruning e sparsity	70
6.11.1	Ideia inicial	70
6.11.2	Definicao cientifica	71
6.11.3	Por que isso importa no nosso trabalho	71
6.11.4	Como implementamos	71
6.11.5	Como testamos	71
6.11.6	Com o que comparamos	72
6.11.7	O que podemos concluir	72
6.11.8	O que ainda nao podemos concluir	72
6.12	Capitulo 12 - Quantizacao	72
6.12.1	Ideia inicial	72
6.12.2	Definicao cientifica	72
6.12.3	Por que isso importa no nosso trabalho	72
6.12.4	Como implementamos	73
6.12.5	Como testamos	73
6.12.6	Com o que comparamos	73

6.12.7	O que podemos concluir	74
6.12.8	O que ainda nao podemos concluir	74
6.13	Capitulo 13 - Latencia, throughput, memoria e FLOPs	74
6.13.1	Ideia inicial	74
6.13.2	Definicao cientifica	74
6.13.3	Por que isso importa no nosso trabalho	74
6.13.4	Como implementamos FLOPs	74
6.13.5	Como testamos FLOPs	75
6.13.6	Como implementamos benchmark controlado	75
6.13.7	Com o que comparamos	75
6.13.8	O que podemos concluir	76
6.13.9	O que ainda nao podemos concluir	76
6.14	Capitulo 14 - MSE, MAE, R2 e similaridade de cosseno	76
6.14.1	Ideia inicial	76
6.14.2	Definicao cientifica	76
6.14.3	Por que isso importa no nosso trabalho	76
6.14.4	Como implementamos	76
6.14.5	Como testamos	77
6.14.6	Com o que comparamos	77
6.14.7	O que podemos concluir	77
6.14.8	O que ainda nao podemos concluir	77
6.15	Capitulo 15 - Protocolo experimental e matriz de validade	78
6.15.1	Ideia inicial	78
6.15.2	Definicao cientifica	78
6.15.3	Por que isso importa no nosso trabalho	78
6.15.4	Como implementamos	78
6.15.5	Como testamos	79
6.15.6	Com o que comparamos	79
6.15.7	O que podemos concluir	79
6.15.8	O que ainda nao podemos concluir	79
6.16	Fechamento - A frase metodologica do projeto	79
7	Perguntas de pesquisa e criterios de validade	81
7.1	Perguntas de pesquisa	81
7.2	Hipoteses	82
7.3	Regra de conclusao	84
8	Protocolo experimental antes dos benchmarks	85
8.1	Niveis de certeza	85
8.2	Portoes antes de benchmark	85
8.3	Validacao cruzada entre frameworks	86

8.4	Recorte experimental	86
8.5	Cenarios experimentais	87
8.6	Metricas	87
8.7	Schema CSV obrigatorio	88
8.8	Runner controlado	88
8.9	Runner principal das operacoes isoladas	88
8.10	Extensao v3 e modelos pre-treinados	89
8.11	Analise dos resultados	90
8.12	Validacao da NN como Transformer	90
8.13	Comparacao com literatura	90
8.14	Criterio para conclusao de hardware	91
9	Matriz de validacao de ferramentas	92
9.1	Regras para atualizar a matriz	97
10	Confronto entre resultados observados e literatura	98
10.1	Criterio usado	98
10.2	Resultados numericos verificados	98
10.3	O que ainda nao esta validado	102
10.4	Conclusao metodologica	102
10.5	Referencias	102
11	Decisão metodológica: inicialização dos pesos do benchmark	104
11.1	Problema observado	104
11.2	Alternativas consideradas	104
11.3	Regra operacional adotada	104
11.4	Confronto v1 versus v2	105
11.5	Decisão para a coleta principal	107
III	Evidencias e acompanhamento	108
12	Comparação numérica da attention local e multi-head	109
12.1	Conclusão	111
13	Relatório preliminar - smoke_cpu_v1	112
13.1	Pergunta, hipótese e desenho	112
13.2	Reprodutibilidade	112
13.3	Resultados por operação e cenário	112
13.4	Interpretação preliminar	114
13.5	Cadeia de evidência	114
13.6	Reprodução	114

14 Relatório preliminar - diagnostico_cpu_l64_d128_v1	115
14.1 Pergunta, hipótese e desenho	115
14.2 Reprodutibilidade	115
14.3 Resultados por operação e cenário	115
14.4 Interpretação preliminar	117
14.5 Cadeia de evidência	117
14.6 Reprodução	117
15 Relatório preliminar - diagnostico_cpu_l64_d128_v2	118
15.1 Pergunta, hipótese e desenho	118
15.2 Reprodutibilidade	118
15.3 Resultados por operação e cenário	118
15.4 Interpretação preliminar	120
15.5 Cadeia de evidência	120
15.6 Reprodução	120
16 Relatório preliminar - operacoes_principais_gpu_v1	121
16.1 Pergunta, hipótese e desenho	121
16.2 Reprodutibilidade	121
16.3 Resultados por operação e cenário	121
16.4 Interpretação preliminar	125
16.5 Cadeia de evidência	125
16.6 Reprodução	125
17 Auditoria do estado da IC - 25/08/2026	126
17.1 Objetivo e fontes examinadas	126
17.2 Estado comprovado	126
17.3 Correspondência com a orientação	127
17.4 Inconsistências e riscos encontrados	128
17.4.1 Escopo experimental	128
17.4.2 Parâmetros divergentes	128
17.4.3 Cenários e alegações de hardware	128
17.4.4 Reprodutibilidade e rastreabilidade	129
17.4.5 Viés de medição	129
17.4.6 Cronograma	129
17.5 Conclusão da auditoria	129
18 Checklist pré-benchmark	130
18.1 Obrigatório antes do benchmark principal	130
18.2 Recomendado antes do benchmark principal	131
18.3 Pode ser feito depois dos primeiros benchmarks	131
18.4 Regra de liberação	132

19 Relatório da execução autônoma - 25/08/2026	133
19.1 Resumo executivo	133
19.2 O que foi feito	133
19.3 Commits criados nesta execução	134
19.4 Orientações recuperadas do professor	134
19.5 Pré-benchmark	135
19.6 Benchmarks e validações executados	136
19.6.1 Comparação da atenção entre frameworks	136
19.6.2 Smoke CPU	136
19.6.3 Diagnóstico CPU v1	136
19.6.4 Diagnóstico CPU v2	136
19.7 Problemas e riscos encontrados	137
19.8 Pendências	137
19.8.1 Críticas	137
19.8.2 Importantes	137
19.8.3 Melhorias futuras	138
19.9 Próximo passo recomendado	138
20 Diário da iniciação científica	139
20.1 31/07/2026 - Reunião de orientação (entrada reconstruída)	139
20.2 24/08/2026 - Validação cruzada da atenção (entrada reconstruída)	139
20.3 25/08/2026 - Auditoria integral e portões pré-benchmark	140
20.3.1 Continuação - infraestrutura das operações isoladas	141
20.3.2 Continuação - análise entre execuções	141
20.3.3 Continuação - smoke canônico e portão da GPU	142
20.3.4 Continuação - diagnóstico CPU v1 em L=64, D=128	142
20.3.5 Continuação - correção metodológica da inicialização	143
20.3.6 Continuação - diagnóstico CPU v2 e confronto com v1	143
20.4 29/08/2026 - Migração integral da documentação para LaTeX	144
20.5 13/09/2026 - Validação multi-head e robustez sintética v3	144
20.6 14/09/2026 - Docker, kernels fisicos e bateria de hardware	145
20.7 15/09/2026 - Execução final dos modelos OPT	146
20.8 16/09/2026 - Extensão de escala OPT-2.7B e OPT-6.7B	147
20.9 17/09/2026 - Benchmarks complementares de lacunas	147
IV Registros e materiais de apoio	149
A Reunião de 31 de julho de 2026	150
A.1 Identificação	150
A.2 Leitura executiva	150
A.3 Decisões e direcionamentos	150

A.4	Reconstrução classificada da orientação	151
A.4.1	Pedidos explícitos do professor	151
A.4.2	Decisões tomadas durante a reunião	151
A.4.3	Dúvidas levantadas na reunião	152
A.4.4	Sugestões metodológicas, sem caráter de requisito adicional	152
A.4.5	Implementações e validações necessárias derivadas diretamente da reunião	152
A.4.6	Tarefas que podem ficar para depois dos primeiros benchmarks	152
A.4.7	Interpretações posteriores - não são pedidos literais do professor	153
A.5	Situação no repositório em 24/08/2026	153
A.6	Próximos passos priorizados	155
A.6.1	P0 - Fechar o contrato experimental	155
A.6.2	P1 - Concluir a validação e coletar evidências	155
A.6.3	P1 - Escrever o relatório junto com a análise	155
A.6.4	P2 - Extensão opcional	155
A.7	Critério de conclusão desta etapa	156
A.8	Trechos da reunião que sustentam os encaminhamentos	156
B	Guia de Estudo do Tema	157
B.1	1. O que é um Transformer	157
B.2	2. Por que a atenção custa caro	157
B.3	3. Diferença entre treino e inferência	157
B.4	4. O que é quantização	158
B.5	5. O que é pruning	158
B.6	6. O que é KV cache	158
B.7	7. O que o TurboQuant propõe	158
B.8	8. Por que quantização parece mais promissora aqui	159
B.9	9. O que significam as métricas	159
B.9.1	Latência	159
B.9.2	Memória	159
B.9.3	FLOPs	159
B.9.4	MSE	159
B.9.5	MAE	159
B.9.6	R^2	160
B.9.7	Similaridade de cosseno	160
B.10	10. Como explicar seu experimento de forma clara	160
B.11	11. Perguntas prováveis da banca	160
B.11.1	'Por que você escolheu Transformer?'	160
B.11.2	'Por que comparar quantização com pruning?'	160
B.11.3	'Por que dizer que quantização é melhor se você ainda não rodou o experimento?'	160

B.11.4	'Por que usar cenário controlado e modelo simplificado?'	161
B.11.5	'O que você espera encontrar?'	161
B.12	12. Frase de segurança para a apresentação	161
C	Roteiro de 10 Minutos	162
C.1	Slide 1 - Capa (20s)	162
C.2	Slide 2 - Roteiro (20s)	162
C.3	Slide 3 - Transformers e a IA atual (35s)	162
C.4	Slide 4 - Custo da inferência (30s)	162
C.5	Slide 5 - Gargalos principais (40s)	163
C.6	Slide 6 - Problema e hipótese (30s)	163
C.7	Slide 7 - Objetivos (25s)	163
C.8	Slide 8 - Projeções densas (30s)	163
C.9	Slide 9 - Self-attention e KV cache (40s)	164
C.10	Slide 10 - Quantização (30s)	164
C.11	Slide 11 - Pruning (30s)	164
C.12	Slide 12 - TurboQuant (45s)	164
C.13	Slide 13 - Metodologia: desenho (35s)	165
C.14	Slide 14 - Metodologia: métricas (35s)	165
C.15	Slide 15 - Resultados parciais (30s)	165
C.16	Slide 16 - Evidências da literatura (40s)	165
C.17	Slide 17 - Conclusões (35s)	165
C.18	Slide 18 - Referências (20s)	166
C.19	Slide 19 - Obrigado (10s)	166
C.20	Onde acelerar se faltar tempo	166
D	Observações	167
D.1	Reunião em 31 de jul. de 2026 às 16:01 GMT-03:00	167
D.1.1	Resumo	167
D.1.2	Próximas etapas	167
D.1.3	Detalhes	168
E	Transcrição	170
E.1	Reunião em 31 de jul. de 2026 às 16:01 GMT-03:00 \- Transcrição	170
E.1.1	00:00:00	170
E.1.2	00:01:28	171
E.1.3	00:02:53	171
E.1.4	00:04:12	172
E.1.5	00:05:35	173
E.1.6	00:06:51	173
E.1.7	00:08:01	174
E.1.8	00:09:18	175

E.1.9	00:10:29	175
E.1.10	00:11:27	176
E.1.11	00:12:54	177
E.1.12	00:14:09	177
E.1.13	00:15:21	178
E.1.14	00:16:15	179
E.1.15	00:17:28	180
E.1.16	00:18:20	180
E.1.17	00:19:20	181
E.1.18	00:20:29	182
E.1.19	00:21:24	183
E.1.20	00:22:20	183
E.1.21	00:23:13	184
E.1.22	00:24:34	185
E.1.23	00:25:52	186
E.1.24	00:26:48	187
E.1.25	00:28:07	187
E.1.26	A transcrição foi encerrada após 00:30:07	189

Lista de Figuras

3.1	Degradacao numerica por nivel de sparsity na bateria sintetica v3.	9
3.2	Speedup e intervalos bootstrap de 95% na bateria fisica; todos os grupos ficaram abaixo do baseline.	10
3.3	Memoria de pico relativa ao baseline, com intervalos bootstrap de 95%. . . .	10
3.4	Speedup mediano e intervalos bootstrap de 95%; nenhum candidato satisfaz o criterio de ganho fisico.	11
3.5	Perplexidade por sparsity e modelo na avaliacao OPT.	12
3.6	Speedup em prefill e TTFT dos modelos OPT; nenhum caso satisfaz todos os criterios pre-registrados.	12
3.7	Memoria de pico relativa nos modelos OPT.	13
3.8	Compromisso entre speedup de prefill e aumento de perplexidade nas extensoes OPT-2.7B e OPT-6.7B.	15
3.9	Speedup em prefill e TTFT na extensao OPT-6.7B.	15
3.10	Memoria de pico relativa na extensao OPT-6.7B.	16
3.11	Latencia relativa do benchmark GPU v1, preservada como evidencia historica.	20
3.12	Fidelidade numerica do benchmark GPU v1.	20

Parte I

Relatorio executivo

Visao geral da pesquisa

1.1 Problema e recorte

A arquitetura Transformer sustenta grande parte dos sistemas modernos de inteligencia artificial, mas suas projecoes lineares densas e o mecanismo de self-attention concentram custo de processamento, memoria e movimentacao de dados. Esta iniciacao cientifica investiga como configuracoes numericas e estruturais alteram esse custo durante a inferencia.

O objeto experimental principal e a self-attention multi-head completa, formada pelas projecoes Q, K, V e O, separacao em cabecas, SDPA e recombinacao. A projecao linear densa permanece como controle das operacoes dominantes, e tres modelos OPT pre-treinados permitem verificar se as tendencias sinteticas se mantem em inferencia real, sem treinamento ou fine-tuning.

1.2 Objetivo

Analisar, em cenarios controlados de inferencia, os compromissos entre eficiencia computacional e preservacao da saida ao aplicar pruning por magnitude e quantizacao linear INT8 sobre operacoes centrais de Transformers.

1.3 Estrategia de validacao

A pesquisa separa quatro niveis de evidencia:

1. **conceitual**: a definicao e o recorte correspondem a literatura;
2. **matematica**: casos pequenos reproduzem valores calculados;
3. **algoritmica**: implementacoes independentes produzem resultados equivalentes;
4. **hardware**: o caminho executado, o tipo numerico, o formato e o kernel sustentam um ganho fisico mensuravel.

A aprovacao de um nivel nao promove automaticamente o resultado ao nivel seguinte. Em particular, zeros logicos e tensores dequantizados nao bastam para afirmar aceleracao em hardware.

1.4 Estado comprovado

Componente	Estado	Evidencia principal
Matematica da atencao	validado	Casos manuais, NumPy independente e testes deterministas.
Arquitetura do re-corte	validado	Auditoria do bloco simplificado, dependencia entre posicoes e controle negativo.
Equivalencia entre frameworks	validado	Dezoito comparacoes entre NumPy, PyTorch manual, PyTorch SDPA e Keras/TensorFlow, incluindo Q/K/V/O e multi-head.
Pipeline experimental	validado	Configuracao versionada, duas execucoes, hashes, manifesto, metadados, analise e graficos.
Diagnostico em CPU	validado	Smoke e duas versoes do ponto $L = 64$, $D = 128$, sem conclusao de hardware.
Benchmark GPU v1	validado	Duas execucoes, 108 registros, hashes, analise e graficos na RTX 4070 Ti SUPER.
Infraestrutura v3	validado	Cinco seeds, 13 perfis, timings brutos, bootstrap, Docker, TorchAO, 2:4 e runners OPT.
Benchmark sintetico v3	validado	2.340 registros completos, 1.950 comparacoes pareadas e 117 mil amostras de latencia na GPU.
Caminhos fisicos	validado	Probe, smoke e 1.650 casos no container Linux; kernels INT8 e 2:4 confirmados pelo profiler.
Modelos OPT	validado	Tres modelos pre-treinados avaliados: 45 registros de qualidade, 2.880 janelas, 900 prompts, 1.296 registros de desempenho e 10.080 timings; decode registrado como falha tecnica.

Conclusao permitida neste momento

As implementacoes local, multi-head e com projecoes completas sao numericamente equivalentes as referencias escolhidas. A bateria v3 sustenta que a fidelidade do INT8 fake foi superior a do pruning nos casos sinteticos pareados, mas nao prova aceleracao fisica. Na bateria com kernels confirmados, INT8 e pruning 2:4 reduziram armazenamento e memoria, mas nenhum dos seis grupos obteve speedup sustentado. Nos modelos OPT, INT8 preservou qualidade muito melhor que pruning e reduziu memoria/armazenamento em varios cenarios, mas prefill e TTFT nao atingiram o criterio completo de speedup sustentado. A medicao de decode ficou limitada por interacao entre `torch.compile`, CUDA Graphs e KV cache.

Plano de trabalho consolidado

2.1 Identificacao

Campo	Valor
Titulo	Estudo de eficiencia computacional de operacoes de Transformers sob diferentes configuracoes numericas e estruturais
Estudante	Lucas Cerqueira Galvao
Curso	Ciencia da Computacao
Grupo de pesquisa	LPCIAD - Laboratorio de Projetos de Circuitos Integrados Analogicos e Digitais
Orientacao	Prof. Me. Walter Silva Oliveira
Vigencia original	Maio a setembro de 2026
Dedicacao registrada	8 horas por semana, 4 meses, 80 horas-aula de atividade curricular de pesquisa
Projeto vinculado	Registro Ipeco 20240004 - Projeto e Otimizacao de Circuitos Integrados utilizando Inteligencia Artificial

O formulario institucional submetido permanece preservado em docs/plano_trabalho/PlanoTrabalhoEP_LucasGalvao.docx. Esta secao e a versao academica editavel em LaTeX e omite instrucoes de preenchimento, assinaturas e dados pessoais que nao interferem no metodo cientifico.

2.2 Resumo

A arquitetura Transformer consolidou-se como alicerce de sistemas contemporaneos de inteligencia artificial, mas impoe custo significativo de execucao, memoria e operacoes aritmeticas durante a inferencia. O estudo analisa como configuracoes numericas e estruturais afetam operacoes de autoatencao e projecoes lineares densas. Um baseline sem otimizacao e comparado com pruning por magnitude e quantizacao linear INT8. A avaliacao combina latencia, pico de memoria e FLOPs analiticos com MSE, MAE, coeficiente de determinacao e similaridade de

cosseno, buscando caracterizar o compromisso entre reducao de custo e preservacao da saida.

2.3 Objetivo geral

Analisar como diferentes configuracoes numericas e estruturais afetam a eficiencia computacional de operacoes centrais da arquitetura Transformer em cenarios controlados de inferencia.

2.4 Objetivos especificos

1. implementar, em Python, um modelo computacional simplificado com foco em autoatencao e projecao linear densa;
2. construir cenarios comparaveis de baseline, pruning por magnitude e quantizacao linear;
3. mensurar latencia, memoria e estimativas analiticas de operacoes;
4. avaliar as saidas por MSE, MAE, R^2 e similaridade de cosseno;
5. interpretar os compromissos entre reducao de complexidade e preservacao da acuracia numerica.

2.5 Contrato experimental

O protocolo principal usa Python 3.11, PyTorch/CUDA, NumPy, Pandas, scikit-learn e Matplotlib. A grade formal tem lote 1, comprimentos de sequencia $L \in \{64, 128, 256\}$ e dimensoes $D \in \{128, 256, 512\}$. Cada combinacao mede baseline, pruning de 50% e quantizacao INT8, com 20 iteracoes de aquecimento, 50 medicoes validas e duas execucoes independentes. A semente principal e 42; entradas seguem $\mathcal{N}(0, 1)$, pesos usam inicializacao Xavier normal ajustada a dimensao e o bias e zero.

2.6 Infraestrutura

A coleta principal foi planejada para uma NVIDIA GeForce RTX 4070 Ti Super de 16 GB. TensorFlow/Keras participa apenas do portao de equivalencia numerica em CPU; o benchmark de desempenho permanece no caminho PyTorch/CUDA. A pesquisa nao requer laboratorio especifico nem material de consumo, alem da infraestrutura computacional ja disponivel.

Estado atual e proximos passos

3.1 Evidencias concluidas

O portao conceitual foi ampliado de nove para dezoito comparacoes. Os casos originais validam o nucleo da scaled dot-product attention; os nove casos novos validam a composicao multi-head com projecoes Q, K, V e O para $H \in \{1, 4, 8\}$. NumPy manual, PyTorch manual, PyTorch SDPA e Keras/TensorFlow receberam as mesmas entradas e pesos. As 18 comparacoes passaram com $\text{atol}=1\text{e-}6$ e $\text{rtol}=1\text{e-}5$; o maior erro absoluto foi $4,76837158 \times 10^{-7}$. Essa diferenca e arredondamento de ponto flutuante, nao divergencia conceitual.

O benchmark GPU v1 tambem esta concluido e preservado. Foram realizadas duas execucoes na RTX 4070 Ti SUPER, com 54 registros em cada, para projecao densa e self-attention de uma cabeca. Configuracao, ambiente, logs, CSVs, metadados, graficos e relatorio foram copiados sem alteracao e verificados por SHA-256.

Limite da evidencia v1

O v1 usa uma seed, poucos shapes, pruning denso e quantizacao fake seguida de dequantizacao. Ele permite comparar erro numerico e tempos observados dessa implementacao, mas nao sustenta uma alegacao de kernel INT8 ou sparse nem uma generalizacao entre entradas e modelos.

3.2 Infraestrutura implementada para a etapa v3

O schema v3 mantem a leitura dos schemas v1/v2 e adiciona identidade de caso, seed, repeticao, perfil, grupo de baseline, dtype, backend, memoria, armazenamento, hashes, status e confirmacao de kernel. Cada registro agregado e ligado as amostras individuais em `timings.csv`. A retomada exige coincidencia dos hashes de configuracao, codigo, ambiente e artefatos existentes.

A bateria sintetica foi fixada em 13 combinacoes de perfil e distribuicao, cinco seeds, seis cenarios e tres repeticoes. Para as duas operacoes isso produz 2.340 registros agregados e 117 mil amostras de latencia. O pruning por magnitude usa 10%, 25%, 50% e 75%; a quantizacao fake e simetrica, por linha, com retorno a FP32 para medir fidelidade numerica.

O caminho fisico foi isolado em um container Linux com PyTorch 2.11, CUDA 12.8, TorchAO e cuSPARSELt fixados. Antes da coleta, o probe verifica GPU, compute capability

8.9, Triton, forward finito, representacao INT8, conversao 2:4 e os operadores do profiler. Candidatos incompatíveis permanecem registrados como unsupported; nunca são removidos silenciosamente.

O runner de modelos carrega separadamente OPT-125M, OPT-350M e OPT-1.3B em revisoes imutáveis. Cada variante parte novamente dos pesos originais e altera somente inferencia. A qualidade usa 64 janelas uniformes de 512 tokens do WikiText-2 versionado e 20 prompts locais; o desempenho separa prefill, TTFT e decode com KV cache. Os tres modelos e o dataset foram armazenados em cache e reabertos durante o prefetch, totalizando 6,63 GB para a coleta offline.

A suite no container desta revisao aprovou 71 testes; o unico skip e a integracao externa com download, protegida por ativacao explicita.

3.3 Resultado da bateria sintetica v3

A bateria terminou com os 2.340 registros em estado complete: 390 baselines e 1.950 candidatos corretamente pareados. As tres repeticoes registraram 39 mil amostras cada, perfazendo 117 mil timings. Os manifests de execucao e analise verificam os hashes de todos os CSVs, figuras e relatorios. A evidencia esta em `fases/01_validacao_conceitual/evidencias/benchmarks/robustez_sintetica_v3_2026-09-13/`.

Cenario	MSE projecao densa	MSE multi-head
INT8 fake por linha	$5,854 \times 10^{-5}$	$1,349 \times 10^{-5}$
Pruning 10%	$5,676 \times 10^{-4}$	$1,285 \times 10^{-4}$
Pruning 25%	$8,984 \times 10^{-3}$	$2,003 \times 10^{-3}$
Pruning 50%	$7,669 \times 10^{-2}$	$1,425 \times 10^{-2}$
Pruning 75%	$2,971 \times 10^{-1}$	$3,394 \times 10^{-2}$

Em 100% dos pares, o INT8 fake apresentou MSE menor que o pruning nos quatro niveis. Os intervalos bootstrap de 95% para a diferenca permaneceram acima de zero, e a mediana do MSE de pruning cresceu monotonicamente com a sparsity. Isso sustenta H1/H5 e H6 no recorte sintetico. Valores absolutos entre as duas operacoes nao devem ser comparados diretamente porque suas escalas de saida diferem.

Nenhum cenario sintetico sustentou ganho de desempenho: todos os intervalos de speedup cruzaram 1,0, nenhum kernel INT8 ou sparse foi usado, e o armazenamento permaneceu denso. Portanto, essa bateria responde fidelidade e robustez numerica, enquanto a bateria fisica apresentada a seguir responde representacao, memoria e latencia no hardware-alvo.

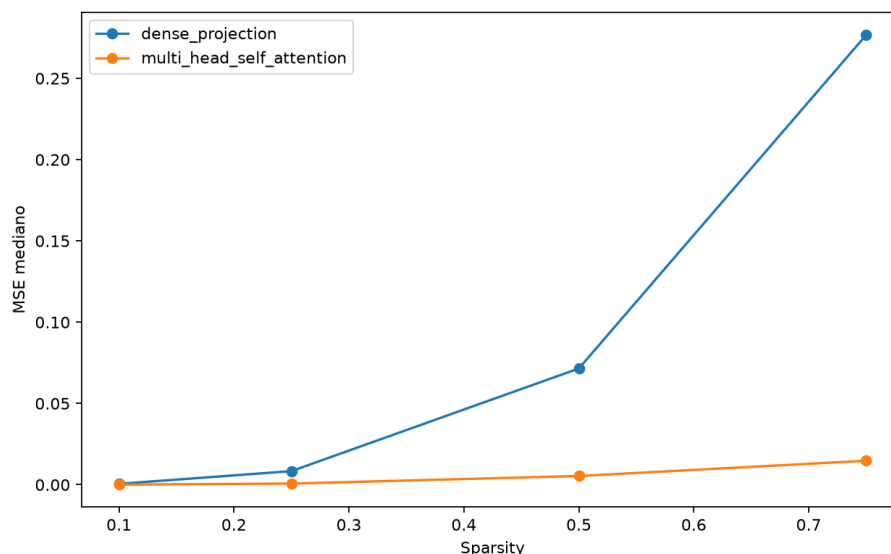


Figura 3.1: Degradação numerica por nível de sparsity na bateria sintetica v3.

3.4 Resultado da bateria fisica v3

O probe aprovou CUDA 12.8, compute capability 8.9, Triton, TorchAO INT8 weight-only e dinamico, além da conversão semiestruturada 2:4 por cuSPARSELt. Os traces registram o kernel packed para pesos INT8, GEMM S8 CUTLASS no caminho dinamico e sparse GEMM cuSPARSELt no caminho 2:4. A coleta completa produziu 1.650 registros em estado complete, 990 pares e 82.500 amostras de latencia. Manifestos e artefatos da análise foram verificados por SHA-256. A evidencia esta em fases/01_validacao_conceitual/evidencias/benchmarks/hardware_nativo_v3_2026-09-14/.

Operacao	Cenario	Speedup mediano	Memoria mediana	Armazenamento
Projecao densa	INT8 dinamico	0,808	0,801	0,504
Projecao densa	INT8 weight-only	0,136	0,801	0,504
Projecao densa	pruning 2:4	0,823	0,825	0,564
Multi-head attention	INT8 dinamico	0,650	0,639	0,503
Multi-head attention	INT8 weight-only	0,097	0,639	0,503
Multi-head attention	pruning 2:4	0,645	0,728	0,562

As razões de memória e armazenamento abaixo de 1 mostram redução física, mas todos os intervalos bootstrap de speedup ficaram inteiramente abaixo de 1. Assim, a bateria sustenta redução de armazenamento e de memória de pico, mas contradiz ganho de latência neste recorte. O weight-only continua classificado como representação compacta de pesos; somente o INT8 dinamico confirma computação inteira de ativações e pesos.

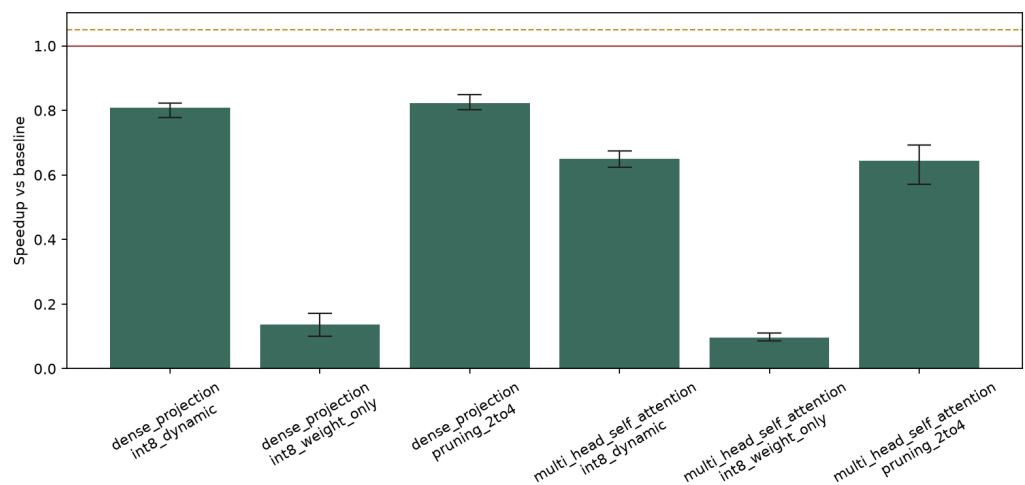


Figura 3.2: Speedup e intervalos bootstrap de 95% na bateria fisica; todos os grupos ficaram abaixo do baseline.

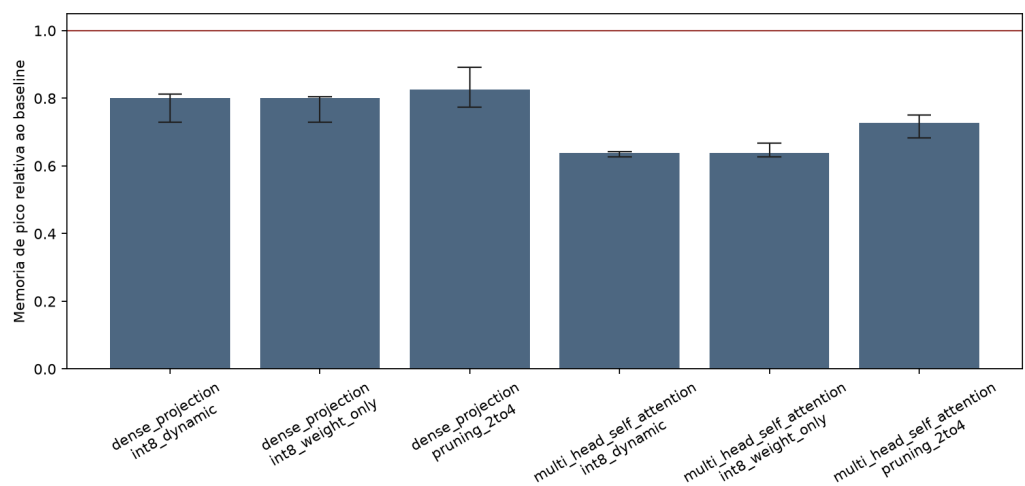


Figura 3.3: Memoria de pico relativa ao baseline, com intervalos bootstrap de 95%.

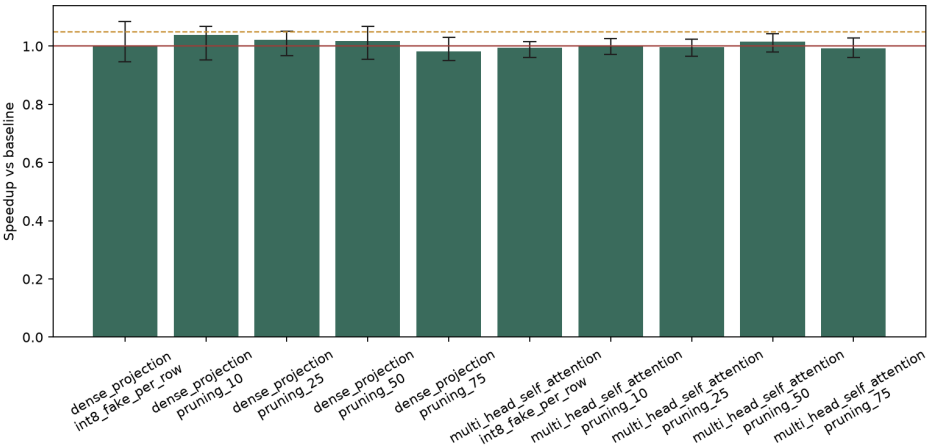


Figura 3.4: Speedup mediano e intervalos bootstrap de 95%; nenhum candidato satisfaz o criterio de ganho fisico.

3.5 Resultado dos modelos OPT pre-treinados

A coleta final dos modelos foi executada em 15/09/2026 apos o portao de GPU confirmar baixa ocupacao: uso medio de 0,4%, pico de 2%, 1.337 MiB de VRAM, 47 graus Celsius, 10,3 W e clock de 210 MHz. A evidencia esta em fases/01_validacao_conceitual/evidencias/benchmarks/modelos_opt_v1_2026-09-15/. O smoke final tambem foi promovido para fases/01_validacao_conceitual/evidencias/benchmarks/modelos_opt_smoke_2026-09-15/.

Foram avaliados facebook/opt-125m, facebook/opt-350m e facebook/opt-1.3b, sem treinamento ou fine-tuning. A execucao produziu 45 registros agregados de qualidade, 2.880 janelas de qualidade/perplexidade, 900 metricas de prompts, 1.296 registros de desempenho, 10.080 amostras de timing, subconjunto de avaliacao versionado, traces de operadores e manifesto de analise.

Modelo	Baseline BF16	INT8 fake atencao	Pruning 50% atencao
OPT-125M	42,371	42,313	65,796
OPT-350M	33,345	33,348	50,668
OPT-1.3B	21,394	21,384	28,644

Na qualidade, 25 de 45 variantes ficaram dentro dos limites operacionais. As quantizacoes INT8 fake e weight-only permaneceram dentro do limite nos tres modelos. O INT8 dinamico foi aceitavel em OPT-125M e OPT-350M, mas excedeu o limite de 2% no OPT-1.3B: 4,11% quando aplicado apenas a atencao e 2,34% quando aplicado ao controle global. O pruning por magnitude em atencao foi aceitavel apenas em 10%. A partir de 25%, a perplexidade ja excedeu levemente o limite de 5%; em 50% e 75%, a degradacao tornou-se grande em todos os modelos. O pruning 2:4 tambem degradou qualidade nos tres modelos.

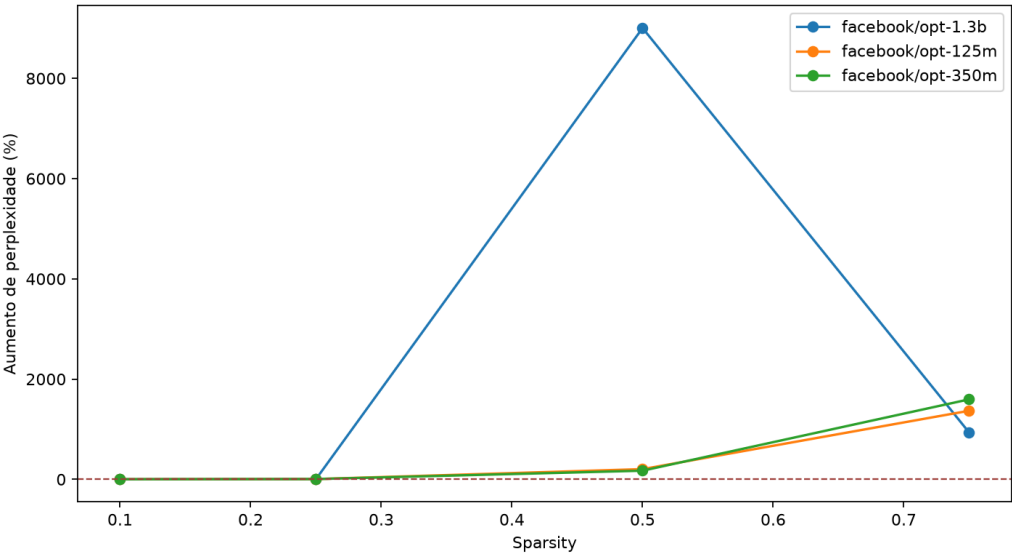


Figura 3.5: Perplexidade por sparsity e modelo na avaliacao OPT.

No desempenho, prefill e TTFT foram coletados com registros completos. Nenhuma comparacao cumpriu simultaneamente speedup mediano de pelo menos 1,05, intervalo bootstrap de 95% inteiramente acima de 1,0 e kernel confirmado. O caso mais favoravel foi blocks_int8_dynamic no OPT-1.3B, com speedup mediano de 1,271 em prefill e 1,256 em TTFT, mas os intervalos ainda cruzaram o efeito nulo. Portanto, essa tendencia e mista, nao sustentada.

Modelo	Operacao	Cenario	Speedup med.	IC95 baixo	IC95 alto	Memoria med.
OPT-125M	prefill	atencao INT8 dinamico	0,806	0,677	0,932	0,869
OPT-125M	TTFT	atencao INT8 dinamico	0,799	0,705	0,902	0,869
OPT-350M	prefill	atencao INT8 dinamico	0,798	0,776	0,903	0,849
OPT-350M	TTFT	atencao INT8 dinamico	0,794	0,692	0,882	0,849
OPT-1.3B	prefill	blocos INT8 dinamico	1,271	0,958	1,400	0,542
OPT-1.3B	TTFT	blocos INT8 dinamico	1,256	0,996	1,512	0,542

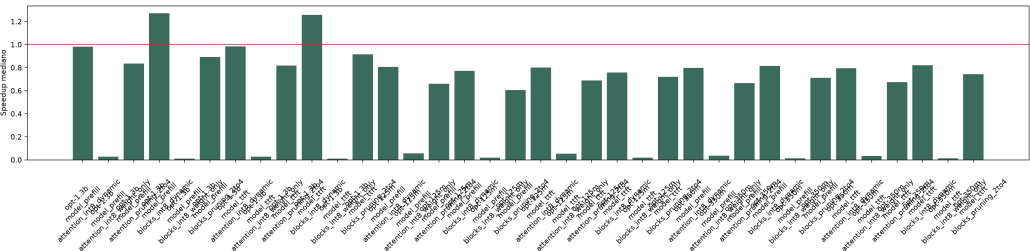


Figura 3.6: Speedup em prefill e TTFT dos modelos OPT; nenhum caso satisfaz todos os criterios pre-registrados.

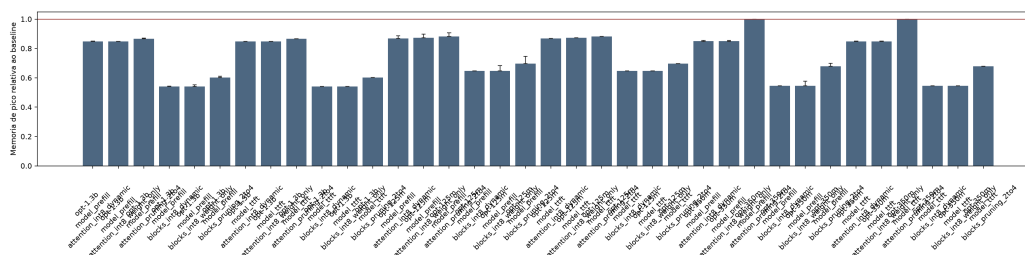


Figura 3.7: Memória de pico relativa nos modelos OPT.

Todos os 288 registros planejados de `model_decode` foram preservados, mas não usados como evidência de latência: 282 ficaram em `failed` e 6 em `unsupported`. O erro predominante foi a interação entre `torch.compile` em modo `reduce-overhead`, CUDA Graphs e a atualização do KV cache, que sobrescreveu saídas de grafos capturados durante a geração token a token. Assim, a coleta OPT responde qualidade, prefill, TTFT, memória, armazenamento e confirmação de operadores, mas não sustenta conclusões de latência por token em `decode`.

3.6 Extensao de escala OPT-2.7B e OPT-6.7B

Depois da bateria OPT principal, foram executadas duas extensoes exploratorias para observar a hipotese de escala: facebook/opt-2.7b e facebook/opt-6.7b. Essas execucoes nao substituem a bateria principal, mas ajudam a responder se tecnicas fisicas tendem a ficar mais vantajosas quando o modelo cresce. As evidencias foram preservadas em fases/01_validacao_conceitual/evidencias/benchmarks/modelos_opt_2_7b_2026-09-15/ e fases/01_validacao_conceitual/evidencias/benchmarks/modelos_opt_6_7b_2026-09-16/.

O OPT-2.7B terminou 15 cenários, 216 registros de desempenho e 1.440 amostras de timing. O OPT-6.7B também terminou 15 cenários, 216 registros de desempenho e 1.440 amostras de timing, usando guarda de VRAM em 15.800 MiB e retomada por cenário. O cenário mais demorado foi `blocks_int8_weight_only`, que levou quase uma hora somente para consolidar uma variante no OPT-6.7B; esse tempo é uma evidência importante de custo operacional do caminho `weight-only` quando aplicado globalmente.

Modelo	Cenario		Perplexidade	Aumento vs baseline		Aceitavel
OPT-2.7B	baseline	BF16	18,178	0,00%		—
OPT-2.7B	atencao	INT8 dinamico	18,955	4,27%		nao
OPT-2.7B	blocos	INT8 dinamico	18,981	4,42%		nao
OPT-2.7B	atencao	2:4	27,670	52,22%		nao
OPT-2.7B	blocos	2:4	519,074	2755,50%		nao
OPT-6.7B	baseline	BF16	15,346	0,00%		—
OPT-6.7B	atencao	INT8 dinamico	16,659	8,55%		nao
OPT-6.7B	blocos	INT8 dinamico	40,675	165,05%		nao
OPT-6.7B	atencao	2:4	21,311	38,87%		nao
OPT-6.7B	blocos	2:4	168,097	995,38%		nao
Modelo	Operacao	Cenario	Speedup med.	IC95 baixo	IC95 alto	Memoria med.
OPT-2.7B	prefill	atencao INT8 dinamico	1,024	0,844	1,134	0,842
OPT-2.7B	prefill	blocos INT8 dinamico	1,305	0,907	1,588	0,526
OPT-2.7B	prefill	atencao 2:4	0,873	0,798	0,907	0,862
OPT-2.7B	prefill	blocos 2:4	1,061	0,975	1,078	0,585
OPT-6.7B	prefill	atencao INT8 dinamico	1,169	1,009	1,249	0,839
OPT-6.7B	prefill	blocos INT8 dinamico	1,756	1,171	2,137	0,516
OPT-6.7B	prefill	atencao 2:4	1,030	0,974	1,111	0,859
OPT-6.7B	prefill	blocos 2:4	1,073	1,017	1,228	0,577

A extensao muda a resposta sobre escala. No OPT-6.7B, o INT8 dinamico aplicado aos blocos teve speedup mediano de 1,756 em prefill e 1,744 em TTFT, com intervalos bootstrap inteiramente acima de 1,0. O pruning 2:4 aplicado aos blocos tambem teve speedup acima do criterio em prefill. Mesmo assim, nenhum dos dois fica cientificamente aceitavel como tecnica final nesta configuracao, porque a perplexidade aumentou 165,05% e 995,38%, respectivamente. A conclusao e que modelos maiores podem tornar alguns kernels mais favoraveis, mas o ganho de desempenho precisa ser lido junto da qualidade.

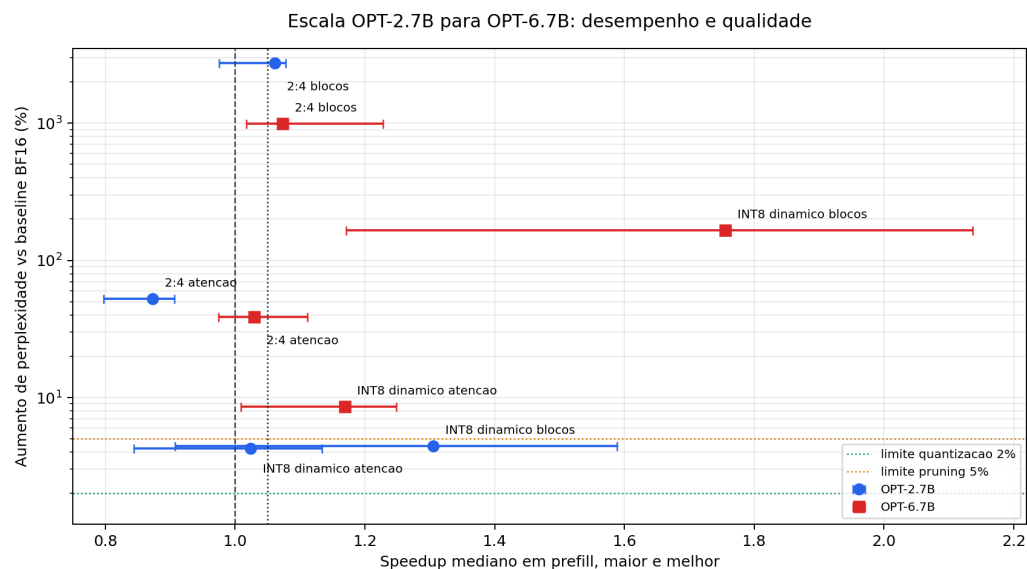


Figura 3.8: Compromisso entre speedup de prefill e aumento de perplexidade nas extensões OPT-2.7B e OPT-6.7B.

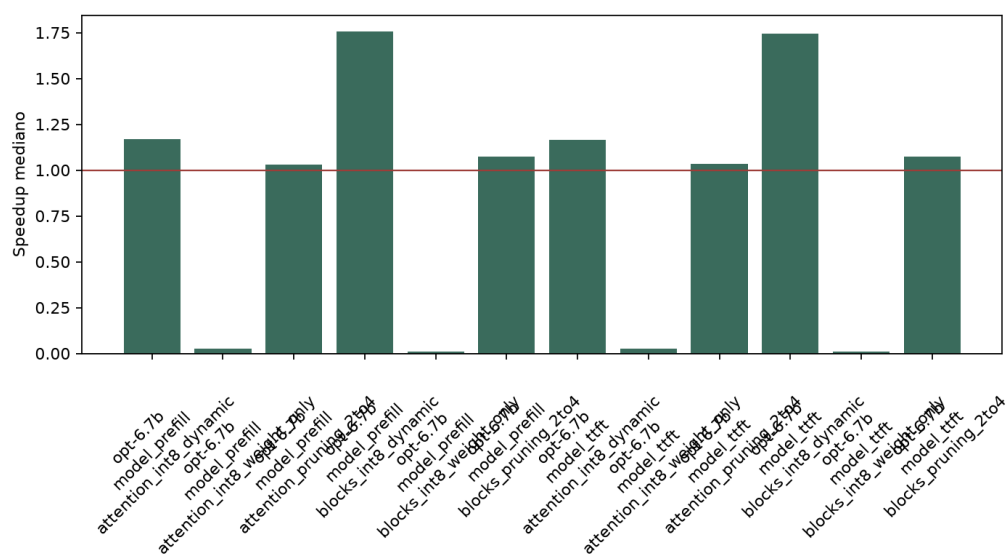


Figura 3.9: Speedup em prefill e TTFT na extensão OPT-6.7B.

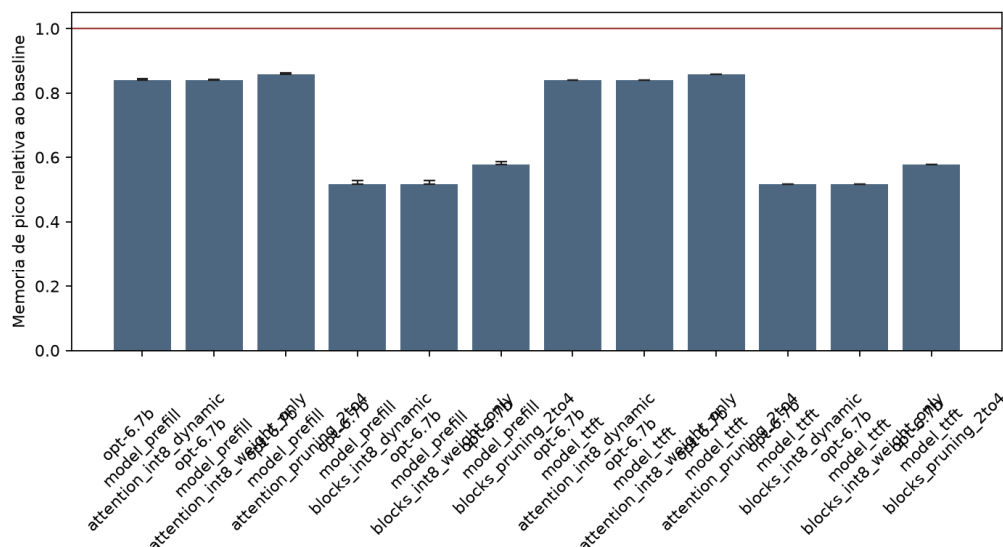


Figura 3.10: Memória de pico relativa na extensão OPT-6.7B.

3.7 Complemento de lacunas experimentais

A bateria complementar de 17/09/2026 foi executada para remover células vazias ambíguas da planilha de resultados. Ela preserva as evidências antigas e acrescenta três blocos: perfis físicos de stress para operações isoladas, desempenho/VRAM de pruning percentual e INT8 fake nos modelos OPT-125M, OPT-350M, OPT-1.3B e OPT-2.7B, e a mesma lacuna no OPT-6.7B com guarda de VRAM em 15.800 MiB. As evidências ficam em fases/01_validacao_conceitual/evidencias/benchmarks/hardware_stress_complementar_v3_2026-09-17/, fases/01_validacao_conceitual/evidencias/benchmarks/modelos_opt_complementar_lacunas_2026-09-17/ e fases/01_validacao_conceitual/evidencias/benchmarks/modelos_opt_6_7b_complementar_lacunas_2026-09-17/.

O complemento fisico de stress produziu 300 registros e 15.000 amostras de timing. O complemento OPT produziu 40 registros de qualidade, 1.080 registros de desempenho e 7.200 amostras de timing. A conclusao principal e negativa: pruning denso percentual e INT8 fake nao ativam caminhos fisicos de memoria ou latencia. Por isso, a VRAM mediana permaneceu essencialmente em 1,000 vez o baseline e os speedups ficaram mistos ou contraditos.

Modelo	Operacao	Pruning 10%	Pruning 25%	Pruning 50%	Pruning 75%	INT8 fake attn.	INT8 fake blocos
OPT-125M	prefill	0,993	0,981	0,984	0,979	0,973	0,976
OPT-350M	prefill	0,997	0,859	0,997	0,983	1,003	1,010
OPT-1.3B	prefill	1,004	1,008	1,002	1,009	1,007	1,005
OPT-2.7B	prefill	0,999	1,000	1,002	0,975	0,976	0,975
OPT-6.7B	prefill	1,000	0,924	0,923	0,926	0,922	0,924

No OPT-6.7B, o pruning de 10% em atencao preservou qualidade e ficou praticamente neutro em latencia. O pruning de 25% ainda permaneceu dentro do limite operacional de

perplexidade, mas reduziu o desempenho para 0,924 vez o baseline em prefill. Em 50% e 75%, a perplexidade cresceu 50,59% e 1.328,60%, respectivamente. Assim, o complemento reforça que zeros densos podem ser uteis como estudo de degradacao numerica, mas nao devem ser vendidos como aceleracao fisica sem representacao e kernel correspondentes.

3.8 Respostas atuais as perguntas de pesquisa

Pergunta	Resposta sustentada ate esta revisao
RQ1	No recorte sintetico, INT8 fake preservou melhor a saida que pruning por magnitude em 100% dos pares.
RQ2	INT8 fisico reduziu armazenamento para cerca de 50% do baseline; 2:4, para cerca de 56%. A memoria de pico mediana tambem caiu nos seis grupos.
RQ3	Nenhuma tecnica sustentou ganho de latencia nas operacoes isoladas. Nos OPT v1 e no complemento de pruning percentual/INT8 fake, prefill e TTFT tambem nao satisfizeram todos os criterios; na extensao OPT-6.7B houve speedup fisico em alguns caminhos, mas sem qualidade aceitavel. Decode ficou tecnicamente limitado.
RQ4	A separacao por nivel foi confirmada: quantizacao fake responde fidelidade, weight-only responde pesos compactos e INT8 dinamico/2:4 exigem trace do kernel.
RQ5	As tendencias numericas principais persistiram entre cinco seeds, distribuicoes e perfis; os efeitos temporais sinteticos permaneceram mistos.
RQ6	A degradacao por pruning cresceu com a sparsity. No complemento OPT, 10% foi geralmente o unico nivel aceitavel; no OPT-6.7B, 25% ainda ficou dentro do limite operacional, mas ja reduziu latencia.
RQ7	As tendencias sinteticas aparecem parcialmente nos OPT: INT8 preserva melhor a qualidade que pruning; pruning degrada com sparsity; o complemento mostrou que pruning denso nao traz economia fisica; em escala maior surgiu speedup apenas em caminhos fisicos, acompanhado de degradacao de perplexidade.

RQ8 INT8 e 2:4 reduziram armazenamento e memoria com caminhos compativeis. No OPT-6.7B, alguns caminhos tambem reduziram latencia de prefill/TTFT, mas nao cumpriram simultaneamente os limites de qualidade.

3.9 Criterios pre-registrados

Uma tendencia sera sustentada quando ocorrer em pelo menos 80% dos casos pareados e o intervalo bootstrap de 95% nao cruzar o efeito nulo. Ela sera mista quando depender de seed, perfil ou modelo, e contradita quando a direcao oposta for predominante e consistente. Ganho fisico relevante exige speedup mediano de pelo menos 1,05, intervalo inteiramente acima de 1,0 e kernel correto confirmado. Aumento de perplexidade de ate 2% para quantizacao e 5% para pruning sao limites operacionais deste estudo, nao regras universais.

3.10 Ordem restante de execucao

1. decidir se a medicao de decode sera reexecutada com ajuste especifico para CUDA Graphs e KV cache;
2. revisar com o orientador a interpretacao do pruning denso e da extensao OPT-6.7B, separando speedup fisico de utilidade com qualidade preservada;
3. preparar uma apresentacao destacando a separacao entre fidelidade, memoria, armazenamento, latencia e perplexidade.

O Docker Desktop 4.63 voltou a operar com backend WSL2 apos a separacao segura dos sockets AF_UNIX residuais. O disco de dados foi realocado para D:\DockerDesktopData; os containers existentes foram preservados e o disco do sistema deixou de receber as camadas grandes. A imagem cientifica foi construida com PyTorch 2.11, CUDA 12.8, TorchAO 0.17, Transformers 5.17, Datasets 5.0.1, Accelerate 1.15 e TensorFlow 2.21. Como a imagem-base adota PEP 668, as dependencias sao instaladas com a autorizacao explicita apropriada a um container imutavel. O pacote auxiliar `spin`, que possuia requisito incompativel e nao participa do experimento, foi removido; `pip check` nao encontrou dependencias quebradas.

O orquestrador possui agora uma acao de estado e uma sequencia exclusiva das etapas pendentes. O mesmo identificador produz diretorios previsiveis e a opcao de retomada e encaminhada aos runners. Antes de probe, smoke, hardware e OPT, cinco amostras precisam confirmar jogo fechado, uso medio da GPU abaixo de 10%, ate 2.048 MiB de VRAM ocupada e temperatura abaixo de 65 graus Celsius. Para a leitura residual do WDDM, processos abaixo de 10%, potencia ate 35 W e clock grafico ate 300 MHz podem confirmar ociosidade, sem relaxar memoria ou temperatura. Etapas completas sao ignoradas apenas quando manifestos, hashes e analises permanecem integros. Nenhum processo e encerrado automaticamente.

powershell

```
.\scripts\run_benchmarks_docker.ps1 -Action Remaining -CacheRoot
↪ "D:\Caches\scientific-initiation\huggingface" -RunId "final-20260913" -Resume
```

powershell

```
.\scripts\run_benchmarks_docker.ps1 -Action Complementary -CacheRoot
↪ "D:\Caches\scientific-initiation\huggingface" -RunId "complementar-20260917"
↪ -Resume
```

Etapa	Estado
Build, probe e bateria fisica	concluido
Testes e smoke OPT final	concluido
OPT-125M, OPT-350M e OPT-1.3B	concluido
OPT-2.7B e OPT-6.7B	concluido
Complemento de lacunas	concluido
Analise, PDF e publicacao	atualizado nesta revisao
Pendente tecnico	decode com KV cache sob torch.compile

3.11 Riscos que permanecem abertos

- pruning denso pode criar zeros sem reduzir armazenamento ou latencia;
- weight-only INT8 prova armazenamento compacto, nao necessariamente computacao inteira;
- kernels otimizados podem depender do shape e tornar a tendencia mista;
- aquecimento, clocks e processos concorrentes podem ampliar a dispersao temporal;
- os modelos OPT responderam de forma diferente dos pesos sinteticos em qualidade absoluta e memoria;
- o decode com KV cache precisa de tratamento proprio quando combinado a CUDA Graphs;
- resultados negativos e casos incompativeis devem permanecer na evidencia final.

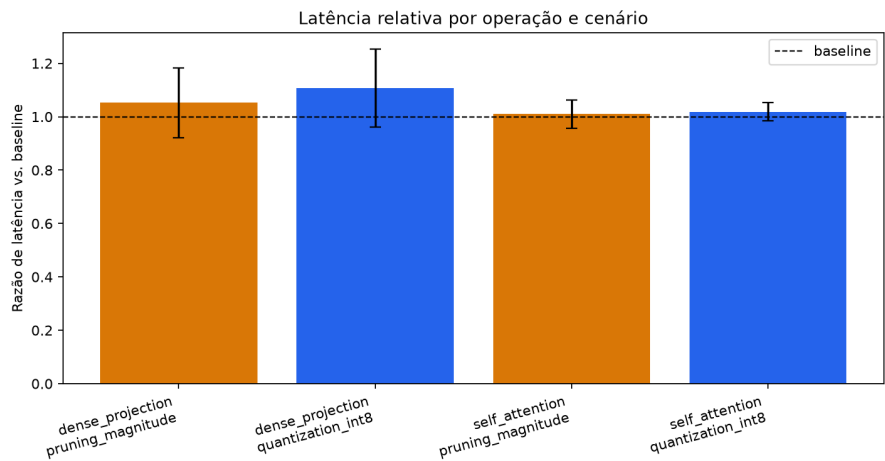


Figura 3.11: Latencia relativa do benchmark GPU v1, preservada como evidencia historica.

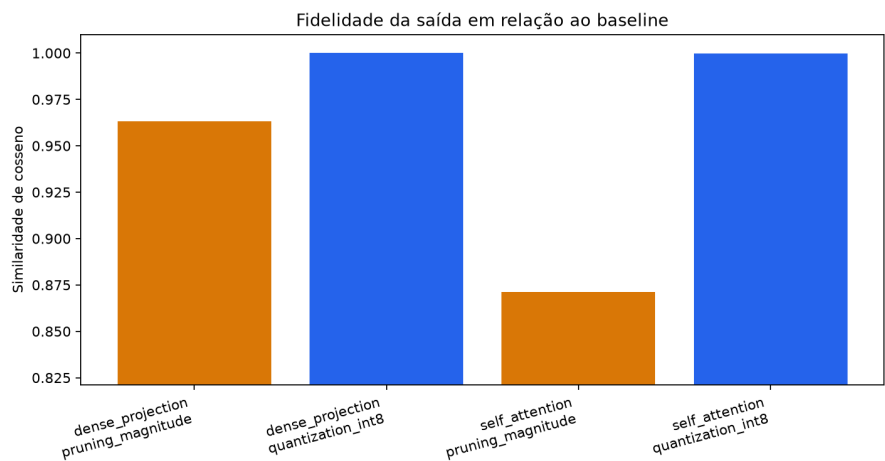


Figura 3.12: Fidelidade numerica do benchmark GPU v1.

Parte II

Fundamentacao e validacao tecnica

Base teorica e validacao cientifica do modelo

4.1 Objetivo do documento

Este documento organiza a primeira fase do trabalho: estabelecer uma base teorica confiavel e um metodo de validacao antes de qualquer benchmark de hardware. A regra de trabalho sera:

Nenhum conceito tecnico deve ser usado no software ou no relatorio sem uma referencia cientifica confiavel, uma definicao operacional e um teste que verifique se a implementacao observada corresponde ao comportamento esperado.

Isso evita um erro comum em experimentos de software: medir desempenho de uma implementacao que usa o nome de uma tecnica, mas nao realiza a tecnica de forma cientificamente adequada. Por exemplo, uma funcao que quantiza e depois volta imediatamente para `float32` pode ser util para estudar erro numerico, mas nao deve ser apresentada como evidencia de ganho real de hardware.

4.2 Principio metodologico

O trabalho deve separar quatro niveis de certeza:

1. Definicao: o termo usado esta definido corretamente pela literatura?
2. Matematica: a formula implementada bate com casos pequenos derivados da definicao?
3. Algoritmo: o software implementa o algoritmo, ou uma aproximacao aceitavel e explicitamente justificada?
4. Hardware: a implementacao realmente usa uma representacao, estrutura de dados, kernel ou caminho de execucao capaz de produzir ganho fisico mensuravel em memoria, latencia ou throughput?

Uma implementacao pode ser valida em um nivel e invalida em outro. Por exemplo, zerar 50% dos pesos de uma matriz e continuar executando uma multiplicacao densa pode ser um pruning conceitual/algoritmico, mas nao prova ganho real de hardware.

4.3 Matriz de rastreabilidade

Todo conceito, ferramenta ou algoritmo usado no projeto deve entrar em uma matriz com os seguintes campos.

Campo	Descricao
Conceito declarado	Nome usado no trabalho, como Transformer, quantizacao ou pruning.
Fonte cientifica	Artigo, livro-texto ou documentacao primaria que define o conceito.
Definicao aceita	Definicao curta usada pelo projeto.
Propriedade esperada no modelo	O que precisa aparecer na implementacao para o nome ser valido.
Teste de verificacao	Como verificar o comportamento esperado.
Resultado observado	Campo a preencher apos executar a verificacao.
Status	Aderente, parcialmente aderente ou nao aderente.
Nivel de validade	Conceitual, numerico, algoritmico ou hardware.

Modelo de linha:

Conceito declarado	Fonte cientifica	Definicao aceita	Propriedade espe- rada no modelo	Teste de verificacao	Resultado obser- vado	Status	Nivel
Transformer	Vaswani et al. (2017)	Arquitetura base- ada em atencao, com tratamento de sequencias e infor- macao posicional.	Entrada sequencial, Q/K/V, atencao escalonada, de- pendencia entre posicoes e informa- cao posicional ou justificativa formal de ausencia.	Comparar a saida de um caso pe- queno com im- plementacao de referencia e audi- tar componentes arquiteturais.	A preencher.	A preencher.	Conceitual/ algoritmico.

4.4 Glossario operacional

4.4.1 Rede neural

- Definicao cientifica: modelo computacional parametrizado composto por unidades ou camadas que aplicam transformacoes lineares e nao lineares para aproximar funcoes a partir de dados.
- Referencia confiavel: Goodfellow, Bengio e Courville, *Deep Learning*, especialmente a discussao de redes feedforward e modelos parametrizados: .
- Explicacao intuitiva: uma rede neural e uma funcao com muitos parametros ajustaveis. Ela transforma uma entrada em uma saida por etapas.
- Uso no trabalho: sera o conceito mais geral por tras dos modelos estudados. Um Transformer e um tipo especifico de arquitetura neural.
- Propriedades exigidas no codigo: parametros numericos, fluxo de entrada para saida, composicao de operacoes e possibilidade de inferencia com parametros fixos.
- Teste de aderencia: verificar se a implementacao possui parametros, executa uma passagem direta deterministica para uma entrada fixa e produz saida com dimensao esperada.

4.4.2 Deep learning

- Definicao cientifica: subarea de aprendizado de maquina baseada em multiplas camadas de representacao, nas quais conceitos mais complexos sao construidos a partir de conceitos mais simples.
- Referencia confiavel: Goodfellow, Bengio e Courville, *Deep Learning*: .
- Explicacao intuitiva: deep learning e o uso de redes neurais com varias etapas de transformacao, permitindo aprender representacoes intermediarias.
- Uso no trabalho: delimita o contexto em que Transformers, compressao e inferencia sao discutidos.
- Propriedades exigidas no codigo: mais de uma transformacao parametrizada ou uma arquitetura explicitamente derivada de modelos profundos.
- Teste de aderencia: inspecionar a arquitetura. Se houver apenas uma multiplicacao linear isolada, o objeto deve ser descrito como uma operacao auxiliar, nao como modelo de deep learning completo.

4.4.3 Transformer

- Definicao cientifica: arquitetura neural baseada em mecanismos de atencao, proposta por Vaswani et al. (2017), dispensando recorrência e convoluções na arquitetura original de sequencia para sequencia.
- Referencia confiavel: Vaswani et al., *Attention Is All You Need*, 2017: .

- Explicacao intuitiva: um Transformer processa uma sequencia permitindo que cada posicao consulte outras posicoes por meio de atencao.
- Uso no trabalho: sera o objeto arquitetural de referencia. O projeto podera estudar uma operacao inspirada em Transformer, um bloco Transformer simplificado ou uma implementacao Transformer completa.
- Propriedades exigidas no codigo: entrada sequencial, projecoes Q/K/V, mecanismo de atencao, mistura de informacao entre posicoes, informacao posicional ou justificativa formal para remove-la, e transformacoes parametrizadas.
- Teste de aderencia: executar checklist arquitetural e comparar um caso pequeno com uma implementacao de referencia. Se faltar componente essencial, classificar corretamente como 'operacao inspirada em Transformer' ou 'bloco Transformer simplificado'.

4.4.4 Self-attention

- Definicao cientifica: mecanismo de atencao no qual consultas, chaves e valores sao derivados da mesma sequencia, permitindo que cada posicao combine informacao de outras posicoes da propria entrada.
- Referencia confiavel: Vaswani et al. (2017): .
- Explicacao intuitiva: cada token 'olha' para os outros tokens da mesma sequencia e decide quais importam mais para sua representacao.
- Uso no trabalho: sera uma das operacoes centrais a validar antes de estudar custo computacional.
- Propriedades exigidas no codigo: Q, K e V calculados da mesma entrada, matriz de atencao com dimensao compativel com o comprimento da sequencia e saida dependente de multiplas posicoes.
- Teste de aderencia: alterar um token nao trivial da sequencia e verificar se a saida de outras posicoes pode mudar. Comparar tambem a saida com uma implementacao direta da formula de atencao.

4.4.5 Scaled dot-product attention

- Definicao cientifica: atencao definida por $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$.
- Referencia confiavel: Vaswani et al. (2017): .
- Explicacao intuitiva: calcula similaridades entre consultas e chaves, normaliza essas similaridades e usa os pesos resultantes para combinar os valores.
- Uso no trabalho: sera a forma padrao para validar a operacao de atencao.
- Propriedades exigidas no codigo: produto QK^T , divisao por $\sqrt{d_k}$, softmax no eixo correto e multiplicacao pelos valores V.
- Teste de aderencia: criar tensores pequenos com valores conhecidos, calcular a saida ma-

nualmente ou por uma implementacao de referencia e exigir diferenca numerica dentro de uma tolerancia definida.

4.4.6 Inferencia

- Definicao cientifica: fase em que um modelo ja definido ou treinado e usado para produzir saidas a partir de entradas, sem atualizar parametros.
- Referencia confiavel: Goodfellow, Bengio e Courville, *Deep Learning*: . Para uso pratico em software, a documentacao do PyTorch tambem distingue execucao de modelos em modo de avaliacao/inferencia.
- Explicacao intuitiva: inferencia e usar o modelo, nao treina-lo.
- Uso no trabalho: o foco sera medir custo e comportamento durante execucao de modelos ou operacoes com pesos fixos.
- Propriedades exigidas no codigo: ausencia de atualizacao de pesos, modo de avaliacao quando aplicavel, desativacao de gradientes quando o objetivo for apenas execucao e entradas controladas.
- Teste de aderencia: executar a mesma entrada duas vezes com os mesmos pesos e semente. Os pesos nao devem mudar e a saida deve ser reproduzivel, salvo operacoes explicitamente nao deterministicas.

4.4.7 Projecao densa

- Definicao cientifica: transformacao linear parametrizada do tipo $Y = XW^T + b$, geralmente aplicada por multiplicacao matricial densa.
- Referencia confiavel: Goodfellow, Bengio e Courville para camadas lineares em redes neurais; Vaswani et al. (2017) para o uso de projecoes aprendidas em Q, K, V e saidas de atencao.
- Explicacao intuitiva: uma matriz de pesos transforma cada vetor de entrada em outro vetor.
- Uso no trabalho: sera um bloco basico para construir Q, K, V e camadas lineares simplificadas.
- Propriedades exigidas no codigo: matriz de pesos densa, dimensoes consistentes e multiplicacao matricial equivalente a transformacao linear.
- Teste de aderencia: comparar a saida da funcao com uma multiplicacao matricial direta em tensores pequenos; verificar dimensoes de entrada, pesos e saida.

4.4.8 KV cache

- Definicao cientifica: armazenamento de chaves e valores ja calculados durante geracao autoregressiva, evitando recomputacao em passos futuros.

- Referencia confiavel: Google Research, **TurboQuant: Redefining AI efficiency with extreme compression**, 2026: . O paper TurboQuant tambem trata quantizacao de vetores e KV cache: .
- Explicacao intuitiva: o modelo guarda informacao intermediaria de tokens anteriores para nao recalculer tudo a cada novo token.
- Uso no trabalho: sera relevante quando o experimento sair de atencao em lote para cenarios autoregressivos ou compressao de memoria.
- Propriedades exigidas no codigo: armazenamento persistente de K e V entre passos de geracao, crescimento com o comprimento do contexto e reutilizacao em novos passos.
- Teste de aderencia: comparar saida com e sem cache para a mesma sequencia. As saidas devem ser numericamente equivalentes dentro de tolerancia, mas a versao com cache deve evitar recomputacao de K/V anteriores.

4.4.9 Pruning

- Definicao cientifica: tecnica de compressao que remove parametros, conexoes, canais ou estruturas consideradas menos relevantes, produzindo esparsidade ou reducao estrutural.
- Referencia confiavel: Tang et al., *A Survey on Transformer Compression*, 2024: . Para implementacao em ferramenta, a documentacao oficial do PyTorch descreve `torch.nn.utils.prune`: .
- Explicacao intuitiva: pruning corta partes do modelo que parecem pouco importantes.
- Uso no trabalho: sera uma tecnica candidata para comparacao com quantizacao.
- Propriedades exigidas no codigo: criterio de importancia, mascara ou remocao estrutural, taxa de poda mensuravel e impacto verificavel na saida.
- Teste de aderencia: medir fracao de pesos/estruturas removidas, comparar saida com baseline e verificar se a execucao usa representacao esparsa ou estrutura reduzida. Se apenas zerar pesos em matriz densa, classificar como pruning conceitual, nao como ganho de hardware.

4.4.10 Sparsity

- Definicao cientifica: grau de esparsidade de uma representacao, normalmente associado a proporcao de elementos nulos ou conexoes ausentes em tensores, matrizes ou estruturas do modelo.
- Referencia confiavel: Tang et al. (2024) no contexto de compressao de Transformers; tutorial oficial de pruning do PyTorch para esparsificacao de redes neurais.
- Explicacao intuitiva: uma estrutura esparsa tem muitos zeros ou muitas conexoes removidas.
- Uso no trabalho: sera uma propriedade esperada de modelos podados.
- Propriedades exigidas no codigo: contagem objetiva de zeros, mascaras ou estruturas

removidas; quando houver alegacao de hardware, uso de formato ou kernel que explore esparsidade.

- Teste de aderencia: calcular $\text{zeros} / \text{total}$ e auditar se a multiplicacao realmente deixou de processar os elementos removidos. Se o custo computacional permanecer denso, a validade sera apenas conceitual ou numerica.

4.4.11 Quantizacao

- Definicao cientifica: tecnica de representacao que mapeia valores de maior precisao para um conjunto menor de niveis discretos, reduzindo bits por valor e introduzindo erro controlado.
- Referencia confiavel: Tang et al. (2024) para quantizacao em compressao de Transformers; Zandieh et al., *TurboQuant*, 2025: ; documentacao torchao para fluxos e tipos de quantizacao em PyTorch: .
- Explicacao intuitiva: quantizar e representar numeros com menos precisao para economizar memoria e potencialmente acelerar computacao.
- Uso no trabalho: sera uma tecnica candidata para reduzir custo de inferencia.
- Propriedades exigidas no codigo: escala ou codigo de quantizacao, valores em menor precisao ou tipo derivado, erro de dequantizacao mensuravel e, para hardware, kernel ou representacao que reduza custo real.
- Teste de aderencia: verificar dtype/bit-width, memoria ocupada, erro em relacao ao baseline e caminho de execucao. Se os tensores forem imediatamente convertidos para float32, classificar como simulacao numerica de quantizacao.

4.4.12 Latencia

- Definicao cientifica: tempo decorrido para completar uma operacao, requisicao ou unidade de trabalho.
- Referencia confiavel: Hennessy e Patterson, **Computer Architecture: A Quantitative Approach**, texto classico de arquitetura de computadores.
- Explicacao intuitiva: latencia e quanto tempo uma execucao demora.
- Uso no trabalho: sera uma metrica de desempenho por operacao ou por passo de inferencia.
- Propriedades exigidas no codigo: medicao com relógio adequado, aquecimento, repeticoes, sincronizacao de GPU quando aplicavel e reporte de media e percentis.
- Teste de aderencia: executar benchmark com entradas fixas, descartar aquecimento, sincronizar antes/depois da medicao em GPU e reportar media, p50 e p95.

4.4.13 Throughput

- Definicao cientifica: quantidade de trabalho completada por unidade de tempo, tambem associada a bandwidth em sistemas computacionais.
- Referencia confiavel: Hennessy e Patterson, *Computer Architecture: A Quantitative Approach*.
- Explicacao intuitiva: throughput mede quanto o sistema processa por segundo.
- Uso no trabalho: podera ser medido como tokens/s, amostras/s ou operacoes/s.
- Propriedades exigidas no codigo: definicao explicita da unidade de trabalho e medicao em janela suficientemente estavel.
- Teste de aderencia: calcular $\text{unidades_processadas} / \text{tempo_total}$ para lote, sequencia ou tokens definidos. Nao misturar throughput com latencia sem indicar a unidade.

4.4.14 Memoria

- Definicao cientifica: recurso de armazenamento usado por parametros, ativacoes, tensores intermediarios, caches e estruturas auxiliares.
- Referencia confiavel: Hennessy e Patterson para memoria em sistemas computacionais; Tang et al. (2024) para reducao de memoria por compressao de Transformers.
- Explicacao intuitiva: memoria e o espaco necessario para guardar modelo, entradas e resultados intermediarios.
- Uso no trabalho: sera uma metrica central para comparar baseline, pruning e quantizacao.
- Propriedades exigidas no codigo: estimativa teorica de bytes e medicao de pico real quando possivel.
- Teste de aderencia: calcular $\text{num_elementos} * \text{bytes_por_elemento}$ para tensores relevantes e comparar com medidores de alocao do framework. Para KV cache, verificar crescimento com o comprimento do contexto.

4.4.15 FLOPs

- Definicao cientifica: contagem de operacoes aritmeticas de ponto flutuante, usada como estimativa analitica de custo computacional.
- Referencia confiavel: Hennessy e Patterson para avaliacao quantitativa de desempenho; Vaswani et al. (2017) para analise de complexidade de atencao e arquiteturas sequenciais.
- Explicacao intuitiva: FLOPs estimam quantas contas numericas o algoritmo faz.
- Uso no trabalho: servira como custo teorico, separado da medicao real de tempo.
- Propriedades exigidas no codigo: formula documentada por operacao e dimensoes usadas no calculo.
- Teste de aderencia: derivar FLOPs esperados para multiplicacoes matriciais e atencao; validar com casos pequenos e manter claro que FLOPs nao equivalem automaticamente

a latencia real.

4.4.16 MSE

- Definicao cientifica: erro quadratico medio, calculado pela media dos quadrados das diferencas entre valores de referencia e valores estimados.
- Referencia confiavel: Hastie, Tibshirani e Friedman, **The Elements of Statistical Learning**: . Para definicao operacional em software, documentacao `mean_squared_error` do `scikit-learn`: .
- Explicacao intuitiva: mede erro medio dando peso maior a erros grandes.
- Uso no trabalho: comparara saidas otimizadas contra a saida baseline.
- Propriedades exigidas no codigo: comparacao ponto a ponto entre saida de referencia e saida candidata, com mesmo shape.
- Teste de aderencia: MSE deve ser zero para vetores identicos e positivo para diferencas. Testar com exemplos pequenos conhecidos.

4.4.17 MAE

- Definicao cientifica: erro absoluto medio, calculado pela media dos valores absolutos das diferencas entre referencia e estimativa.
- Referencia confiavel: Hastie, Tibshirani e Friedman, **The Elements of Statistical Learning**; definicao operacional `mean_absolute_error` do `scikit-learn`: .
- Explicacao intuitiva: mede, em media, o tamanho do erro sem elevar ao quadrado.
- Uso no trabalho: complementara o MSE, por ser menos sensivel a erros extremos.
- Propriedades exigidas no codigo: comparacao ponto a ponto com mesmo shape.
- Teste de aderencia: MAE deve ser zero para vetores identicos e deve bater com calculo manual em exemplos pequenos.

4.4.18 R2

- Definicao cientifica: coeficiente de determinacao, usado para avaliar quanto da variacao da referencia e explicada pela estimativa em relacao a um baseline medio.
- Referencia confiavel: Hastie, Tibshirani e Friedman; definicao operacional `r2_score` do `scikit-learn`: .
- Explicacao intuitiva: indica se a saida candidata acompanha bem a variacao da saida de referencia.
- Uso no trabalho: medira preservacao global do comportamento da saida em relacao ao baseline.
- Propriedades exigidas no codigo: referencia e candidato com mesma dimensao; tratamento explicito de casos constantes.

- Teste de aderencia: R^2 deve ser 1 para previsoes perfeitas, 0 para predicao igual a media em casos nao constantes e pode ser negativo quando o candidato e pior que esse baseline.

4.4.19 Similaridade de cosseno

- Definicao cientifica: similaridade entre vetores calculada como produto interno normalizado pelas normas dos vetores.
- Referencia confiavel: Manning, Raghavan e Schutze, *Introduction to Information Retrieval*, Cambridge University Press, 2008: . Definicao operacional `cosine_similarity` do `scikit-learn`: .
- Explicacao intuitiva: mede se dois vetores apontam em direcao parecida, independentemente da escala.
- Uso no trabalho: avaliara se a direcao da representacao foi preservada apos pruning ou quantizacao.
- Propriedades exigidas no codigo: achatamento ou agregacao definida dos vetores, tratamento de vetores nulos e formula normalizada.
- Teste de aderencia: vetores identicos nao nulos devem ter similaridade 1; vetores ortogonais devem ter similaridade 0; vetores opostos devem tender a -1.

4.5 Criterios para chamar uma implementacao de Transformer

Uma implementacao so deve ser chamada de Transformer se atender aos criterios minimos abaixo ou justificar formalmente qualquer variacao.

Criterio	Verificacao minima
Entrada sequencial	Recebe sequencia de vetores ou embeddings com dimensoes explicitas.
Q/K/V	Projeta a entrada em consultas, chaves e valores.
Atencao escalonada	Implementa $\text{softmax}(QK^T / \sqrt{d_k})V$ ou documenta uma alternativa.
Dependencia entre posicoes	A saida de uma posicao pode depender de outras posicoes da sequencia.

Criterio	Verificacao minima
Informacao posicional	Usa encoding/embedding posicional ou justifica por que o recorte nao precisa dele.
Componentes do bloco	Inclui ou justifica ausencia de residual, normalizacao e feed-forward quando o nome usado for bloco Transformer.
Referencia numerica	Tem saida comparada contra implementacao direta, biblioteca confiavel ou caso derivado manualmente.

Classificacao correta:

- Operacao inspirada em Transformer: quando apenas uma operacao isolada for testada, como projecao densa ou scaled dot-product attention.
- Bloco Transformer simplificado: quando houver atencao, projecoes e parte dos componentes arquiteturais, mas com simplificacoes assumidas.
- Transformer: apenas quando a arquitetura tiver os elementos essenciais definidos pela literatura ou uma justificativa formal para cada diferenca.

4.6 Validacao de ferramentas e algoritmos

Antes de usar uma ferramenta em benchmark, ela deve ser validada por tres perguntas:

1. O que a ferramenta afirma implementar?
2. Qual referencia cientifica ou documentacao primaria sustenta essa afirmacao?
3. O comportamento observado confirma a afirmacao em nivel conceitual, numerico, algoritmico ou de hardware?

Matriz inicial de ferramentas candidatas:

Ferramenta/algoritmo	Conceito declarado	Fonte de referencia	Verificacao necessaria	Classificacao esperada
Implementacao manual de atencao	Scaled dot-product attention	Vaswani et al. (2017)	Comparar com calculo manual e/ou biblioteca de referencia.	Conceitual/algoritmica.
<code>torch.nn.MultiheadAttention</code> OU equivalente	Multi-head attention	Documentacao PyTorch + Vaswani et al.	Verificar shapes, mascaras, projecoes e saida contra referencia.	Algoritmica.
<code>torch.nn.utils.prune</code>	Pruning/esparsificacao	Tutorial oficial PyTorch + Tang et al.	Medir esparsidade, mascara e impacto na saida; verificar se ha ganho real de execucao.	Conceitual/algoritmica; hardware somente se houver kernel/formato adequado.
<code>torchao</code>	Quantizacao	Documentacao <code>torchao</code> + Tang et al.	Verificar dtype, bit-width, escalas, memoria e kernel usado.	Numerica/hardware se usar caminho eficiente.
TurboQuant	Quantizacao vetorial/KV cache	Zandieh et al. (2025), Google Research (2026)	Confirmar se ha implementacao disponivel e se o metodo usado corresponde ao paper.	Algoritmica/hardware apenas apos validacao.

Regras de classificacao:

- Quantizacao valida para hardware exige representacao menor, kernel compativel ou reducao real de memoria. Caso contrario, e simulacao numerica.
- Pruning valido para hardware exige que a esparsidade seja explorada pela execucao. Caso contrario, e pruning conceitual.
- Uma biblioteca so sera aceita quando sua documentacao ou artigo explicar claramente o metodo usado e quando o comportamento observado bater com a definicao esperada.

4.7 Procedimento inicial de trabalho

1. Revisar e completar este glossario com citacoes formatadas no padrao exigido pelo relatorio.
2. Escolher uma implementacao de referencia para atencao/Transformer.
3. Criar testes pequenos e deterministas para Q/K/V, atencao e metricas.
4. Validar ferramentas candidatas de pruning e quantizacao contra a matriz.
5. Somente depois disso, definir benchmarks de hardware.

O protocolo detalhado de passagem entre validacao e benchmark esta em `fases/01_validacao_conceitual/docs/protocolo_experimental.md`.

As perguntas de pesquisa, hipoteses e criterios de conclusao estao em `fases/01_validacao_conceitual/docs/perguntas_pesquisa.md`.

A validacao especifica da rede neural usada como bloco Transformer esta em `fases/01_validacao_conceitual/docs/validacao_transformer.md`.

4.8 Criterios de aceite desta fase

- Todo termo tecnico usado no trabalho possui referencia confiavel.
- Toda definicao possui uma propriedade verificavel no modelo ou software.
- Toda ferramenta futura pode ser avaliada pela matriz conceito-fonte-teste.
- O trabalho consegue dizer, com evidencia, se uma implementacao e valida para estudo conceitual, teste numerico, teste algoritmico ou teste real de hardware.
- Nenhuma conclusao sobre desempenho e apresentada antes da validacao conceitual e algoritmica.

4.9 Referencias iniciais

- Goodfellow, I.; Bengio, Y.; Courville, A. *Deep Learning*. MIT Press, 2016. Disponivel em: .

- Vaswani, A. et al. *Attention Is All You Need*. arXiv:1706.03762, 2017. Disponível em: .
- Tang, Y. et al. *A Survey on Transformer Compression*. arXiv:2402.05964, 2024. Disponível em: .
- Zandieh, A. et al. **TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate**. arXiv:2504.19874, 2025. Disponível em: .
- Google Research. **TurboQuant: Redefining AI efficiency with extreme compression**. 2026. Disponível em: .
- PyTorch. *Pruning Tutorial*. Disponível em: .
- PyTorch. *Quantization Overview - torchao*. Disponível em: .
- Hastie, T.; Tibshirani, R.; Friedman, J. **The Elements of Statistical Learning**. Springer, 2009. Disponível em: .
- Manning, C. D.; Raghavan, P.; Schütze, H. **Introduction to Information Retrieval**. Cambridge University Press, 2008. Disponível em: .
- Hennessy, J. L.; Patterson, D. A. **Computer Architecture: A Quantitative Approach**. Morgan Kaufmann. Referência base para latência, throughput, memória e avaliação quantitativa de desempenho.
- scikit-learn. *Model evaluation and metrics*. Disponível em: .

Validação inicial dos conceitos e scripts do projeto

4.10 Nota metodológica

Os artigos nem sempre dão uma definição formal, no estilo 'X significa Y'. Muitas vezes eles usam o conceito dentro da arquitetura, da fórmula ou do procedimento experimental. Por isso, neste documento eu não tratei as definições como definições de dicionário. Tratei como definições operacionais:

o que este termo significa no nosso estudo -> por que essa interpretação vem da literatura -> qual comportamento precisa aparecer no código -> como isso foi testado.

A estrutura usada em cada item foi:

referência científica -> definição/conceito -> propriedade esperada no código -> script Python -> teste determinístico -> limite da conclusão

4.11 1. Rede neural

- Referência científica: Goodfellow, Bengio e Courville, Deep Learning.
- Definição/conceito: Neste estudo, 'rede neural' significa uma função computacional parametrizada, composta por operações diferenciáveis organizadas em camadas. A justificativa é que o livro trata redes neurais como modelos compostos por camadas e parâmetros

ajustáveis. No nosso caso, isso aparece como camadas lineares, normalização e feed-forward.

- Propriedade esperada no código: O bloco precisa receber um tensor, aplicar camadas com parâmetros e produzir outro tensor como saída.
- Script Python: Foi usada uma lógica de módulo neural com camadas lineares, normalização e feed-forward.
- Teste determinístico: Verificou-se se o bloco executa uma entrada fixa e gera saída com formato esperado. Também foi usada uma rede Linear + ReLU + Linear como controle negativo para mostrar que 'ser NN' não basta para ser Transformer.
- Limite da conclusão: Isso valida que existe uma NN no experimento, mas não valida que ela seja uma Transformer.

4.12 2. Deep learning

- Referência científica: Goodfellow, Bengio e Courville, Deep Learning.
- Definição/conceito: Neste estudo, 'deep learning' não é um resultado que estamos tentando provar. Ele serve como contexto: usamos uma rede com várias transformações encadeadas, e não uma única fórmula isolada. A justificativa vem do uso de múltiplas camadas para compor representações.
- Propriedade esperada no código: Deve haver uma sequência de transformações: projeções, attention, projeção de saída, normalização e feed-forward.
- Script Python: Foi usada uma lógica de bloco neural composto por mais de uma etapa computacional.
- Teste determinístico: Verificou-se se o bloco completo recebe a entrada, atravessa essas etapas e mantém o formato esperado da sequência.
- Limite da conclusão: Isso não valida aprendizado, treinamento ou generalização. Só valida a estrutura computacional usada no recorte.

4.13 3. Transformer

- Referência científica: Vaswani et al., Attention Is All You Need, 2017.
- Definição/conceito: Neste estudo, 'Transformer' é entendido operacionalmente como uma arquitetura para sequências baseada em attention, sem recorrência ou convolução como mecanismo central. A justificativa vem do artigo de Vaswani, que apresenta o Transformer como arquitetura baseada em mecanismos de atenção e descreve blocos com

self-attention, feed-forward, residual, normalização e informação posicional.

- Propriedade esperada no código: Para pertencer à família Transformer, o bloco precisa ter entrada sequencial, Q/K/V, scaled dot-product attention, dependência entre posições, informação posicional e componentes de bloco.
- Script Python: Foi usada uma lógica de auditoria que procura essas propriedades no bloco e classifica o resultado.
- Teste determinístico: O bloco foi testado contra a checklist arquitetural e classificado como bloco Transformer simplificado. Uma NN comum foi rejeitada.
- Limite da conclusão: Isso sustenta dizer que usamos um bloco da família Transformer. Não sustenta dizer que implementamos a Transformer completa do artigo.

4.14 4. Bloco Transformer simplificado

- Referência científica: Vaswani et al., Attention Is All You Need, 2017.
- Definição/conceito: Neste estudo, 'bloco Transformer simplificado' significa um recorte controlado com as partes essenciais de um bloco: self-attention multi-head, projeção de saída, residual, normalização, feed-forward e informação posicional. A justificativa é que o artigo descreve cada camada do encoder como composta por multi-head self-attention e uma rede feed-forward, com residual e layer norm.
- Propriedade esperada no código: O bloco precisa combinar sequência de entrada, Q/K/V, attention, recombinação de heads, projeção de saída, residual, normalização e FFN.
- Script Python: Foi usada uma lógica de bloco único, pequeno e controlado, em vez de uma pilha completa de encoder/decoder.
- Teste determinístico: Verificou-se shape de entrada/saída, presença dos componentes e aprovação na auditoria arquitetural.
- Limite da conclusão: O bloco é adequado para estudar trechos centrais da Transformer. Ele não é a arquitetura completa.

4.15 5. Projeção linear densa

- Referência científica: Goodfellow, Bengio e Courville, Deep Learning; Vaswani et al., 2017.
- Definição/conceito: Neste estudo, projeção linear densa é a operação $Y = XW^T + b$. A justificativa é que Vaswani usa projeções lineares aprendidas para gerar Q, K, V e também para projetar a saída da multi-head attention.

- Propriedade esperada no código: A saída deve ser exatamente o produto matricial da entrada pelos pesos transpostos, com soma de viés quando houver.
- Script Python: Foi usada uma lógica direta de multiplicação matricial e soma opcional de viés.
- Teste determinístico: Comparou-se um caso pequeno com resultado manual conhecido e com cálculo NumPy independente.
- Limite da conclusão: Isso valida a operação usada para projeção. Não valida que os pesos sejam treinados ou semanticamente úteis.

4.16 6. Q, K e V

- Referência científica: Vaswani et al., Attention Is All You Need, 2017.
- Definição/conceito: Neste estudo, Q, K e V significam três projeções lineares da representação de entrada usadas pela attention. A justificativa é que Vaswani descreve queries, keys e values como matrizes projetadas linearmente antes da attention.
- Propriedade esperada no código: A mesma sequência precisa gerar três tensores separados: query, key e value.
- Script Python: Foi usada uma lógica de três projeções lineares independentes aplicadas antes da attention.
- Teste determinístico: A auditoria verificou se existem as três projeções e se o núcleo de attention recebe Q, K e V.
- Limite da conclusão: Isso valida a estrutura Q/K/V do bloco. Não valida encoder-decoder completo.

4.17 7. Scaled dot-product attention

- Referência científica: Vaswani et al., Attention Is All You Need, 2017; documentação PyTorch da `scaled_dot_product_attention`.
- Definição/conceito: Neste estudo, scaled dot-product attention significa exatamente a operação $\text{softmax}(QK^T / \sqrt{d_k})V$. A justificativa é que essa é a fórmula apresentada por Vaswani para a attention escalonada e é a mesma lógica documentada na API de referência do PyTorch.
- Propriedade esperada no código: O código precisa calcular QK^T , dividir por $\sqrt{d_k}$, aplicar softmax e usar esses pesos para combinar V.
- Script Python: Foi usada a lógica literal da fórmula, sem otimização escondida.

- Teste determinístico: Comparou-se a saída com um valor esperado pequeno, com implementação NumPy independente e com a função de referência do PyTorch.
- Limite da conclusão: Isso valida o núcleo matemático da attention. Attention isolada não é Transformer completa.

4.18 8. Self-attention

- Referência científica: Vaswani et al., Attention Is All You Need, 2017.
- Definição/conceito: Neste estudo, self-attention significa attention em que Q, K e V vêm da mesma sequência. A justificativa é que Vaswani descreve self-attention no encoder como o caso em que queries, keys e values vêm da saída da camada anterior do próprio encoder.
- Propriedade esperada no código: A alteração de uma posição da sequência deve poder afetar a saída de outra posição.
- Script Python: Foi usada uma lógica em que a própria sequência alimenta Q, K e V.
- Teste determinístico: Alterou-se um token e verificou-se se outra posição da saída mudava.
- Limite da conclusão: Isso valida dependência entre posições. Não valida qualidade linguística ou desempenho em tarefa real.

4.19 9. Multi-head attention

- Referência científica: Vaswani et al., Attention Is All You Need, 2017.
- Definição/conceito: Neste estudo, multi-head attention significa dividir a representação em múltiplas cabeças, aplicar scaled dot-product attention em paralelo e recombina os resultados. A justificativa é que Vaswani descreve a projeção de Q/K/V em várias heads, seguida de attention paralela, concatenação e nova projeção.
- Propriedade esperada no código: O tensor precisa ser separado em heads, processado por attention e reconstruído no formato original.
- Script Python: Foi usada uma lógica de split de heads, attention por head e recombinação.
- Teste determinístico: Comparou-se o resultado com a attention do PyTorch aplicada em cada head e recombina manualmente.
- Limite da conclusão: Isso valida a mecânica multi-head. Não prova que a quantidade de heads escolhida seja ótima.

4.20 10. Informação posicional

- Referência científica: Vaswani et al., Attention Is All You Need, 2017.
- Definição/conceito: Neste estudo, informação posicional significa um sinal somado à sequência para representar ordem. A justificativa é que Vaswani explica que, como o modelo não usa recorrência nem convolução, precisa injetar informação sobre posição dos tokens.
- Propriedade esperada no código: A entrada sequencial precisa receber informação dependente da posição antes da attention.
- Script Python: Foi usada uma lógica de positional encoding sinusoidal somado à sequência.
- Teste determinístico: A auditoria verificou a existência de informação posicional antes das projeções.
- Limite da conclusão: Isso valida a presença de posição. Não prova que esse encoding seja o melhor possível.

4.21 11. Inferência

- Referência científica: Documentação PyTorch sobre modo de avaliação e reprodutibilidade.
- Definição/conceito: Neste estudo, inferência significa executar o bloco para obter saída sem treinar, sem atualizar pesos e com comportamento determinístico. Essa definição é operacional porque os benchmarks medirão execução, não aprendizado.
- Propriedade esperada no código: A mesma entrada, no mesmo estado do modelo, deve produzir a mesma saída.
- Script Python: Foi usada uma lógica de execução em modo de avaliação e sem atualização de parâmetros.
- Teste determinístico: A mesma entrada foi executada duas vezes e as saídas foram comparadas.
- Limite da conclusão: Isso valida determinismo do núcleo. Não substitui repetição estatística nos benchmarks.

4.22 12. KV cache

- Referência científica: Literatura de inferência autoregressiva em Transformers e otimizações de KV cache.
- Definição/conceito: Neste estudo, KV cache significa armazenar chaves e valores já calculados para não recomputá-los a cada novo token. A definição é operacional porque o que importa para nós é reaproveitamento de K/V e equivalência de saída.
- Propriedade esperada no código: A saída com cache deve bater com a saída sem cache, usando menos recomputação de K/V.
- Script Python: Foi usada uma lógica token a token que acumula K/V anteriores e reutiliza esse cache na attention causal.
- Teste determinístico: Comparou-se a saída com cache contra a saída causal completa e contou-se o reaproveitamento de K/V.
- Limite da conclusão: Isso valida equivalência algorítmica. Não prova ganho de hardware sem medição.

4.23 13. Pruning

- Referência científica: Tang et al., A Survey on Transformer Compression, 2024; documentação PyTorch pruning.
- Definição/conceito: Neste estudo, pruning significa remover ou zerar pesos segundo algum critério, como magnitude. A justificativa é que a literatura de compressão usa pruning como técnica de reduzir parâmetros/conexões, mas o ganho físico só existe se a execução conseguir explorar essa remoção.
- Propriedade esperada no código: Após pruning, deve existir máscara ou aumento verificável de pesos zerados.
- Script Python: Foi usada uma lógica de pruning por magnitude em camada linear pequena.
- Teste determinístico: Mediu-se a quantidade de zeros e verificou-se a máscara gerada.
- Limite da conclusão: Isso valida pruning conceitual/algorítmico. Não valida ganho de hardware se a matriz continuar sendo executada como densa.

4.24 14. Sparsity

- Referência científica: Literatura de pruning e compressão de redes neurais.
- Definição/conceito: Neste estudo, sparsity significa a razão entre elementos iguais a zero

e o total de elementos. A justificativa é que essa é a propriedade observável que indica se o pruning gerou esparsidade.

- Propriedade esperada no código: O código deve contar zeros e dividir pelo número total de elementos.
- Script Python: Foi usada uma lógica direta de contagem de zeros.
- Teste determinístico: Comparou-se a taxa calculada com tensor pequeno de esparsidade conhecida.
- Limite da conclusão: Sparsity mede zeros. Não mede automaticamente redução de tempo ou memória real.

4.25 15. Quantização

- Referência científica: Tang et al., A Survey on Transformer Compression, 2024; TurboQuant; documentação torchao/PyTorch.
- Definição/conceito: Neste estudo, quantização significa representar valores com menor precisão, especialmente int8, mantendo uma escala para aproximar os valores originais. A justificativa é que a literatura de compressão trata quantização como mudança de representação numérica para reduzir custo/armazenamento, com erro medido em relação ao baseline.
- Propriedade esperada no código: Deve aparecer dtype menor, escala de quantização, armazenamento menor ou estimado e erro numérico em relação ao tensor original.
- Script Python: Foi usada uma lógica de quantização simétrica int8, seguida de dequantização apenas para medir erro.
- Teste determinístico: Verificou-se dtype int8, bytes por elemento, erro após dequantização e uma configuração int8 weight-only com torchao.
- Limite da conclusão: Isso valida quantização numérica/algorítmica. Ganho de hardware exige kernel ou caminho de execução compatível.

4.26 16. Latência

- Referência científica: Literatura de desempenho computacional e documentação PyTorch/CUDA.
- Definição/conceito: Neste estudo, latência significa tempo por execução do bloco em um cenário definido. A definição é operacional porque será medida diretamente no benchmark.

- Propriedade esperada no código: A medição precisa usar warmup, repetições e sincronização quando o dispositivo for CUDA.
- Script Python: Foi usada uma lógica de aquecimento, medição repetida e sincronização de GPU.
- Teste determinístico: Verificou-se se o protocolo registra média, p50 e p95 de latência no schema de saída.
- Limite da conclusão: O protocolo está pronto. A conclusão de desempenho só vem após benchmark.

4.27 17. Throughput

- Referência científica: Literatura de desempenho computacional.
- Definição/conceito: Neste estudo, throughput significa tokens processados por segundo. A definição é operacional porque depende diretamente de batch, sequência e tempo medido.
- Propriedade esperada no código: Deve calcular tokens/segundo a partir do número de tokens processados e da latência.
- Script Python: Foi usada uma lógica de $\text{batch_size} * \text{seq_len}$ dividido pelo tempo médio.
- Teste determinístico: Verificou-se se o schema dos resultados exige registro de throughput.
- Limite da conclusão: O cálculo está definido. A comparação depende da execução experimental.

4.28 18. Memória

- Referência científica: Literatura de arquitetura de computadores e documentação PyTorch.
- Definição/conceito: Neste estudo, memória significa armazenamento/alocação associado ao cenário executado. A definição é operacional porque será registrada como pico medido ou estimativa teórica.
- Propriedade esperada no código: Deve haver uma medida ou estimativa comparável entre baseline, pruning e quantização.
- Script Python: Foi usada uma lógica de registrar pico de memória quando disponível e estimar armazenamento dos tensores.

- Teste determinístico: Verificou-se se o schema exige coluna de memória.
- Limite da conclusão: Menor armazenamento teórico não prova automaticamente menor uso real de hardware.

4.29 Conclusão geral

O documento agora separa definição operacional de definição formal. A ideia não é fingir que todo artigo traz uma frase fechada definindo cada termo. A ideia é mostrar que, quando a literatura usa um conceito, nós extraímos dele uma propriedade testável e verificamos essa propriedade no código.

Com isso, considero validado neste momento:

- que o núcleo experimental implementa trechos centrais do Transformer de Vaswani: Q/K/V, scaled dot-product attention, self-attention, multi-head, informação posicional, residual, normalização e FFN;
- que o núcleo deve ser chamado de bloco Transformer simplificado, não de Transformer completa;
- que pruning, sparsity, quantização, latência, throughput e memória foram definidos de forma operacional e testável;
- que ganho real de hardware ainda não foi afirmado, porque isso depende dos benchmarks controlados.

Validacao arquitetural da NN usada no estudo

Este documento registra se a rede neural usada no estudo pode ser chamada de Transformer.

Ele tambem registra a ponte metodologica entre:

```
Vaswani et al. (2017) -> propriedade esperada -> script Python -> teste deterministico
↪ -> limite da conclusao
```

Essa ponte e importante porque o estudo nao deve depender do nome de uma classe ou de uma afirmacao informal. Para dizer que um trecho e de Transformer, precisamos mostrar que ele possui a propriedade descrita no artigo e que essa propriedade foi testada.

5.1 Resultado atual

A implementacao `validacao.transformer.SimplifiedTransformerBlock` esta validada como:

```
bloco_transformer_simplificado
```

Ela pertence a familia Transformer porque passa nos criterios essenciais de um bloco:

- recebe entrada sequencial `[batch, seq_len, d_model]`;
- preserva shape sequencial na saida;
- projeta a entrada em Q, K e V;
- usa atencao escalonada compativel com $\text{softmax}(QK^T / \sqrt{d_k})V$;
- usa multi-head attention;
- mistura informacao entre posicoes;
- usa informacao posicional sinusoidal;
- possui residual, normalizacao e feed-forward;
- e deterministica em modo de avaliacao.

Ela nao deve ser chamada de 'Transformer completo' no artigo, porque o recorte experimental usa um bloco controlado, nao uma arquitetura completa com pilha de camadas, cabeca de tarefa, treinamento e protocolo de modelo final.

5.2 Criterio de rastreabilidade

Cada componente aceito no estudo precisa passar por cinco perguntas:

1. O componente existe no Transformer de Vaswani et al.?
2. Qual propriedade observavel esse componente deve ter?
3. Onde essa propriedade aparece no nosso codigo?
4. Qual teste deterministico verifica essa propriedade?
5. O que esse teste ainda nao permite concluir?

Assim, a validacao nao diz apenas 'o codigo roda'. Ela diz 'esta operacao corresponde a este trecho definido pela literatura, dentro destes limites'.

5.3 Matriz Vaswani -> codigo -> teste

Trecho no Transformer de Vaswani	Propriedade esperada	Onde esta no codigo	Teste deterministico	Comparacao usada	O que o teste prova	Limite metodologico
Entrada sequencial	O modulo recebe tensores organizados como sequencia, com dimensao de lote, posicao e representacao.	SimplifiedTransformer Block.forward, em fases/01_validacao_conceitual/validacao/transformer.py.	test_simplified_transformer_block_has_expected_components_and_shape.	Shape esperado [batch, seq_len, d_model].	O bloco recebe e devolve uma representacao sequencial.	Nao prova tokenizacao, embedding real ou tarefa de linguagem.
Projecoes lineares densas	Entradas sao transformadas por matrizes aprendidas; no attention original essas projecoes geram Q, K, V e saida.	q_proj, k_proj, v_proj, out_proj em fases/01_validacao_conceitual/validacao/transformer.py; dense_projection_torch em fases/01_validacao_conceitual/validacao/dense.py.	test_dense_projection_matches_manual_definition; auditoria de Q/K/V.	Formula $Y = XW^T + b$ calculada manualmente.	A operacao linear usada no bloco segue a definicao matematica esperada.	Nao prova que os pesos treinados sao bons; os pesos do bloco controlado sao fixos/iniciais.
Projecoes Q, K e V	A mesma sequencia gera consultas, chaves e valores para self-attention.	project_qkv em fases/01_validacao_conceitual/validacao/transformer.py.	test_transformer_audit_validates_current_nn_as_simplified_transformer_block.	Inspecao estrutural: existencia de q_proj, k_proj, v_proj.	O bloco tem as tres projecoes exigidas para attention de Transformer.	Nao prova arquitetura completa encoder-decoder.
Scaled dot-product attention	O nucleo de atencao deve calcular $\text{softmax}(QK^T / \sqrt{d_k})V$.	scaled_dot_product_attention_torch e scaled_dot_product_attention_numpy em fases/01_validacao_conceitual/validacao/attention.py; adaptador em validacao/comparacao_frameworks.py.	Testes de referencia NumPy/PyTorch e test_all_frameworks_match_in_every_case.	Valor manual pequeno, NumPy independente, torch.nn.functional.scaled_dot_product_attention e keras.ops.nn.dot_product_attention com backend TensorFlow.	O algoritmo bate com a formula e com APIs publicas de dois frameworks nos mesmos tensores.	Nao prova por si so que existe um bloco completo nem compara desempenho; prova o nucleo de attention.
Mascara de atencao	A atencao pode receber restricoes aditivas, como ocorre em variantes mascaradas.	Parametro additive_attention_mask em fases/01_validacao_conceitual/validacao/attention.py.	test_scaled_dot_product_attention_additive_mask_matches_pytorch_reference.	torch.nn.functional.scaled_dot_product_attention com attn_mask.	A mascara usada no nosso nucleo e compativel com a referencia do PyTorch.	O bloco experimental padrao nao usa mascara causal por padrao.

Trecho no Transformer de Vaswani	Propriedade esperada	Onde esta no codigo	Teste deterministico	Comparacao usada	O que o teste prova	Limite metodologico
Dependencia entre posicoes	A saida de uma posicao pode depender de outra posicao da sequencia.	Self-attention em fases/01_validacao_conceitual/validacao/attention.py e auditoria em fases/01_validacao_conceitual/validacao/auditoria_transformer.py.	test_attention_output_depends_on_other_sequence_positions; test_transformer_audit_validates_current_nn_as_simplified_transformer_block.	Alteracao controlada de um token e observacao de mudanca em outra posicao.	A operacao nao trata cada posicao de forma isolada; ha mistura de informacao sequencial.	Nao mede qualidade semantica dessa dependencia em uma tarefa real.
Multi-head attention	A representacao e dividida em cabecas, cada cabeca aplica attention, e depois as cabecas sao recombinadas.	split_heads, combine_heads e multi_head_attention_from_qkv em fases/01_validacao_conceitual/validacao/attention.py.	test_multi_head_attention_preserves_shape_and_matches_split_reference.	Aplicacao da SDPA do PyTorch em cada head e recombinacao manual.	A mecanica de separar/aplicar/recombinar heads esta correta.	Nao prova que o numero de heads escolhido e otimo.
Informacao posicional	Como attention pura nao conhece ordem por si so, o bloco precisa receber algum sinal de posicao.	sinusoidal_position_encoding e add_positional_information em fases/01_validacao_conceitual/validacao/transformer.py.	Auditoria: report.has_positional_information_or_justification.	Criterio arquitetural derivado de Vaswani et al.	O bloco adiciona informacao posicional antes das projecoes.	Nao prova que essa codificacao e a melhor para todas as tarefas.
Residual, normalizacao e FFN	O bloco Transformer usa subcamadas com conexoes residuais, normalizacao e rede feed-forward.	norm1, norm2, ffn e out_proj em fases/01_validacao_conceitual/validacao/transformer.py.	test_simplified_transformer_block_has_expected_components_and_shape; auditoria estrutural.	Checklist arquitetural do bloco.	O bloco tem os componentes essenciais para ser classificado como bloco Transformer simplificado.	Ainda falta pilha completa de camadas e cabeca de tarefa para chamar de Transformer completo.
Inferencia deterministica	Em avaliacao, a mesma entrada deve produzir a mesma saida, evitando ruido experimental.	audit_transformer_module em fases/01_validacao_conceitual/validacao/auditoria_transformer.py.	test_transformer_audit_validates_current_nn_as_simplified_transformer_block.	Duas execucoes em eval() com a mesma entrada.	O nucleo pode ser usado em benchmarks controlados.	Nao substitui avaliacao estatistica de desempenho em varias repeticoes.
Controle negativo	Uma NN comum nao deve ser aceita como Transformer so por ser rede neural.	Auditoria aplicada a torch.nn.Sequential(Linear, ReLU, Linear).	test_transformer_audit_rejects_plain_neural_network_as_transformer.	Ausencia de Q/K/V e de scaled attention.	O validador distingue uma NN generica de um bloco da familia Transformer.	Nao classifica todos os tipos possiveis de arquitetura; classifica o recorte do estudo.

5.4 O que esta comprovado

Os testes comprovam que os scripts Python implementam deterministicamente os trechos centrais escolhidos do Transformer de Vaswani et al. para o escopo do projeto:

- projecao linear densa;
- geracao de Q, K e V;
- scaled dot-product attention;
- mascara aditiva de attention;
- dependencia entre posicoes da sequencia;
- multi-head attention;
- informacao posicional sinusoidal;
- subcamadas residual, normalizacao e FFN;
- comportamento deterministico em inferencia;
- rejeicao de uma NN comum como controle negativo.

Portanto, o objeto experimental atual e valido para estudar:

operacoes centrais de Transformer

e tambem:

bloco Transformer simplificado

5.5 O que nao esta comprovado

Os testes nao comprovam que temos a arquitetura completa do artigo *Attention Is All You Need*. Para isso, seria necessario incluir e validar elementos fora do recorte atual, como:

- pilha completa de blocos;
- organizacao completa de encoder e/ou decoder;
- embeddings de entrada e saida;
- cabeca de tarefa;
- treinamento ou carregamento de pesos de um modelo real;
- avaliacao em tarefa ou dataset.

Por conta disso, a frase metodologicamente correta e:

O estudo avalia trechos computacionalmente centrais de um bloco Transformer simplificado, derivados da arquitetura de Vaswani et al. (2017), com foco em inferencia controlada.

E a frase que deve ser evitada neste momento e:

O estudo avalia uma Transformer completa.

5.6 Como a verificacao e feita

A auditoria esta implementada em `validacao.auditoria_transformer`. Ela executa testes comportamentais e estruturais:

1. passa uma sequencia pelo modulo;
2. verifica se a saida preserva `[batch, seq_len, d_model]`;
3. inspeciona Q/K/V, output projection, norm e FFN;
4. compara a atencao interna com `torch.nn.functional.scaled_dot_product_attention`;
5. altera um token e verifica se outra posicao pode mudar;
6. roda a mesma entrada duas vezes em `eval()` para testar determinismo;
7. aplica a classificacao `nao_aderente`, `operacao_inspirada_em_transformer`, `bloco_transformer_simplificado` ou `transformer`.

5.7 Controle negativo

A mesma auditoria rejeita uma rede neural comum composta por camadas lineares e ReLU. Esse controle e importante: ele mostra que o teste nao aceita qualquer NN como Transformer.

5.8 Regra para o texto final

No artigo, usar:

- 'bloco Transformer simplificado' para o nucleo experimental atual;
- 'operacao inspirada em Transformer' quando o teste isolar attention, projecao densa ou KV cache;
- 'Transformer' apenas se houver arquitetura completa ou justificativa formal para todos os componentes ausentes.

Explicação narrativa da metodologia de validação

Este documento explica, em linguagem de apresentação de projeto de pesquisa, como estamos ligando os conceitos científicos ao código. A ideia é construir uma história: primeiro entendemos o que cada conceito significa na literatura; então transformamos essa definição em uma propriedade observável; por conta disso criamos uma implementação pequena; e, por fim, testamos se o resultado bate com o comportamento esperado.

O ponto central é simples: antes de medir desempenho, precisamos saber se aquilo que estamos medindo é realmente o que dizemos que é. Se chamamos uma operação de Transformer, ela precisa ter propriedades de Transformer. Se chamamos uma técnica de quantização, ela precisa representar valores com menos precisão. Se chamamos pruning de ganho de hardware, ele precisa usar algum formato ou kernel que realmente explore esparsidade.

Então, a metodologia do projeto segue esta cadeia:

```
conceito científico
-> referencia confiável
-> definição operacional
-> implementação pequena e transparente
-> teste determinístico
-> classificação do nível de validade
```

Os quatro níveis de validade usados no projeto são:

Nível	O que responde	Exemplo
Definição	O termo está correto de acordo com a literatura?	Transformer como arquitetura baseada em atenção.
Matemática	A fórmula local bate com um caso pequeno calculado?	$\text{softmax}(QK^T / \sqrt{d_k})V$.
Algoritmo	A ferramenta implementa o comportamento esperado?	PyTorch SDPA bate com nossa attention manual.

Nivel	O que responde	Exemplo
Hardware	Ha evidencia de ganho fisico real?	Kernel, dtype ou formato que reduza memoria/latencia.

Por conta disso, varias conclusoes do projeto ainda sao propositalmente cautelosas. Por exemplo, uma quantizacao pode estar numericamente correta, mas isso ainda nao quer dizer que ela gerou ganho real de hardware. Do mesmo modo, uma poda pode zerar pesos corretamente, mas se a execucao continuar densa, a conclusao e conceitual/algoritmica, nao de hardware.

6.1 Capitulo 1 - Rede neural e deep learning

6.1.1 Ideia inicial

Comecamos por rede neural porque Transformer nao aparece do nada. Um Transformer e um tipo especifico de rede neural. Entao, antes de perguntar se uma NN e Transformer, precisamos perguntar se ela pelo menos tem as caracteristicas basicas de uma rede neural: parametros, entrada, transformacoes e saida.

Deep learning entra logo depois porque, em geral, estamos falando de redes com varias etapas de transformacao. Cada camada transforma a representacao anterior em outra representacao. Entao, quando falamos de Transformer, estamos falando de uma arquitetura de deep learning que processa sequencias.

6.1.2 Definicao cientifica

A referencia principal para rede neural e deep learning e:

- Goodfellow, Bengio e Courville, *Deep Learning*, MIT Press, 2016: .

No nosso uso, uma rede neural e um modelo parametrizado que aplica operacoes lineares e nao lineares sobre uma entrada para produzir uma saida. Isso importa porque o nosso bloco usa pesos treinaveis, como `torch.nn.Linear`, e operacoes nao lineares, como `ReLU`.

6.1.3 Por que isso importa no nosso trabalho

Se a gente nao separa 'rede neural' de 'Transformer', corremos o risco de chamar qualquer NN de Transformer. Entao, primeiro aceitamos que o bloco atual e uma rede neural. Depois, criamos testes adicionais para saber se ela pertence a familia Transformer.

6.1.4 Como implementamos

O nucleo atual esta em `validacao.transformer.SimplifiedTransformerBlock`. Dentro dele ha camadas lineares, normalizacao e FFN:

```
python
self.q_proj = torch.nn.Linear(d_model, d_model)
self.k_proj = torch.nn.Linear(d_model, d_model)
self.v_proj = torch.nn.Linear(d_model, d_model)
self.out_proj = torch.nn.Linear(d_model, d_model)
self.norm1 = torch.nn.LayerNorm(d_model)
self.ffn = torch.nn.Sequential(
    torch.nn.Linear(d_model, ffn_hidden_dim),
    torch.nn.ReLU(),
    torch.nn.Linear(ffn_hidden_dim, d_model),
)
self.norm2 = torch.nn.LayerNorm(d_model)
```

6.1.5 Como testamos

Testamos se a rede executa com entrada sequencial e preserva shape:

```
python
block = Simplified\allowbreak{}Transformer\allowbreak{}Block(d_model=4, num_heads=2,
↪ ffn_hidden_dim=8).eval()
sequence = torch.randn(2, 3, 4)
output = block(sequence)

assert output.shape == sequence.shape
```

6.1.6 Com o que comparamos

Aqui a comparacao ainda nao e com um paper de resultado. A comparacao e com a definicao operacional de uma rede neural: existe entrada, existem parametros, existe composicao de camadas e existe saida.

6.1.7 O que podemos concluir

Podemos concluir que o objeto atual e uma rede neural profunda pequena e controlada.

6.1.8 O que ainda nao podemos concluir

Ainda nao podemos concluir que ela e Transformer so por ser uma NN. Por conta disso, vem o proximo passo: validar os criterios especificos de Transformer.

6.2 Capitulo 2 - Transformer

6.2.1 Ideia inicial

Um Transformer processa sequencias usando atencao. A ideia intuitiva e: cada posicao da sequencia pode consultar outras posicoes e decidir quais informacoes devem influenciar sua representacao.

Entao, se o nosso estudo fala em eficiencia de operacoes de Transformers, a NN usada precisa ter pelo menos as pecas centrais dessa familia: entrada sequencial, Q/K/V, attention, dependencia entre posicoes, informacao posicional, residual, normalizacao e feed-forward.

6.2.2 Definicao cientifica

A referencia principal e:

- Vaswani et al., *Attention Is All You Need*, 2017: .

Esse trabalho define a arquitetura Transformer como uma arquitetura baseada em mecanismos de atencao. A formula central da atencao escalonada usada no paper e:

```
Attention(Q, K, V) = softmax(QK^T / sqrt(d_k))V
```

6.2.3 Por que isso importa no nosso trabalho

Isso importa porque o nome 'Transformer' carrega uma afirmacao cientifica. Se usarmos uma NN comum e chamarmos de Transformer, o estudo perde validade. Entao, em vez de confiar no nome da classe, criamos uma auditoria arquitetural.

6.2.4 Como implementamos

A classe atual e um bloco pequeno:

```
class Simplified\allowbreak{ }Transformer\allowbreak{ }Block(torch.nn.Module):  
    """Bloco pequeno para validacao: MHA + residual/norm + FFN."""
```

Ela nao se declara como Transformer completo. Ela se declara como bloco simplificado. Isso e importante porque a implementacao nao tem uma pilha completa de camadas, cabeca de tarefa e treinamento final.

6.2.5 Como testamos

Usamos `validacao.auditoria_transformer.audit_transformer_module`:

python

```
report = audit_transformer_module(block, d_model=4, seq_len=3)

assert report.architecture_label == "bloco_transformer_simplificado"
assert report.is_transformer_family
assert not report.can_be_called_transformer
assert report.can_be_called_simplified_block
```

6.2.6 Com o que comparamos

Comparamos com uma lista de critérios derivados do Transformer original:

- entrada sequencial;
- projeções Q/K/V;
- atenção escalonada;
- dependência entre posições;
- informação posicional;
- residual, normalização e FFN;
- saída determinística em inferência.

Também comparamos contra um controle negativo:

python

```
plain_nn = torch.nn.Sequential(
    torch.nn.Linear(4, 4),
    torch.nn.ReLU(),
    torch.nn.Linear(4, 4),
).eval()

report = audit_transformer_module(plain_nn, d_model=4, seq_len=3)

assert report.architecture_label == "nao_aderente"
```

6.2.7 O que podemos concluir

Podemos concluir que a NN atual pertence à família Transformer e deve ser chamada de:

text

```
bloco Transformer simplificado
```

6.2.8 O que ainda nao podemos concluir

Nao podemos chamar a implementacao atual de Transformer completo. Entao, no texto do artigo, a formulacao correta e: 'avaliamos um bloco Transformer simplificado controlado'.

6.2.9 Ponte direta entre Vaswani e os nossos scripts

Entao, para nao ficar uma validacao apenas verbal, criamos uma ponte de rastreabilidade. A logica e simples: primeiro olhamos para o artigo de Vaswani, depois extraimos uma propriedade que pode aparecer no codigo, depois apontamos o script Python que implementa essa propriedade, e por fim apontamos o teste que confere se o comportamento bate com o esperado.

Essa ponte ficou documentada de forma completa em `fases/01_validacao_conceitual/docs/validacao_transformer.md`. A ideia central dela e:

```
Vaswani et al. (2017)
-> propriedade esperada
-> script Python
-> teste deterministico
-> limite da conclusao
```

Por exemplo, Vaswani define a attention escalonada como:

```
Attention(Q, K, V) = softmax(QK^T / sqrt(d_k))V
```

Entao, por conta disso, o nosso script `fases/01_validacao_conceitual/validacao/attention.py` precisa fazer essa conta explicitamente:

```
scores = torch.matmul(query, key.transpose(-1, -2)) / math.sqrt(d_k)
weights = torch.softmax(scores, dim=-1)
return torch.matmul(weights, value)
```

Mas ainda nao basta o codigo 'parecer' correto. Por conta disso, o teste `test_scaled_dot_product_attention_matches_independent_numpy_reference` compara a saida com um caso pequeno calculado de forma independente, e o teste `test_scaled_dot_product_attention_matches_pytorch_reference` compara com `torch.nn.functional.scaled_dot_product_attention`.

Entao, para esse trecho especifico, podemos dizer:

```
A nossa scaled dot-product attention bate com a formula de Vaswani e com uma
implementacao de referencia do PyTorch.
```

Agora, isso ainda nao autoriza dizer que temos uma Transformer completa. Isso autoriza dizer que temos o trecho de attention validado. Por isso fazemos a mesma coisa para as

outras pecas:

Trecho do artigo	O que exigimos no codigo	Como testamos
Entrada sequencial	Tensor [batch, seq_len, d_model].	O bloco recebe a sequencia e preserva o shape.
Projecoes lineares	Operacao $Y = XW^T + b$.	Comparacao com valor manual conhecido.
Q, K e V	Tres projecoes separadas da sequencia.	Auditoria verifica q_proj, k_proj e v_proj.
Scaled attention	$\text{softmax}(QK^T / \sqrt{d_k})V$.	Comparacao com NumPy, valor manual e PyTorch.
Multi-head	Separar heads, aplicar attention e recombina.	Comparacao com SDPA aplicada por head.
Dependencia sequencial	Mudar um token pode mudar outra posicao.	Teste altera uma posicao e observa outra.
Posicao	Algum sinal posicional antes da attention.	Auditoria verifica positional encoding.
Residual/norm/FFN	Componentes de bloco Transformer.	Auditoria estrutural do bloco.
Controle negativo	NN comum deve ser rejeitada.	Linear + ReLU + Linear recebe nao_aderente.

Essa tabela e importante para a apresentacao porque ela mostra que nao estamos dizendo 'e Transformer porque eu chamei de Transformer'. Estamos dizendo:

Este trecho pertence a familia Transformer porque a propriedade X aparece no artigo, aparece no codigo, e passa no teste Y.

Com isso, a conclusao fica calibrada. Podemos afirmar que os scripts atuais validam trechos centrais de Transformer e um bloco Transformer simplificado. Ainda nao podemos afirmar que reproduzimos uma Transformer completa, porque a arquitetura completa exigiria pilha de blocos, encoder/decoder ou pelo menos um encoder completo, embeddings, cabeca de tarefa e validacao em tarefa real.

6.3 Capitulo 3 - Bloco Transformer simplificado

6.3.1 Ideia inicial

Depois de entender Transformer, a pergunta vira: qual parte do Transformer vamos estudar? Como o projeto ainda esta em fase controlada, usamos um bloco. Isso permite entender causa e efeito sem misturar muitas variaveis.

Entao, em vez de medir um modelo enorme logo de inicio, estudamos um bloco pequeno onde conseguimos auditar cada parte.

6.3.2 Definicao cientifica

O bloco se inspira no Transformer de Vaswani et al. Ele tem attention, projecoes, residual, normalizacao e FFN. Como e uma simplificacao, ele nao e uma arquitetura completa.

6.3.3 Por que isso importa no nosso trabalho

Isso importa porque a validade cientifica vem do controle. Se usarmos um modelo grande sem entender o caminho interno, talvez vejamos uma mudanca de latencia sem saber se veio da attention, da quantizacao, do kernel, da memoria ou de alguma otimizacao escondida.

6.3.4 Como implementamos

O forward do bloco mostra a historia inteira:

```
python
hidden = self.add_positional_information(sequence)
query, key, value = self.project_qkv(hidden)
attention = self.attention_core(query, key, value)
attention = combine_heads(attention)

hidden = self.norm1(hidden + self.out_proj(attention))
return self.norm2(hidden + self.ffn(hidden))
```

Entao, o fluxo e:

```
text
sequencia
-> informacao posicional
-> Q/K/V
-> attention
-> recombinação das heads
-> residual + norm
-> FFN
-> residual + norm
```

6.3.5 Como testamos

O teste confere componentes e shape:

python

```
assert output.shape == sequence.shape
assert hasattr(block, "q_proj")
assert hasattr(block, "k_proj")
assert hasattr(block, "v_proj")
assert hasattr(block, "norm1")
assert hasattr(block, "ffn")
```

E a auditoria confere o comportamento:

python

```
assert report.has_qkv_projection
assert report.has_scaled_attention
assert report.has_cross_position_dependency
assert report.has_positional_information_or_justification
assert report.has_residual_norm_ffn_or_justification
```

6.3.6 Com o que comparamos

Comparamos com os criterios arquiteturais do Transformer e com a API de referencia do PyTorch para a attention interna.

6.3.7 O que podemos concluir

Podemos concluir que o bloco e adequado para o recorte experimental.

6.3.8 O que ainda nao podemos concluir

Ainda nao podemos concluir desempenho final de um Transformer real em producao, porque isso exigiria arquitetura completa, tarefa, dataset e benchmark maior.

6.4 Capitulo 4 - Self-attention

6.4.1 Ideia inicial

Self-attention e a ideia de uma sequencia olhar para ela mesma. Entao, cada posicao calcula o quanto deve considerar outras posicoes.

Isso importa porque, sem dependencia entre posicoes, uma operacao nao esta capturando o aspecto sequencial que torna Transformers interessantes.

6.4.2 Definicao cientifica

A referencia e Vaswani et al. (2017). No nosso recorte, self-attention significa que Q, K e V sao derivados da mesma sequencia.

6.4.3 Por que isso importa no nosso trabalho

Se mudamos um token e nenhuma outra posicao muda, entao a operacao nao esta misturando informacao da sequencia. Por conta disso, testamos explicitamente se alterar uma posicao pode alterar outra.

6.4.4 Como implementamos

A attention central usa Q, K e V:

```
python
scores = torch.matmul(query, key.transpose(-1, -2)) / math.sqrt(d_k)
weights = torch.softmax(scores, dim=-1)
return torch.matmul(weights, value)
```

6.4.5 Como testamos

O teste cria duas versoes de value, alterando a segunda posicao:

```
python
query = torch.zeros(1, 2, 2)
key = torch.zeros(1, 2, 2)
value_a = torch.tensor([[[1.0, 1.0], [3.0, 3.0]]])
value_b = torch.tensor([[[1.0, 1.0], [9.0, 9.0]]])

output_a = scaled_dot_product_attention_torch(query, key, value_a)
output_b = scaled_dot_product_attention_torch(query, key, value_b)

assert not torch.allclose(output_a[:, 0, :], output_b[:, 0, :])
```

6.4.6 Com o que comparamos

Comparamos com a propriedade esperada de self-attention: uma posicao pode depender de outra posicao da mesma sequencia.

6.4.7 O que podemos concluir

Podemos concluir que a operacao permite dependencia entre posicoes.

6.4.8 O que ainda nao podemos concluir

Isso nao prova qualidade de modelagem, nem desempenho, nem acuracia em tarefa. Prova apenas a propriedade conceitual necessaria.

6.5 Capitulo 5 - Scaled dot-product attention

6.5.1 Ideia inicial

Scaled dot-product attention e a conta central. Ela calcula similaridade entre queries e keys, normaliza essas similaridades e usa o resultado para combinar values.

Entao, esse e um dos testes mais importantes: se essa formula estiver errada, o resto do estudo perde base.

6.5.2 Definicao cientifica

A formula vem de Vaswani et al.:

text

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

6.5.3 Por que isso importa no nosso trabalho

Essa formula e o nucleo matematico da attention usada no bloco. Por conta disso, nao basta confiar no PyTorch. Implementamos uma versao manual e comparamos com uma referencia independente.

6.5.4 Como implementamos

Em PyTorch:

python

```
d_k = query.shape[-1]
scores = torch.matmul(query, key.transpose(-1, -2)) / math.sqrt(d_k)
weights = torch.softmax(scores, dim=-1)
return torch.matmul(weights, value)
```

Em NumPy, a mesma ideia:

python

```
scores = np.matmul(query, np.swapaxes(key, -1, -2)) / math.sqrt(d_k)
weights = softmax_numpy(scores, axis=-1)
return np.matmul(weights, value)
```

6.5.5 Como testamos

Usamos tensores pequenos:

```
python
query = torch.tensor([[[[1.0, 0.0], [0.0, 1.0]]]])
key = torch.tensor([[[[1.0, 0.0], [0.0, 1.0]]]])
value = torch.tensor([[[[1.0, 2.0], [3.0, 4.0]]]])
```

E exigimos o resultado esperado pela formula:

```
python
expected_from_formula = torch.tensor(
    [[1.6604769, 2.6604769], [2.3395231, 3.3395231]])
)

assert torch.allclose(actual, expected_from_formula, atol=1e-6)
```

6.5.6 Com o que comparamos

Comparamos com:

1. calculo NumPy independente;
2. resultado numerico derivado da formula;
3. torch.nn.functional.scaled_dot_product_attention;
4. keras.ops.nn.dot_product_attention com backend TensorFlow, usando os mesmos Q, K, V e mascara.

6.5.7 O que podemos concluir

Podemos concluir que a attention manual e sua composicao multi-head com projecoes Q/K/V/O estao matematicamente aderentes ao conceito e numericamente compativeis com APIs publicas de dois frameworks. A execucao v2 aprovou 18/18 comparacoes, com erro absoluto maximo de 4.76837158e-07 para atol=1e-6 e rtol=1e-5.

6.5.8 O que ainda nao podemos concluir

Attention correta nao significa Transformer completo. Ela e uma parte necessaria, mas nao suficiente.

6.6 Capitulo 6 - Multi-head attention

6.6.1 Ideia inicial

Multi-head attention divide a representacao em varias heads. Entao, em vez de uma unica attention olhar para tudo de uma forma, varias attentions menores processam partes da representacao.

6.6.2 Definicao cientifica

A referencia continua sendo Vaswani et al. (2017). A arquitetura Transformer usa varias heads para projetar, aplicar attention e recombinar informacoes.

6.6.3 Por que isso importa no nosso trabalho

Se o bloco usa multi-head attention, precisamos saber se o split e o combine estao corretos. Por conta disso, validamos shapes e comparamos com a referencia do PyTorch.

6.6.4 Como implementamos

O split:

python

```
return tensor.reshape(batch_size, seq_len, num_heads, head_dim).transpose(1, 2)
```

O combine:

python

```
return tensor.transpose(1, 2).contiguous().reshape(
    batch_size,
    seq_len,
    num_heads * head_dim,
)
```

E a attention multi-head transparente:

python

```
query_heads = split_heads(query, num_heads)
key_heads = split_heads(key, num_heads)
value_heads = split_heads(value, num_heads)
attention_heads = scaled_dot_product_attention_torch(query_heads, key_heads,
    ↪ value_heads)
return combine_heads(attention_heads)
```

6.6.5 Como testamos

Testamos shape e equivalencia:

python

```
sequence = torch.randn(2, 3, 4)
```

```
num_heads = 2

actual = multi_head_attention_from_qkv(
    sequence,
    sequence,
    sequence,
    num_heads=num_heads,
)

assert actual.shape == sequence.shape
```

6.6.6 Com o que comparamos

Comparamos com `torch.nn.functional.scaled_dot_product_attention` aplicada nas heads ja separadas.

6.6.7 O que podemos concluir

Podemos concluir que a mecanica multi-head esta correta no caso pequeno.

6.6.8 O que ainda nao podemos concluir

Nao podemos concluir que o numero de heads escolhido e otimo. Isso seria uma pergunta experimental diferente.

6.7 Capitulo 7 - Projecao densa

6.7.1 Ideia inicial

Projecao densa e uma camada linear. Ela pega um vetor e transforma em outro por meio de uma matriz de pesos. Entao, antes de Q, K e V existirem, precisamos projetar a entrada.

6.7.2 Definicao cientifica

A operacao e:

$$Y = XW^T + b$$

Ela aparece em redes neurais em Goodfellow et al. e no Transformer de Vaswani et al., onde projecoes aprendidas geram Q, K, V e saidas de attention.

6.7.3 Por que isso importa no nosso trabalho

Se a projecao densa estiver errada, Q/K/V tambem ficam errados. Entao, validamos essa operacao separadamente antes de confiar no bloco.

6.7.4 Como implementamos

```
python
def dense_projection_torch(inputs, weight, bias=None):
    output = torch.matmul(inputs, weight.transpose(-1, -2))
    if bias is not None:
        output = output + bias
    return output
```

6.7.5 Como testamos

Usamos um exemplo pequeno:

```
python
inputs = torch.tensor([[[[1.0, 2.0], [3.0, 4.0]]]])
weight = torch.tensor([[1.0, 0.0], [0.0, 2.0]])
bias = torch.tensor([0.5, -0.5])

expected = torch.tensor([[[[1.5, 3.5], [3.5, 7.5]]]])

assert torch.allclose(actual, expected)
```

6.7.6 Com o que comparamos

Comparamos com o resultado manual e com uma implementacao NumPy independente.

6.7.7 O que podemos concluir

Podemos concluir que a projecao densa do projeto segue a formula linear esperada.

6.7.8 O que ainda nao podemos concluir

Projecao densa sozinha nao e Transformer. Ela e uma operacao auxiliar usada dentro do bloco.

6.8 Capitulo 8 - Informacao posicional

6.8.1 Ideia inicial

Attention pura enxerga relacoes entre vetores, mas nao sabe automaticamente a ordem dos tokens. Entao, Transformers usam alguma forma de informacao posicional para indicar que a posicao 1 vem antes da posicao 2.

6.8.2 Definicao cientifica

Vaswani et al. usam positional encoding no Transformer original. No nosso recorte, usamos encoding sinusoidal, que adiciona sinais dependentes da posicao aos vetores da sequencia.

6.8.3 Por que isso importa no nosso trabalho

Se removemos informacao posicional sem justificativa, o bloco fica incompleto para ser chamado de bloco Transformer. Por conta disso, a auditoria verifica explicitamente se existe informacao posicional ou justificativa formal.

6.8.4 Como implementamos

```
python
position = torch.arange(seq_len, device=device, dtype=dtype).unsqueeze(1)
div_term = torch.exp(
    torch.arange(0, d_model, 2, device=device, dtype=dtype)
    * (-math.log(10000.0) / d_model)
)
encoding[:, 0::2] = torch.sin(position * div_term)
encoding[:, 1::2] = torch.cos(position * div_term[: encoding[:, 1::2].shape[1]])
```

E no bloco:

```
python
hidden = self.add_positional_information(sequence)
```

6.8.5 Como testamos

A auditoria verifica:

```
python
assert report.has_positional_information_or_justification
```

6.8.6 Com o que comparamos

Comparamos com o criterio arquitetural do Transformer: a sequencia precisa ter alguma informacao de posicao, ou a ausencia precisa ser justificada.

6.8.7 O que podemos concluir

Podemos concluir que o bloco atual tem informacao posicional.

6.8.8 O que ainda nao podemos concluir

Nao podemos concluir que esse encoding e o melhor para qualquer tarefa. Ele e adequado para validar o conceito.

6.9 Capitulo 9 - Inferencia

6.9.1 Ideia inicial

Inferencia e usar o modelo para produzir saidas, sem atualizar pesos. Entao, quando medimos custo computacional, queremos medir execucao, nao treinamento.

6.9.2 Definicao cientifica

Goodfellow et al. separam o uso de modelos para predicacao da fase de treinamento. No PyTorch, operacionalmente usamos `eval()` e `torch.inference_mode()`.

6.9.3 Por que isso importa no nosso trabalho

Se os pesos mudam durante o experimento, a comparacao com baseline deixa de ser justa. Entao, fixamos seed, colocamos modelo em `eval()` e usamos inferencia.

6.9.4 Como implementamos

Na auditoria:

```
python
module.eval()
with torch.inference_mode():
    return module(sequence)
```

No benchmark controlado:

```
python
with torch.inference_mode():
    baseline_output = baseline_model(inputs).detach()
```

6.9.5 Como testamos

A auditoria roda a mesma entrada duas vezes:

python

```
first_output = _safe_forward(module, sequence)
second_output = _safe_forward(module, sequence)

deterministic_in_eval = torch.allclose(first_output, second_output, atol=atol)
```

6.9.6 Com o que comparamos

Comparamos uma saida contra a outra, para verificar determinismo.

6.9.7 O que podemos concluir

Podemos concluir que o bloco se comporta de forma deterministica em modo de avaliacao.

6.9.8 O que ainda nao podemos concluir

Isso nao garante reproducibilidade universal em qualquer hardware e qualquer kernel. Por conta disso, o benchmark registra ambiente, seed e dispositivo.

6.10 Capitulo 10 - KV cache

6.10.1 Ideia inicial

KV cache e uma economia de recomputacao. Em geracao autoregressiva, quando o modelo gera token por token, as chaves e valores dos tokens antigos nao precisam ser recalculados toda vez.

Entao, o cache guarda K e V passados. A cada novo token, calculamos K/V apenas do token novo e juntamos com o cache.

6.10.2 Definicao cientifica

O conceito aparece em inferencia de Transformers autoregressivos e em trabalhos de compressao de KV cache. Usamos como referencia moderna TurboQuant:

- Zandieh et al., *TurboQuant*, 2025: .
- Google Research, **TurboQuant: Redefining AI efficiency with extreme compression**: .

6.10.3 Por que isso importa no nosso trabalho

KV cache e importante para memoria e latencia em inferencia. Mas, antes de medir latencia, precisamos saber se o cache preserva a mesma saida.

6.10.4 Como implementamos

O caminho com cache concatena K/V antigos com K/V do token novo:

```
python
key_cache = key if key_cache is None else torch.cat([key_cache, key], dim=1)
value_cache = value if value_cache is None else torch.cat([value_cache, value], dim=1)
outputs.append(scaled_dot_product_attention_torch(query, key_cache, value_cache))
```

6.10.5 Como testamos

Comparamos atencao causal completa contra atencao incremental com cache:

```
python
full_output = causal_self_attention_full(sequence, identity, identity, identity)
cached = causal_self_attention_with_cache(sequence, identity, identity, identity)

assert torch.allclose(cached.output, full_output, atol=1e-6)
assert cached.cached_key_value_tokens == 4
assert cached.naive_key_value_tokens == 10
assert cached.reused_key_value_tokens == 6
```

6.10.6 Com o que comparamos

Comparamos a saida com e sem cache. Se as saidas batem, o cache preserva o comportamento. Tambem comparamos a quantidade de K/V calculados: 4 contra 10 no caso pequeno.

6.10.7 O que podemos concluir

Podemos concluir equivalencia algoritmica e reducao de recomputacao no exemplo.

6.10.8 O que ainda nao podemos concluir

Ainda nao podemos concluir ganho de hardware. Para isso, precisamos medir memoria, latencia e throughput em execucao controlada.

6.11 Capitulo 11 - Pruning e sparsity

6.11.1 Ideia inicial

Pruning e remover ou zerar partes do modelo. A intencao e criar esparsidade: muitos zeros ou conexoes removidas. Entao, teoricamente, haveria menos trabalho a fazer.

Mas existe um detalhe importante: zerar pesos nao significa necessariamente que o hardware vai pular esses pesos. Se a multiplicacao continuar densa, a conta ainda pode custar

quase a mesma coisa.

6.11.2 Definição científica

Usamos:

- Tang et al., *A Survey on Transformer Compression*, 2024: .
- PyTorch pruning tutorial: .

6.11.3 Por que isso importa no nosso trabalho

Como vamos comparar pruning com quantização, precisamos classificar pruning sem exagero. Então, se o pruning apenas cria máscara, classificamos como conceitual/algorítmico.

6.11.4 Como implementamos

A observação de sparsity conta zeros:

python

```
total = int(tensor.numel())
zeros = int((tensor == 0).sum().item())
ratio = 0.0 if total == 0 else zeros / total
```

E a classificação considera hardware apenas se houver armazenamento e kernel esparsos:

python

```
if self.uses_sparse_storage and self.uses_sparse_kernel:
    return "hardware"
return "conceitual_algoritmico"
```

6.11.5 Como testamos

Usamos uma camada com 8 pesos e aplicamos pruning de 50%:

python

```
prune.l1_unstructured(layer, name="weight", amount=0.5)

observation = observe_sparsity(
    layer.weight,
    uses_sparse_storage=False,
    uses_sparse_kernel=False,
)

assert observation.zero_elements == 4
assert observation.sparsity_ratio == pytest.approx(0.5)
assert observation.validity_level == "conceitual_algoritmico"
```

```
assert hasattr(layer, "weight_mask")
```

6.11.6 Com o que comparamos

Comparamos com a expectativa operacional: 50% de 8 pesos = 4 zeros. Tambem verificamos a presenca de `weight_mask`, coerente com o mecanismo do PyTorch.

6.11.7 O que podemos concluir

Podemos concluir que o pruning cria esparsidade conceitual/algoritmica.

6.11.8 O que ainda nao podemos concluir

Nao podemos dizer que houve ganho real de hardware, porque o teste nao usa storage esparso nem kernel esparso.

6.12 Capitulo 12 - Quantizacao

6.12.1 Ideia inicial

Quantizacao e representar numeros com menos precisao. Entao, em vez de guardar tudo em `float32`, podemos representar pesos em `int8`, por exemplo.

Isso economiza memoria representacional, mas introduz erro numerico. Por conta disso, precisamos medir duas coisas: se a representacao realmente ficou menor e se o erro ficou controlado.

6.12.2 Definicao cientifica

Usamos:

- Tang et al. (2024), sobre compressao de Transformers.
- Zandieh et al. (2025), TurboQuant.
- Documentacao torchao: .

6.12.3 Por que isso importa no nosso trabalho

Quantizacao e uma das tecnicas centrais do estudo. Mas se quantizamos e logo voltamos para `float32`, isso e apenas simulacao numerica. Entao, separamos validade numerica de validade de hardware.

6.12.4 Como implementamos

A quantização manual simétrica:

python

```
max_abs = tensor.abs().max()
scale = max_abs / 127.0
quantized = torch.clamp(torch.round(tensor / scale), -127, 127).to(torch.int8)
dequantized = quantized.to(torch.float32) * scale
```

E com torchao v2:

python

```
from torchao.quantization import Int8WeightOnlyConfig, quantize_

quantize_(model, Int8WeightOnlyConfig(version=2))
```

6.12.5 Como testamos

Para a quantização manual:

python

```
tensor = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0], dtype=torch.float32)
quantized, scale, dequantized = symmetric_int8_quantize(tensor)

assert quantized.dtype == torch.int8
assert q_observation.storage_bytes == 5
assert tensor.numel() * tensor.element_size() == 20
assert torch.max(torch.abs(tensor - dequantized)).item() <= scale.item()
```

Para torchao:

python

```
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    quantize_(model, Int8WeightOnlyConfig(version=2))

assert caught == []
assert type(model.weight).__name__ == "Int8Tensor"
assert model.weight.qdata.dtype == torch.int8
```

6.12.6 Com o que comparamos

Comparamos:

- float32: 5 valores * 4 bytes = 20 bytes;
- int8: 5 valores * 1 byte = 5 bytes;
- erro máximo contra o limite dado pela escala;

- representacao interna qdata em int8 no torchao.

6.12.7 O que podemos concluir

Podemos concluir validade numerica e algoritmica da quantizacao no recorte.

6.12.8 O que ainda nao podemos concluir

Ainda nao podemos dizer 'ganho de hardware' sem demonstrar kernel/caminho de execucao compativel.

6.13 Capitulo 13 - Latencia, throughput, memoria e FLOPs

6.13.1 Ideia inicial

Depois que os conceitos estao validados, queremos medir custo. Mas custo nao e uma coisa so. Latencia e tempo por execucao. Throughput e quantidade processada por tempo. Memoria e armazenamento usado. FLOPs e uma estimativa teorica de operacoes.

Entao, se misturarmos tudo, a conclusao fica confusa. Por conta disso, medimos cada dimensao separadamente.

6.13.2 Definicao cientifica

Usamos Hennessy e Patterson, *Computer Architecture: A Quantitative Approach*, como referencia geral para avaliacao quantitativa de desempenho, memoria, latencia e throughput.

Tambem usamos Vaswani et al. para entender o custo de operacoes de attention em sequencias.

6.13.3 Por que isso importa no nosso trabalho

Uma tecnica pode reduzir memoria e nao reduzir latencia. Outra pode reduzir FLOPs teoricos e nao acelerar no hardware real. Entao, cada resultado precisa dizer exatamente qual metrica mudou.

6.13.4 Como implementamos FLOPs

Para projecao densa:

```
python
matmul_flops = 2 * batch_size * seq_len * in_features * out_features
bias_flops = batch_size * seq_len * out_features if bias else 0
```

```
return int(matmul_flops + bias_flops)
```

Para attention:

```
score_flops = 2 * batch_size * num_heads * seq_len * seq_len * head_dim
weighted_value_flops = 2 * batch_size * num_heads * seq_len * seq_len * head_dim
return int(score_flops + weighted_value_flops)
```

6.13.5 Como testamos FLOPs

```
assert dense_projection_flops(
    batch_size=2,
    seq_len=3,
    in_features=4,
    out_features=5,
) == 270

assert attention_matmul_flops(
    batch_size=2,
    seq_len=3,
    d_model=4,
    num_heads=2,
) == 288
```

6.13.6 Como implementamos benchmark controlado

O runner registra ambiente, seed, warmup, repeticoes e sincronizacao:

```
metadata = {
    **env,
    "warmup": config.warmup,
    "repetitions": config.repetitions,
    "scenarios": list(config.scenarios),
    "used_cuda_synchronization": used_cuda_synchronization,
}
```

6.13.7 Com o que comparamos

FLOPs sao comparados com formulas teoricas. Latencia, throughput e memoria serao comparados contra baseline no CSV gerado pelo benchmark.

6.13.8 O que podemos concluir

Podemos concluir que temos um protocolo de medicao pronto e validado por teste.

6.13.9 O que ainda nao podemos concluir

Ainda nao podemos concluir desempenho final antes de rodar benchmarks amplos e controlados.

6.14 Capitulo 14 - MSE, MAE, R2 e similaridade de coseno

6.14.1 Ideia inicial

Quando aplicamos pruning ou quantizacao, a saida pode mudar. Entao, precisamos medir quanto ela mudou em relacao ao baseline.

Por conta disso usamos quatro metricas: MSE, MAE, R2 e similaridade de cosseno. Elas olham para erro e preservacao da direcao da representacao.

6.14.2 Definicao cientifica

Usamos:

- Hastie, Tibshirani e Friedman, *The Elements of Statistical Learning*: .
- scikit-learn MSE: .
- scikit-learn MAE: .
- scikit-learn R2: .
- scikit-learn cosine similarity: .

6.14.3 Por que isso importa no nosso trabalho

Se uma tecnica acelera mas destroi a saida, ela pode nao ser util. Entao, desempenho precisa ser lido junto com fidelidade numerica.

6.14.4 Como implementamos

MSE:

python

```
return float(np.mean(np.square(reference_arr - candidate_arr)))
```

MAE:

python

```
return float(np.mean(np.abs(reference_arr - candidate_arr)))
```

R2:

python

```
return float(1.0 - (ss_res / ss_tot))
```

Cosseno:

python

```
return float(np.dot(reference_arr, candidate_arr) / (reference_norm * candidate_norm))
```

6.14.5 Como testamos

Para MSE, MAE e R2:

python

```
reference = np.array([1.0, 2.0, 3.0])
candidate = np.array([1.0, 2.0, 4.0])

assert mean_squared_error(reference, candidate) == pytest.approx(1.0 / 3.0)
assert mean_absolute_error(reference, candidate) == pytest.approx(1.0 / 3.0)
assert r2_score(reference, candidate) == pytest.approx(0.5)
```

Para cosseno:

python

```
reference = np.array([1.0, 1.0, 0.0])
candidate = np.array([1.0, 0.0, 0.0])

expected = 1.0 / math.sqrt(2.0)
assert cosine_similarity(reference, candidate) == pytest.approx(expected)
```

6.14.6 Com o que comparamos

Comparamos com valores manuais conhecidos e com scikit-learn.

6.14.7 O que podemos concluir

Podemos concluir que as metricas estao corretas para comparar saidas contra o baseline.

6.14.8 O que ainda nao podemos concluir

As metricas nao dizem, sozinhas, se uma tecnica e boa. Elas precisam ser interpretadas junto com latencia, memoria, throughput e nivel de validade.

6.15 Capitulo 15 - Protocolo experimental e matriz de validade

6.15.1 Ideia inicial

Agora juntamos tudo. Se cada conceito tem teste, ainda precisamos garantir que os resultados futuros sejam registrados de forma comparavel. Entao, criamos um schema CSV e uma matriz de validacao.

6.15.2 Definicao cientifica

Aqui a referencia e metodologica: experimentos precisam de rastreabilidade. No nosso caso, rastreabilidade significa ligar cada conclusao a uma fonte, a um teste e a um resultado.

6.15.3 Por que isso importa no nosso trabalho

Sem matriz, podemos acabar escrevendo conclusoes mais fortes do que os dados permitem. Entao, o pipeline obriga cada cenario a ter nivel de validade.

6.15.4 Como implementamos

O schema CSV obrigatorio:

```
python
CSV_COLUMNS = [
    "torch_version",
    "cuda_version",
    "gpu_name",
    "device",
    "model_kind",
    "scenario",
    "batch_size",
    "seq_len",
    "d_model",
    "num_heads",
    "dtype",
    "pruning_sparsity",
    "quantization_method",
    "validity_level",
    "latency_ms_mean",
    "latency_ms_p50",
    "latency_ms_p95",
    "throughput_tokens_s",
    "max_memory_bytes",
```

```
"theoretical_flops",  
"mse",  
"mae",  
"r2",  
"cosine_similarity",  
]
```

E a validacao rejeita colunas faltando, colunas extras e niveis invalidos.

6.15.5 Como testamos

```
python  
validation = validate_benchmark_record(record)  
invalid = validate_benchmark_record({"validity_level": "promessa_sem_evidencia"})  
  
assert validation.is_valid  
assert not invalid.is_valid
```

Tambem testamos que cenarios nao registrados sao rejeitados:

```
python  
with pytest.raises(ScenarioValidationError, match="cenario sem matriz"):  
    validate_scenarios_registered(("baseline", "cenario_sem_evidencia"))
```

6.15.6 Com o que comparamos

Comparamos cada resultado contra o baseline e contra a matriz de validade.

6.15.7 O que podemos concluir

Podemos concluir que o estudo tem um caminho metodologico controlado para aceitar ou rejeitar resultados.

6.15.8 O que ainda nao podemos concluir

Ainda nao podemos concluir qual tecnica vence no desempenho final. O protocolo so garante que, quando os benchmarks forem rodados, a conclusao sera rastreavel.

6.16 Fechamento - A frase metodologica do projeto

A historia inteira pode ser apresentada assim:

O estudo nao mede desempenho antes de validar significado. Primeiro definimos cada conceito pela literatura. Entao transformamos essa definicao em uma propriedade observavel no codigo. Depois testamos essa propriedade com casos pequenos, deterministas e comparaveis com formulas ou bibliotecas de referencia. Por conta disso, quando formos medir pruning, quantizacao, latencia, memoria e throughput, saberemos exatamente o nivel de validade de cada conclusao.

Essa frase protege o trabalho de uma fragilidade comum: usar palavras fortes sem evidencia proporcional. Entao, quando dissermos 'bloco Transformer simplificado', sabemos quais criterios ele passou. Quando dissermos 'quantizacao numerica', sabemos que houve int8, escala e erro medido. Quando dissermos 'pruning conceitual/algoritmico', sabemos que ha zeros e mascara, mas nao necessariamente ganho de hardware.

O estado atual dos testes confirma essa base:

```
.\.venv\Scripts\python.exe -m pytest -q -W error
```

Resultado observado no ambiente de validacao cruzada em 24/08/2026:

```
32 passed, 1 skipped
```

O teste ignorado exige CUDA. Os testes NumPy, PyTorch e TensorFlow/Keras passaram sem skip.

Entao, o proximo passo do projeto nao e 'inventar a metodologia'; a metodologia ja esta desenhada. O proximo passo e usar esse protocolo para rodar benchmarks controlados e interpretar os resultados sem ultrapassar o nivel de evidencia observado.

Perguntas de pesquisa e criterios de validade

Este documento formaliza o que o estudo pretende responder e quais evidencias sao necessarias para uma conclusao ser aceita no artigo.

7.1 Perguntas de pesquisa

ID	Pergunta	Metricas associa- das	Evidencia mi- nima
RQ1	Quantizacao int8 preserva melhor a saida do bloco Transformer simplificado do que pruning por magnitude?	MSE, MAE, R2, similaridade de cosseno	Saida baseline, saida candidata e metricas de erro.
RQ2	Quantizacao e pruning reduzem memoria teorica ou real no ambiente do estudo?	max_memory_bytes	Ambiente registrado, medicao ou estimativa de memoria e nivel de validade.
RQ3	Alguma tecnica reduz latencia ou aumenta throughput sem degradacao excessiva da saida?	Latencia media, p50, p95, throughput, MSE e cosseno	Warmup, repeticoes, sincronizacao em CUDA e baseline comparavel.

ID	Pergunta	Metricas associa- das	Evidencia mi- nima
RQ4	Quando uma tecnica e ape- nas concei- tual/numerica e quando pode ser chamada de hardware?	validity_level	Matriz de validacao, dtype/formato e kernel/caminho de execucao.
RQ5	Os resultados permanecem consistentes entre seeds, distribuicoes e configuracoes arquiteturais?	Taxa de efeito pareado, IC bo- otstrap, MSE e latencia	Cinco seeds, tres repeticoes, perfis, distribu- icoes e pareaa- mento.
RQ6	Como a degra- dacao varia com sparsity de 10%, 25%, 50% e 75%?	Sparsity, MSE, cosseno e per- plexidade	Curvas quali- dade por spar- sity, pares e in- tervalos boots- trap.
RQ7	As tendencias sinteticas apa- recem tambem em modelos pre- treinados?	Perplexidade, KL dos logits, top-1 e concordancia de geracao	Revisoes OPT fixadas, Wiki- Text versionado e subconjunto hasheado.
RQ8	Quais tecnicas reduzem arma- zenamento, me- moria ou latencia com caminhos fisicos compati- veis?	Armazenamento, pico de memo- ria, speedup e kernel	Profiler, kernel INT8 ou 2:4 e ba- seline do mesmo dtype.

7.2 Hipoteses

ID	Pergunta	Hipotese	Confirma se	Rejeita se
H1	RQ1	Quantizacao int8 tera menor erro numerico do que pruning por magnitude no bloco controlado.	MSE e MAE menores, R2 e cosseno maiores que pruning na mesma configuracao.	Pruning empata ou supera quantizacao nas metricas de fidelidade.
H2	RQ2	Quantizacao int8 reduz memoria representacional; pruning por mascara densa nao prova reducao real de hardware.	Quantizacao apresenta menor armazenamento observado/estimado e pruning sem sparse kernel fica sem nivel hardware.	Quantizacao nao reduz armazenamento ou pruning demonstra formato/kernel esparso exploravel.
H3	RQ3	Ganhos de latencia/throughput dependem do caminho real de execucao, nao apenas da tecnica declarada.	Cenarios sem kernel/formato compativel nao sao promovidos a hardware mesmo que tenham erro aceitavel.	Ha evidencia de kernel/formato otimizado e medicao consistente de ganho fisico.
H4	RQ4	A matriz de validacao separa corretamente conclusoes conceituais, numericas, algoritmicas e de hardware.	Toda conclusao aponta para registro CSV e linha conceitual da matriz.	Alguma conclusao exige evidencia que nao esta registrada.
H5	RQ5	As tendencias principais persistem entre seeds, distribuicoes e perfis.	A direcao ocorre em pelo menos 80% dos pares e o IC95% nao cruza o efeito nulo.	A direcao oposta predomina; ausencia de predominio e classificada como mista.
H6	RQ6	A degradacao cresce com a sparsity do pruning por magnitude.	As metricas pareadas formam tendencia monotona entre os quatro niveis.	Sparsity maior melhora a qualidade de modo predominante e consistente.
H7	RQ7	As tendencias sinteticas de qualidade aparecem nos modelos OPT.	A direcao coincide nos tres modelos para perplexidade e logits.	A direcao oposta predomina; divergencia por modelo e classificada como mista.
H8	RQ8	Apenas representacoes compactas e kernels confirmados sustentam ganhos fisicos.	Speedup mediano $\geq 1,05$, IC95% acima de 1 e kernel correto no profiler.	Armazenamento/memoria nao caem ou speedup, intervalo e kernel nao sao satisfeitos juntos.

7.3 Regra de conclusao

Uma tendencia e classificada como sustentada quando aparece em pelo menos 80% dos casos pareados e o intervalo bootstrap de 95% nao cruza o efeito nulo. Ela e mista quando depende de seed, perfil ou modelo, e contradita quando a direcao oposta e predominante e consistente. Ganho de desempenho exige speedup mediano de pelo menos 1,05, intervalo inteiramente acima de 1,0 e confirmacao do kernel. Os limites de aumento de perplexidade de 2% para quantizacao e 5% para pruning sao criterios operacionais deste experimento, nao regras universais.

Uma conclusao so pode entrar no texto final se ela apontar para:

1. uma pergunta de pesquisa;
2. uma hipotese ou criterio de verificacao;
3. uma linha da matriz de validacao;
4. um resultado CSV ou teste deterministico;
5. um nivel de validade coerente com a evidencia observada.

Essa regra e implementada no codigo por `validacao.pesquisa`, `validacao.benchmark_controlado` e `validacao.analise_resultados`.

Protocolo experimental antes dos benchmarks

Este protocolo define os portoes que uma tecnica precisa atravessar antes de ser usada em comparacoes de desempenho. A intencao e evitar que o trabalho misture tres coisas diferentes: definicao cientifica, validade numerica e ganho real de hardware.

8.1 Niveis de certeza

Nivel	Pergunta respondida	Evidencia minima
Definicao	O termo esta sendo usado corretamente?	Fonte cientifica ou primaria e definicao operacional.
Matematica	A formula local bate com a definicao?	Caso pequeno calculado manualmente e teste deterministico.
Algoritmo	A ferramenta implementa o metodo esperado?	Comparacao contra implementacao transparente ou API de referencia.
Hardware	A execucao pode sustentar ganho fisico?	Dtype/formato/kernel/medidor mostrando menor memoria, latencia ou maior throughput.

Uma tecnica pode passar nos tres primeiros niveis e ainda continuar pendente em hardware. Isso nao e falha; e uma conclusao experimental importante.

8.2 Portoes antes de benchmark

1. O conceito precisa aparecer na matriz de validacao.

2. A definicao precisa apontar para fonte cientifica ou documentacao primaria.
3. Deve existir teste pequeno e deterministico para o comportamento essencial.
4. A classificacao deve dizer se o resultado e conceitual, numerico, algoritmico ou hardware.
5. O nucleo da attention deve bater com uma referencia matematica independente e com APIs publicas de frameworks consolidados.
6. Benchmark amplo so pode ser executado depois dos testes conceituais e da comparacao entre frameworks passarem.

8.3 Validacao cruzada entre frameworks

Antes dos benchmarks, `validacao.comparacao_frameworks` executa os mesmos tensores Q, K e V em quatro caminhos:

- formula transparente em NumPy, usada como referencia matematica;
- formula transparente em PyTorch;
- `torch.nn.functional.scaled_dot_product_attention`;
- `keras.ops.nn.dot_product_attention` com backend TensorFlow.

A comparacao usa `float32`, CPU, dropout desativado, flash attention desativada, seed 2026, `atol=1e-6` e `rtol=1e-5`. Os casos cobrem valor manual pequeno, multi-head com entradas pseudoaleatorias compartilhadas e mascara aditiva. A entrada e gerada uma unica vez pelo NumPy para evitar divergencia entre os geradores aleatorios dos frameworks.

O CSV desta etapa tem schema proprio e nao altera o schema dos benchmarks de desempenho. A evidencia canônica fica em `evidencias/comparacao_frameworks/` e so e aprovada se shape, dtype, finitude e tolerancias passarem em todas as comparacoes.

8.4 Recorte experimental

O nucleo obrigatorio sera um bloco Transformer pequeno e controlado, com:

- entrada `[batch, seq_len, d_model]`;
- projecoes densas para Q, K e V;
- atencao multi-head baseada em $\text{softmax}(QK^T / \sqrt{d_k})V$;
- informacao posicional sinusoidal;
- residual, normalizacao e feed-forward;
- saida comparada contra baseline para medir erro.

Esse nucleo deve ser chamado de 'bloco Transformer simplificado'. O termo 'Transformer' fica reservado para uma arquitetura completa ou para uma implementacao que tenha justificativa formal para todos os componentes.

8.5 Cenarios experimentais

Cenario	Descricao	Validade inicial
baseline	Bloco em float32 ou dtype definido como referencia.	Algoritmica
pruning_magnitude	Poda por magnitude em pesos lineares.	Conceitual/algoritmica; hardware pendente
quantization_int8	Quantizacao int8 validada por dtype, bytes e erro.	Numerica/algoritmica; hardware pendente ate validar kernel
pruning_plus_quantization	Combinacao das duas tecnicas, apenas se ambas passarem nos testes isolados.	Pendente

Na extensao v3, esses cenarios iniciais foram mantidos para fidelidade numerica e separados dos caminhos fisicos. TorchAO INT8 weight-only/dinamico e pruning semiestruturado 2:4 receberam validade de hardware somente apos o profiler confirmar os operadores esperados. A coleta reduziu armazenamento e memoria, mas nao sustentou ganho de latencia.

8.6 Metricas

As medicoes de qualidade e custo devem ser separadas.

Metricas de qualidade:

- MSE;
- MAE;
- R2;
- similaridade de cosseno.

Metricas de custo:

- latencia media, p50 e p95;
- throughput em tokens/s;
- pico de memoria;
- FLOPs teoricos.

Em GPU, o benchmark deve usar warmup, sincronizacao antes/depois da medicao e

registro explicito do dispositivo.

8.7 Schema CSV obrigatorio

Todo resultado experimental deve ser salvo com estas colunas, nesta ordem:

```
torch_version,cuda_version,gpu_name,device,model_kind,scenario,batch_size,seq_len,d_model,
→ num_heads,dtype,pruning_sparsity,quantization_method,validity_level,latency_ms_mean,
→ latency_ms_p50,latency_ms_p95,throughput_tokens_s,max_memory_bytes,theoretical_flops,
→ mse,mae,r2,cosine_similarity
```

O modulo `validacao.protocolo` valida esse schema antes de qualquer resultado ser aceito como dado experimental do trabalho.

8.8 Runner controlado

O benchmark validado deve ser executado pelo modulo `validacao.benchmark_controlado`. Ele:

- rejeita cenarios que nao estao registrados no catalogo de validacao;
- evita APIs depreciadas no pipeline automatizado;
- fixa seed e registra ambiente em arquivo de metadados;
- usa o bloco Transformer simplificado como nucleo experimental;
- mede latencia com warmup, repeticoes e sincronizacao CUDA quando aplicavel;
- calcula erro contra baseline e FLOPs teoricos;
- grava CSV apenas se todas as colunas obrigatorias forem validas.

Este runner mede o bloco simplificado e deve ser tratado como piloto de infraestrutura. Em CPU, `max_memory_bytes` e uma estimativa do armazenamento do modelo; apenas em CUDA o campo usa `torch.cuda.max_memory_allocated`. A quantizacao manual dequantiza pesos para `float32`, portanto nao alega armazenamento nem kernel INT8. O benchmark principal das operacoes isoladas e definido separadamente no checklist pre-benchmark.

8.9 Runner principal das operacoes isoladas

`validacao.benchmark_operacoes` implementa o recorte reconciliado entre o plano formal e a reuniao:

- `dense_projection`: operacao mantida porque aparece explicitamente no plano;
- `self_attention`: uma cabeca com projecoes Q/K/V/saida e SDPA, usada como foco principal sugerido pelo orientador;

- o bloco Transformer simplificado fica fora do resultado principal.

Os dados sao sinteticos pseudoaleatorios com distribuicao normal, pois o estudo mede operacoes e fidelidade de saida, nao uma tarefa supervisionada ou um dataset. Mesma seed, shapes, entradas e pesos-base sao usados em todos os cenarios. A configuracao schema v2 fixa entradas $N(0, 1)$, pesos com inicializacao Xavier normal ($\text{desvio}=1/\sqrt{D}$) para matrizes quadradas) e bias zero. Essa escala evita que a variancia das projecoes cresca artificialmente com D . Pruning e quantizacao afetam apenas as matrizes de pesos.

A configuracao `experimentos/benchmark_principal_gpu.json` codifica seed 42, lote 1, $L=\{64, 128, 256\}$, $D=\{128, 256, 512\}$, 20 iteracoes de aquecimento, 50 medicoes e duas execucoes independentes. Cada execucao preserva os mesmos dados e alterna deterministicamente a ordem dos cenarios. O p50 e p95 usam `numpy.percentile` com interpolacao linear.

Cada lote independente grava CSV e JSON de metadados imediatamente. O directorio tambem recebe snapshot da configuracao, ambiente, log e manifesto com SHA-256. O runner recusa directorio nao vazio, `device=auto`, campos nao finitos e benchmark principal sem CUDA/GPU nominal. Assim, uma interrupcao posterior nao apaga o lote ja concluido.

8.10 Extensao v3 e modelos pre-treinados

O schema v3 preserva a leitura integral das evidencias v1/v2 e acrescenta `data_seeds`, perfis com batch, sequencia, dimensao e cabecas, distribuicoes de entrada, cenarios parametrizados, grupos de comparacao e blocos de medicao, execucao e requisitos de hardware. O CSV agregado registra a identidade do caso, baseline associado, repeticao, memoria alocada e reservada, armazenamento, hashes, status e confirmacao dos kernels. As amostras individuais ficam em `timings.csv`.

A bateria de robustez usa cinco seeds, tres repeticoes, 13 perfis/distribuicoes, duas operacoes e seis cenarios. Toda comparacao e pareada pela mesma entrada, peso, seed, perfil e repeticao. A analise calcula media, mediana, desvio padrao e intervalo bootstrap de 95% com 10 mil reamostragens e seed 2026.

Os cenarios fisicos usam BF16/FP16 compilados, TorchAO INT8 weight-only, TorchAO INT8 dinamico e pruning semiestruturado 2:4. Compilacao e transformacao ficam fora da regioa medida. O profiler precisa confirmar operacao inteira ou cuSPARSELt; caso contrario, o resultado permanece unsupported ou sem validade fisica.

Nos modelos OPT, os pesos sao recarregados da revisao original para cada variante e nunca sao treinados. A avaliacao de qualidade usa WikiText-2 e prompts locais; a de desempenho separa prefill, TTFT e decode com KV cache. Embeddings e `lm_head` sao excluidos das transformacoes. A retomada so e aceita quando configuracao, codigo, ambiente e artefatos preservados possuem os mesmos hashes.

`validacao.analise_benchmark_operacoes` valida os checksums do manifesto antes de ler os dados. Cada candidato e comparado somente com o baseline da mesma configuracao e

execucao independente. Depois, as execucoes sao consolidadas com media e desvio-padrao, preservando tambem os resultados individuais. Hashes de entrada e saida precisam coincidir entre execucoes; latencias podem variar.

Comando padrao:

```
powershell
.\.venv\Scripts\python.exe -m validacao.benchmark_controlado --output
↪ resultados\benchmark_controlado.csv
```

Para uma execucao curta de verificacao:

```
powershell
.\.venv\Scripts\python.exe -m validacao.benchmark_controlado --output
↪ resultados\benchmark_teste.csv --warmup 1 --repetitions 3
```

8.11 Analise dos resultados

A analise final deve usar `validacao.analise_resultados`. Ela valida o CSV, compara cada cenario com o baseline, gera tabelas de resumo, gera graficos e escreve conclusoes com origem explicita:

- evidencia no CSV;
- linha conceitual da matriz;
- nivel de validade observado.

Nenhuma conclusao deve ser escrita como ganho de hardware se o `validity_level` do resultado nao for hardware.

8.12 Validacao da NN como Transformer

A rede neural usada no nucleo experimental deve ser auditada por `validacao.auditoria_transformer` antes dos benchmarks. O resultado aceito para o recorte atual e `bloco_transformer_simplificado`.

Uma NN comum com camadas lineares nao pode passar nessa auditoria. Isso e um controle negativo obrigatorio para garantir que o estudo nao esta chamando uma rede neural generica de Transformer.

8.13 Comparacao com literatura

Nesta fase, a comparacao com literatura deve ser feita assim:

- formulas e definicoes: comparacao direta com valores derivados;
- bibliotecas: comparacao contra implementacao transparente ou documentacao primaria;

- desempenho: comparacao apenas como tendencia ou ordem de grandeza, salvo se modelo, hardware, kernel, dtype, entrada e protocolo forem equivalentes.

8.14 Criterio para conclusao de hardware

Uma tecnica so pode receber nivel 'hardware' quando houver evidencia de que o caminho executado usa a representacao otimizada:

- quantizacao: armazenamento menor e kernel/caminho compativel com o dtype;
- pruning: formato ou kernel que explore esparsidade;
- KV cache: reutilizacao de K/V entre passos e equivalencia numerica com a execucao sem cache;
- memoria: medicao real ou estimativa teorica claramente separada;
- latencia/throughput: medicao com repeticoes, warmup e sincronizacao.

Matriz de validacao de ferramentas

Esta matriz registra se cada ferramenta candidata implementa, de forma observavel, o conceito que afirma implementar. Ela deve ser atualizada sempre que novos testes forem executados.

Os valores numericos derivados das definicoes e a comparacao com as fontes primarias estao registrados em `fases/01_validacao_conceitual/docs/confronto_resultados_literatura.md`.

Status possiveis:

- Aderente: o comportamento observado bate com a definicao e com o teste.
- Parcialmente aderente: a ferramenta implementa parte do conceito, mas nao sustenta todas as conclusoes desejadas.
- Nao aderente: o comportamento observado nao sustenta o uso do conceito.
- Pendente: ainda nao foi possivel executar ou completar a verificacao.
- Fora do pipeline: ferramenta conhecida, mas excluida do fluxo atual por depreciacao, incompatibilidade ou falta de evidencia adequada.

Niveis de validade:

- Conceitual: serve para demonstrar a ideia.
- Numerico: permite medir erro ou fidelidade da saida.
- Algoritmico: implementa de fato o algoritmo esperado.
- Hardware: usa representacao, kernel ou estrutura capaz de sustentar alegacao de ganho fisico.
- Fora do pipeline: nao participa dos benchmarks ou testes automatizados.

Niveis de certeza usados no metodo:

- Definicao: termo sustentado por fonte cientifica ou primaria.
- Matematica: formula local bate com caso pequeno derivado.
- Algoritmo: ferramenta bate com implementacao transparente ou referencia.
- Hardware: execucao usa representacao, formato ou kernel compativel com ganho fisico mensuravel.

Ferramenta/algoritmo	Conceito declarado	Fonte científica ou primaria	Teste aplicado	Resultado observado	Status	Nível de validade
Implementacao ma-nual em validacao. dense	Projecao densa	Goodfellow et al. + Vaswani et al. (2017)	Comparar $Y=XW^T+b$ com calculo manual e NumPy.	Passou em pytest: saida <code>[[1.5, 3.5], [3.5, 7.5]]</code> bate com a formula.	Aderente	Numerico/algoritmico
Implementacao ma-nual em validacao. attention	Scaled dot-product attention	Vaswani et al. (2017), $\text{Attention}(Q,K,V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$	Comparar tensores pequenos contra calculo NumPy independente.	Passou em pytest: saida PyTorch ma-nual igual ao calculo NumPy independente dentro de tolerancia $1e-6$.	Aderente	Conceitual/algoritmico
<code>torch.nn.functional.scaled_dot_product_attention</code>	Scaled dot-product attention	Documentacao Py-Torch + Vaswani et al. (2017)	Comparar saida Py-Torch com imple-mentacao manual transparente.	Passou em pytest: saida da API PyTorch igual a implementacao manual transparente dentro de tolerancia $1e-6$.	Aderente	Algoritmico
<code>keras.ops.nn.dot_product_attention</code> com backend TensorFlow	Scaled dot-product at-tention em framework independente	Documentacao Keras Ops + Vaswani et al. (2017)	Reutilizar os mesmos Q, K, V , pesos $Q/K/V/O$ e mascara dos cami-nhos NumPy/PyTorch em casos determinis-tas.	Passou em pytest e na execucao v2: 18/18 comparacoes aprovadas, inclu-ndo multi-head com 1, 4 e 8 cabecas; maior erro absoluto $4.76837158e-07$ para $\text{atol}=1e-6$ e $\text{rtol}=1e-5$.	Aderente	Numerico/algoritmico
<code>validacao.attention</code> com mascara e multi-head	Multi-head/self-attention	Vaswani et al. (2017) + documentacao PyTorch SDPA	Validar split/combine de heads, shape e mascara aditiva contra PyTorch.	Passou em pytest: saida multi-head preserva shape <code>[batch, seq, d_model]</code> e bate com referencia por heads em $1e-6$.	Aderente	Algoritmico

Ferramenta/ algoritmo	Conceito declarado	Fonte cientifica ou primaria	Teste aplicado	Resultado observado	Status	Nivel de validade
validacao.transformer.SimplifiedTransformerBlock	Bloco Transformer simplificado	Vaswani et al. (2017) + criterios do documento teorico	Verificar entrada sequencial, Q/K/V, atencao, posicao, residual, norm e FFN.	Passou em pytest: saida preserva shape e checklist classifica como bloco_transformer_simplificado.	Aderente	Conceitual/algoritmico
validacao.auditoria_transformer	Auditoria arquitetural de NN Transformer	Vaswani et al. (2017) + criterios do protocolo	Auditar componentes, comportamento sequencial, attention interna e controle negativo.	Passou em pytest: o bloco atual e classificado como bloco_transformer_simplificado; uma NN linear comum e rejeitada como nao_aderente.	Aderente	Conceitual/algoritmico
validacao.kv_cache	KV cache	Literatura/ documentacao de inferencia autoregressiva e compressao de KV cache	Comparar atencao causal completa com versao incremental usando cache.	Passou em pytest: saida com cache bate com saida completa em 1e-6; para 4 tokens, projeta 4 K/V contra 10 no caminho ingenuo.	Aderente	Algoritmico; hardware pendente
validacao.flops	FLOPs teóricos	Hennessy e Patterson + analise de complexidade em Vaswani et al. (2017)	Calcular casos pequenos para projecao densa, attention e bloco.	Passou em pytest: projecao densa 270, attention 288, bloco 1992 FLOPs no caso definido.	Aderente	Numerico
validacao.protocolo	Schema experimental CSV	Protocolo metodologico do projeto	Validar colunas fixas e nivel de validade permitido.	Passou em pytest: schema preserva 24 colunas e rejeita nivel sem evidencia.	Aderente	Conceitual
validacao.pesquisa	Perguntas de pesquisa e hipoteses	Metodo experimental do projeto	Verificar se toda pergunta tem metrica, evidencia e hipotese vinculada.	Passou em pytest: contrato de pesquisa nao apresenta inconsistencias.	Aderente	Conceitual

Ferramenta/algoritmo	Conceito declarado	Fonte cientifica ou primaria	Teste aplicado	Resultado observado	Status	Nivel de validade
validacao.benchmark_controlado	Runner experimental controlado	Protocolo experimental do projeto + PyTorch	Rejeitar cenario sem matriz, registrar ambiente/seed, medir com repeticoes e validar CSV.	Passou em pytest: registros de benchmark sao validos, baseline tem erro zero e cenarios nao sao promovidos indevidamente a hardware.	Aderente	Algoritmico
validacao.analise_resultados	Analise final e evidencias do artigo	Protocolo experimental do projeto	Comparar cenarios contra baseline, gerar tabelas/graficos e conclusoes com fonte.	Passou em pytest: tabelas, graficos e conclusoes apontam para CSV e matriz.	Aderente	Algoritmico
Checklist em validacao.classificacao	Classificacao de operacao/bloco/Transformer	Vaswani et al. (2017) + criterios do documento teorico	Verificar casos de operacao inspirada, bloco simplificado e Transformer.	Passou em pytest: os casos minimos sao classificados corretamente.	Aderente	Conceitual
Metricas manuais em validacao.metrics	MSE, MAE, R2 e similaridade de cosseno	Hastie, Tibshirani e Friedman; Manning, Raghavan e Schutze; scikit-learn	Comparar exemplos pequenos contra resultados conhecidos e scikit-learn.	Passou em pytest: metricas manuais batem com valores conhecidos e com scikit-learn.	Aderente	Numerico
torch.nn.utils.prune	Pruning/esparsificacao	Tang et al. (2024) + tutorial oficial PyTorch pruning	Aplicar poda L1 em camada linear pequena e medir proporcao de zeros.	Passou em pytest: poda L1 produziu 50% de zeros e mascara weight_mask; nao usa armazenamento/kernel esparsos no teste.	Parcialmente aderente	Conceitual/algoritmico; hardware pendente
Quantizacao simetrica int8 em validacao.quantization	Quantizacao numerica	Tang et al. (2024), Zandieh et al. (2025)	Verificar dtype int8, reducao de bytes e erro de dequantizacao.	Passou em pytest: tensor quantizado usa int8, reduz bytes e erro de dequantizacao fica limitado pela escala.	Aderente	Numerico

Ferramenta/algoritmo	Conceito declarado	Fonte cientifica ou primaria	Teste aplicado	Resultado observado	Status	Nivel de validade
torch.ao.quantization.quantize_dynamic	Quantizacao dinamica de camada linear	Documentacao Py-Torch quantization	Avaliar manutencao da ferramenta no pipeline.	Removida da suite automatizada: API depreciada no ambiente atual e substituida por torchao v2.	Fora do pipeline	Fora do pipeline
torchao v2 weight-only	Pesos INT8 compactos	Documentacao oficial torchao	Verificar armazenamento interno, forward compilado e operador no profiler.	Peso Int8Tensor usa qdata int8; o probe registrou weight_int8pack_mm.	Aderente	Hardware para armazenamento e caminho weight-only; ativacoes nao inteiras
torchao v2 dinamico	Ativacoes e pesos INT8	Documentacao oficial torchao	Compilar, medir e exigir GEMM inteira no profiler.	Forward finito e kernel CUTLASS S8 confirmados; armazenamento e memoria cairam, mas latencia piorou.	Aderente	Hardware, sem ganho de latencia no recorte
to_sparse_semi_structured	Pruning semiestruturado 2:4	Documentacao Py-Torch e NVIDIA cuSPARSElt	Converter pesos FP16, compilar e exigir sparse GEMM.	Esparsidade 50%, armazenamento compacto e kernel cuSPARSElt confirmados; latencia piorou.	Aderente	Hardware, sem ganho de latencia no recorte

9.1 Regras para atualizar a matriz

1. Nenhuma ferramenta deve ser promovida para nível hardware apenas por ter menor erro numerico.
2. Quantizacao so recebe nível hardware se houver representacao menor e caminho de execucao compativel com essa representacao.
3. Pruning so recebe nível hardware se a esparsidade for explorada por formato ou kernel adequado.
4. Se uma ferramenta passar apenas em testes pequenos, registrar validade conceitual/algoritmica e deixar a validade de hardware como pendente.

Confronto entre resultados observados e literatura

Este documento responde uma pergunta central da validacao: os numeros obtidos nos testes batem com o comportamento esperado pelas definicoes aceitas na literatura?

A resposta curta e: sim para os testes matematicos e de aderencia conceitual. A bateria fisica posterior confirmou representacoes e kernels INT8/2:4, mas contradisse ganho de latencia nas operacoes isoladas. A comparacao de desempenho com modelos completos e setups equivalentes aos artigos continua em andamento.

10.1 Criterio usado

Os artigos e documentacoes primarias nem sempre fornecem os mesmos tensores pequenos que usamos nos testes. Por isso, a comparacao inicial nao e 'mesmo dataset, mesmo resultado final do artigo'. A comparacao correta nesta fase e:

1. pegar a definicao publicada;
2. derivar um caso pequeno e deterministico;
3. calcular o valor esperado pela formula;
4. comparar a implementacao local com esse valor;
5. limitar a conclusao ao nivel que o teste realmente sustenta.

10.2 Resultados numericos verificados

Conceito	Fonte usada	Valor esperado pela defini- cao	Resultado observado	Bate?	Limite da conclusao
Projecao densa	Goodfellow et al.; Vaswani et al. (2017) para proje- coes lineares em Q/K/V	Para $x=[1,2],[3,4]$, $w=[1,0],[0,2]$, $b=[0.5,-0.5]$, a saida esperada e $[[1.5,3.5],[3.5,7.5]]$.	O teste retornou exata- mente $[[1.5, 3.5], [3.5, 7.5]]$.	Sim	Valida a operacao linear $Y=XW^T+b$, nao uma rede completa.
Scaled dot-product atten- tion	Vaswani et al. (2017), formula $\text{softmax}(QK^T/\sqrt{d_k})V$	Para $Q=K=[1,0],[0,1]$, $V=[1,2],[3,4]$, a saida esperada e $[[1.6604769, 2.6604769], [2.3395231, 3.3395231]]$.	O teste retornou $[[1.6604769, 2.6604769], [2.3395228, 3.3395233]]$.	Sim	Valida a operacao mate- matica de atencao escalo- nada, nao uma arquitetura Transformer completa.
Attention PyTorch	Documentacao PyTorch + Vaswani et al. (2017)	A API deve produzir a mesma operacao de sca- led dot-product attention, com tolerancia numerica.	A saida de <code>torch.nn. functional.scaled_dot_ product_attention</code> bateu com a implementacao manual em $1e-6$.	Sim	Valida a API como refe- rencia algoritmica para attention pequena e deter- ministica.
Attention Ke- ras/TensorFlow	Documentacao Keras Ops + Vaswani et al. (2017)	A API publica deve pro- duzir a mesma scaled dot-product attention para os mesmos Q, K, V e mas- cara.	Os tres casos contra NumPy foram aprovados; o maior erro absoluto Keras/TensorFlow foi $2.38418579e-07$, abaixo de $atol=1e-6$.	Sim	Valida equivalencia nume- rica entre frameworks em CPU/float32; nao compara desempenho.
Mascara em attention	Documentacao PyTorch SDPA	Mascara aditiva deve alterar os scores antes do <code>softmax</code> e produzir a mesma saida da API de referencia.	A implementacao manual com mascara bateu com PyTorch em $1e-6$.	Sim	Valida mascara aditiva no caso pequeno; outros tipos de mascara devem ser testados se usados.
Multi-head attention	Vaswani et al. (2017), divisao da representacao em multiplas heads	A saida apos split, atten- tion por head e combine deve preservar shape $[batch, seq, d_model]$.	Para entrada $[2,3,4]$ e 2 heads, a saida preser- vou $[2,3,4]$ e bateu com PyTorch em $1e-6$.	Sim	Valida a mecanica multi- head, ainda sem treino ou avaliacao de tarefa.
Dependencia entre posi- coes	Vaswani et al. (2017), ideia de self-attention sobre sequencias	Alterar o valor de uma posicao pode alterar a saida calculada em outra posicao.	O teste alterou um token e a saida de outra posicao mudou.	Sim	Valida dependencia se- quencial minima; ainda nao prova qualidade de modelagem.

Conceito	Fonte usada	Valor esperado pela definicao	Resultado observado	Bate?	Limite da conclusao
Bloco Transformer simplificado	Vaswani et al. (2017) + criterios do documento teorico	Deve conter entrada sequencial, Q/K/V, attention escalonada, posicao, residual, norm e FFN.	O bloco preservou shape [2,3,4] e foi classificado como bloco_transformer_simplificado.	Sim	Valida um bloco controlado; nao deve ser chamado de Transformer completo.
Auditoria Transformer da NN atual	Vaswani et al. (2017) + protocolo do projeto	Um bloco Transformer deve passar em Q/K/V, attention escalonada, dependencia posicional, informacao posicional, residual, norm e FFN; uma NN comum nao deve passar.	A NN atual passou como bloco_transformer_simplificado; uma NN linear comum foi rejeitada como nao_aderente.	Sim	Valida que o nucleo pertence a familia Transformer, mas nao autoriza chama-lo de Transformer completo.
KV cache	Literatura de inferencia autoregressiva e compressao de KV cache	Saida com cache deve bater com saida causal completa e reduzir recomputacao de K/V.	Para 4 tokens, cache projetou 4 K/V contra 10 no caminho ingenuo; saidas bateram em 1e-6.	Sim	Valida equivalencia e reaproveitamento algoritmico; ganho de hardware ainda exige medicao.
FLOPs teóricos	Hennessy e Patterson; analise de complexidade de atencao em Vaswani et al. (2017)	Para o caso pequeno definido, projecao densa 270, attention 288, bloco 1992.	Os testes retornaram 270, 288 e 1992.	Sim	Valida contagem teorica do recorte; FLOPs nao equivalem automaticamente a latencia.
MSE	Definicao estatistica e sklearn.metrics.mean_squared_error	Para $y=[1,2,3]$, $y_{\text{hat}}=[1,2,4]$, $MSE=(0^2+0^2+1^2)/3=0.333333$.	0.3333333333333333.	Sim	Valida a metrica numerica.
MAE	Definicao estatistica e sklearn.metrics.mean_absolute_error	Para o mesmo exemplo, $MAE=(0+0+1)/3=0.333333$.	0.3333333333333333.	Sim	Valida a metrica numerica.
R2	Definicao de coeficiente de determinacao e sklearn.metrics.r2_score	Para o mesmo exemplo, $R2=1-SS_{\text{res}}/SS_{\text{tot}}=1-1/2=0.5$.	0.5.	Sim	Valida a metrica numerica.
Similaridade de cosseno	Manning, Raghavan e Schutze; scikit-learn pairwise metrics	Para $[1,1,0]$ e $[1,0,0]$, $\cos=1/\sqrt{2}=0.70710678$.	0.7071067811865475.	Sim	Valida a metrica numerica.

Conceito	Fonte usada	Valor esperado pela definicao	Resultado observado	Bate?	Limite da conclusao
Pruning L1 PyTorch	Tutorial/API PyTorch pruning	Em matriz 2x4, amount=0.5 deve zerar 4 de 8 pesos e criar mascara.	4/8 zeros, esparsidade 0.5, mascara weight_mask presente.	Sim, parcialmente	Valida pruning conceitual/algoritmico; nao valida ganho real de hardware porque o caminho continua denso.
Quantizacao simetrica int8 manual	Literatura de compressao/quantizacao e definicao operacional	Para tensor de 5 valores float32, esperado reduzir de 20 bytes para 5 bytes em armazenamento int8; escala 2/127=0.015748.	dtype=torch.int8, 5 bytes contra 20 bytes; erro maximo 0.007874, dentro do limite de arredondamento da escala.	Sim	Valida quantizacao numerica; nao valida kernel de inferencia.
torchao int8 weight-only v2	Documentacao torchao Int8WeightOnlyConfig	Configuracao deve aplicar quantizacao simetrica int8 weight-only em camadas lineares.	Peso Int8Tensor possui qdata em int8; o profiler fisico confirmou weight_int8pack_mm.	Sim	Valida representacao compacta e caminho weight-only; nao implica ativacoes inteiras.

10.3 O que ainda nao esta validado

Ja podemos relatar compressao, memoria e latencia das operacoes isoladas na RTX 4070 Ti SUPER. Ainda nao podemos afirmar reproducao de artigos de desempenho: os setups, modelos e kernels publicados nao sao necessariamente equivalentes, e a avaliacao final dos tres modelos OPT permanece pendente.

Para confrontar desempenho diretamente com a literatura, o protocolo precisa fixar:

1. modelo ou bloco usado;
2. tamanho de sequencia;
3. batch size;
4. dtype;
5. hardware;
6. kernel usado;
7. metrica medida;
8. baseline;
9. tolerancia de erro numerico;
10. repeticoes e intervalo de confianca.

10.4 Conclusao metodologica

As ferramentas atuais sao adequadas para iniciar o trabalho porque passam na validacao de definicao: projecao densa, attention, bloco simplificado, KV cache, FLOPs, metricas, pruning conceitual e quantizacao numerica se comportam como esperado pelas formulas e documentacoes primarias.

Os caminhos fisicos agora sao suficientes para afirmar reducao de armazenamento e memoria no recorte medido, pois formato e kernels foram confirmados. Eles nao sustentam ganho de latencia: os seis grupos ficaram abaixo do baseline. Uma generalizacao para Transformers pre-treinados depende da coleta OPT e uma comparacao direta com artigos exige equivalencia de setup.

10.5 Referencias

- Vaswani et al. (2017). *Attention Is All You Need*: .
- PyTorch. `scaled_dot_product_attention`: .
- Keras. `keras.ops.nn.dot_product_attention`: .
- PyTorch. `torch.nn.utils.prune.l1_unstructured`: .
- PyTorch Tutorials. *Pruning Tutorial*: .

- PyTorch. `torchao.quantization.Int8WeightOnlyConfig`: .
- scikit-learn. `mean_squared_error`: .
- scikit-learn. `r2_score`: .
- scikit-learn. *Cosine similarity*: .

Decisão metodológica: inicialização dos pesos do benchmark

11.1 Problema observado

O diagnóstico CPU v1 usou entradas e pesos independentes com distribuição $N(0,1)$. Para uma projeção com dimensão D , somar D produtos com pesos de variância 1 faz a variância da saída crescer com a dimensão. Na self-attention, esse efeito se repete nas projeções Q/K/V, nos scores e na projeção de saída.

O caso $B=1$, $L=64$, $D=128$ mostrou o risco: a quantização ainda foi melhor que o pruning nas métricas relativas, mas o MSE absoluto da self-attention foi 732.65. Esse valor não era comparável de forma limpa com dimensões menores, porque misturava erro da técnica e aumento da escala do baseline.

11.2 Alternativas consideradas

1. **Manter $N(0,1)$ e interpretar cada dimensão separadamente.** Preservaria o primeiro gerador, mas enfraqueceria MSE/MAE como métricas da grade.
2. **Normalizar apenas MSE/MAE.** Acrescentaria novas métricas e mudaria o schema do benchmark, contrariando a decisão de preservar a interface.
3. **Escalar os pesos-base pela dimensão.** Mantém as métricas, controla a variância das projeções e continua usando exatamente os mesmos pesos entre cenários da mesma configuração.

A alternativa 3 foi escolhida. É uma decisão metodológica posterior à reunião, não um pedido literal do professor.

11.3 Regra operacional adotada

- entradas sintéticas: $N(0,1)$;
- matrizes quadradas: Xavier normal, com desvio = $\sqrt{2 / (fan_in + fan_out))} = 1/\sqrt{D}$;
- bias da projeção densa: zero;
- mesma seed 42, dados e pesos-base para baseline, pruning e quantização;

- parâmetros registrados no schema de configuração v2 e nos metadados;
- evidências de schema v1 permanecem legíveis e versionadas.

11.4 Confronto v1 versus v2

As duas coletas abaixo usam $B=1$, $L=64$, $D=128$, 20 warm-ups, 50 medições e duas execuções. A única mudança metodológica relevante para a qualidade é a inicialização.

Operação/cenário	Métrica	v1 - pesos N(0,1)	v2 - Xavier	Leitura
Projeção/quantização	MSE	0.00995697	0.000077789	Escala absoluta controlada.
Projeção/pruning	MSE	9.30825	0.0727207	Escala absoluta controlada.
Self-attention/quantização	MSE	732.65	0.0000163925	O caso deixa de ser dominado pela explosão de escala.
Self-attention/quantização	R²	0.95428	0.999647	Fidelidade relativa também melhora com scores não saturados.
Self-attention/quantização	Cosseno	0.977141	0.999824	Direção da saída quase preservada em v2.
Self-attention/pruning	MSE	12144.35	0.0110127	Pruning continua claramente mais destrutivo.

Não se comparam as latências de v1 e v2 para concluir desempenho: foram coletas CPU separadas, os caminhos não usam kernels esparsos/INT8 e a variabilidade observada foi alta.

11.5 Decisão para a coleta principal

O schema v2 com Xavier/bias zero é o contrato vigente. A v1 não foi apagada nem reclassificada; ela documenta a ameaça à validade que motivou a correção. A conclusão sobre H1 continua preliminar até a grade completa, mas agora MSE/MAE têm uma escala experimental mais controlada entre dimensões.

Parte III

Evidencias e acompanhamento

Comparação numérica da attention local e multi-head

Esta evidência valida correção numérica. Ela não compara desempenho e não sustenta alegações de hardware.

- Referência: `numpy_manual`.
- Seed: 2026.
- Dtype e dispositivo: `float32` em `cpu`.
- Tolerâncias: `atol=1e-06` e `rtol=1e-05`.
- Backend Keras: `tensorflow`.

Caso	Candidato	Máscara	Erro máximo	Erro médio	MSE	Cosseno	Passou
tiny_manual_unmasked	pytorch_manual	none	2.38418579e-07	5.96046448e-08	1.42108547e-14	1.00000000e+00	sim
tiny_manual_unmasked	pytorch_sdpa	none	0.00000000e+00	0.00000000e+00	0.00000000e+00	1.00000000e+00	sim
tiny_manual_unmasked	keras_tensorflow_sdpa	none	2.38418579e-07	5.96046448e-08	1.42108547e-14	1.00000000e+00	sim
seeded_multihead_unmasked	pytorch_manual	none	1.19209290e-07	2.45636329e-08	1.77382703e-15	1.00000000e+00	sim
seeded_multihead_unmasked	pytorch_sdpa	none	1.19209290e-07	2.79008721e-08	1.74115641e-15	1.00000000e+00	sim
seeded_multihead_unmasked	keras_tensorflow_sdpa	none	1.19209290e-07	2.33218695e-08	1.57491208e-15	1.00000000e+00	sim
seeded_multihead_additive_mask	pytorch_manual	additive_bias	5.96046448e-08	6.20881716e-09	3.33066907e-16	1.00000000e+00	sim
seeded_multihead_additive_mask	pytorch_sdpa	additive_bias	5.96046448e-08	1.67638063e-08	9.34437712e-16	1.00000000e+00	sim
seeded_multihead_additive_mask	keras_tensorflow_sdpa	additive_bias	1.19209290e-07	2.73187955e-08	2.09092003e-15	1.00000000e+00	sim
projected_multihead_h1	pytorch_manual	none_projected_qkvo	3.57627869e-07	1.22934580e-07	2.38512913e-14	1.00000000e+00	sim
projected_multihead_h1	pytorch_sdpa	none_projected_qkvo	2.38418579e-07	1.31626924e-07	2.19454085e-14	1.00000000e+00	sim
projected_multihead_h1	keras_tensorflow_sdpa	none_projected_qkvo	1.19209290e-07	3.10440858e-08	2.53500924e-15	1.00000000e+00	sim
projected_multihead_h4	pytorch_manual	none_projected_qkvo	4.17232513e-07	9.80993112e-08	1.71754971e-14	1.00000000e+00	sim
projected_multihead_h4	pytorch_sdpa	none_projected_qkvo	4.17232513e-07	8.76995424e-08	1.52950569e-14	1.00000000e+00	sim
projected_multihead_h4	keras_tensorflow_sdpa	none_projected_qkvo	3.57627869e-07	8.81652037e-08	1.47145028e-14	1.00000000e+00	sim
projected_multihead_h8	pytorch_manual	none_projected_qkvo	4.76837158e-07	1.08993845e-07	1.99298269e-14	1.00000000e+00	sim
projected_multihead_h8	pytorch_sdpa	none_projected_qkvo	4.76837158e-07	1.11787813e-07	2.04438833e-14	1.00000000e+00	sim
projected_multihead_h8	keras_tensorflow_sdpa	none_projected_qkvo	3.57627869e-07	1.07528952e-07	1.89554906e-14	1.00000000e+00	sim

12.1 Conclusão

Todos os casos ficaram dentro das tolerâncias definidas.

As versões completas do ambiente estão em `comparacao_atencao.metadata.json` e os valores auditáveis em `comparacao_atencao.csv`.

Relatório preliminar - smoke_cpu_v1

Esta coleta é um smoke test em CPU. Os tempos servem apenas para validar a pipeline e não sustentam conclusão de hardware.

13.1 Pergunta, hipótese e desenho

- Pergunta principal: quantização INT8 simulada preserva melhor a saída que pruning por magnitude de 50% nas mesmas operações?
- Hipótese: a quantização terá MSE/MAE menores e R^2 /cosseno maiores que o pruning.
- Amostra: tensores sintéticos de distribuição normal, gerados deterministicamente; não há dataset ou pré-processamento externo.
- Seed: 42; lote(s): [1]; sequências: [4]; dimensões: [8].
- Medição: 2 warm-ups, 5 medições e 2 execuções independentes.
- Dispositivo observado: cpu
- Inclusão: todos os casos válidos da grade e os três cenários registrados; exclusão: qualquer artefato com checksum, shape, dtype ou valor numérico inválido.

13.2 Reprodutibilidade

- Hashes de entrada iguais entre execuções: True.
- Hashes de saída iguais entre execuções: True.
- As latências não precisam ser idênticas: elas medem ruído e estado do sistema; entradas e saídas determinísticas precisam coincidir.

13.3 Resultados por operação e cenário

Operação	Cenário	Latência média (ms)	DP entre execuções	MSE	R²	Cosseno
dense_projection	baseline	0.01942	0.0154715	0	1	1
dense_projection	pruning_magnitude	0.01916	0.0155563	0.352201	0.89254	0.947344
dense_projection	quantization_int8	0.03504	0.0070145	7.28672e-05	0.999978	0.999989
self_attention	baseline	0.25577	0.0244518	0	1	1
self_attention	pruning_magnitude	0.31862	0.193012	25.8574	0.1343	0.570018
self_attention	quantization_int8	0.51548	0.0405879	0.00503124	0.999832	0.999919

13.4 Interpretação preliminar

- dense_projection: MSE da quantização 7.28672e-05 e do pruning 0.352201; evidência compatível com H1 nesta amostra.
- self_attention: MSE da quantização 0.00503124 e do pruning 25.8574; evidência compatível com H1 nesta amostra.
- Pruning usa matriz densa e não usa kernel esparsos; quantização é dequantizada para float32 e não usa kernel INT8.
- Razões de latência e memória estão nos CSVs de análise, mas só podem sustentar alegação de hardware se o experimento for principal e o caminho executado tiver validade hardware.
- Resultados deste smoke são diagnósticos da infraestrutura, não estimativas finais de desempenho.

13.5 Cadeia de evidência

pergunta → H1 → configuração versionada → CSVs por execução → checksums/hashes → resumo com média e DP → interpretação limitada

13.6 Reprodução

powershell

```
$env:PYTHONPATH = "fases\01_validacao_conceitual"
.\.venv\Scripts\python.exe -m validacao.benchmark_operacoes --config <config.json>
↪ --output-dir <diretorio-novo>
.\.venv\Scripts\python.exe -m validacao.analise_benchmark_operacoes --evidence-dir
↪ <diretorio-novo>
```

Relatório preliminar - diagnostico_cpu_l64_d128_v1

Esta coleta é um diagnóstico em CPU de um ponto da grade formal. Os tempos não substituem a coleta na GPU-alvo e não sustentam conclusão de hardware.

14.1 Pergunta, hipótese e desenho

- Pergunta principal: quantização INT8 simulada preserva melhor a saída que pruning por magnitude de 50% nas mesmas operações?
- Hipótese: a quantização terá MSE/MAE menores e R^2 /cosseno maiores que o pruning.
- Amostra: tensores sintéticos de distribuição normal, gerados deterministicamente; não há dataset ou pré-processamento externo.
- Seed: 42; lote(s): [1]; sequências: [64]; dimensões: [128].
- Medição: 20 warm-ups, 50 medições e 2 execuções independentes.
- Dispositivo observado: cpu
- Inclusão: todos os casos válidos da grade e os três cenários registrados; exclusão: qualquer artefato com checksum, shape, dtype ou valor numérico inválido.

14.2 Reprodutibilidade

- Hashes de entrada iguais entre execuções: True.
- Hashes de saída iguais entre execuções: True.
- As latências não precisam ser idênticas: elas medem ruído e estado do sistema; entradas e saídas determinísticas precisam coincidir.

14.3 Resultados por operação e cenário

Operação	Cenário	Latência média (ms)	DP entre execuções	MSE	R²	Cosseno
dense_projection	baseline	0.088291	0.0207253	0	1	1
dense_projection	pruning_magnitude	0.076017	0.0643283	9.30825	0.927085	0.962897
dense_projection	quantization_int8	0.061625	0.00428648	0.00995697	0.999922	0.999961
self_attention	baseline	0.93542	0.383818	0	1	1
self_attention	pruning_magnitude	0.609696	0.0349735	12144.4	0.242143	0.596755
self_attention	quantization_int8	0.867666	0.183593	732.65	0.95428	0.977141

14.4 Interpretação preliminar

- dense_projection: MSE da quantização 0.00995697 e do pruning 9.30825; evidência compatível com H1 nesta amostra.
- self_attention: MSE da quantização 732.65 e do pruning 12144.4; evidência compatível com H1 nesta amostra.
- Pruning usa matriz densa e não usa kernel esparsos; quantização é dequantizada para float32 e não usa kernel INT8.
- Razões de latência e memória estão nos CSVs de análise, mas só podem sustentar alegação de hardware se o experimento for principal e o caminho executado tiver validade hardware.
- Resultados de smoke ou diagnóstico CPU não são estimativas finais de desempenho no hardware-alvo.

14.5 Cadeia de evidência

pergunta → H1 → configuração versionada → CSVs por execução → checksums/hashes → resumo com média e DP → interpretação limitada

14.6 Reprodução

powershell

```
$env:PYTHONPATH = "fases\01_validacao_conceitual"
.\.venv\Scripts\python.exe -m validacao.benchmark_operacoes --config <config.json>
↪ --output-dir <diretorio-novo>
.\.venv\Scripts\python.exe -m validacao.analise_benchmark_operacoes --evidence-dir
↪ <diretorio-novo>
```

Relatório preliminar - diagnostico_cpu_l64_d128_v2

Esta coleta é um diagnóstico em CPU de um ponto da grade formal. Os tempos não substituem a coleta na GPU-alvo e não sustentam conclusão de hardware.

15.1 Pergunta, hipótese e desenho

- Pergunta principal: quantização INT8 simulada preserva melhor a saída que pruning por magnitude de 50% nas mesmas operações?
- Hipótese: a quantização terá MSE/MAE menores e R^2 /cosseno maiores que o pruning.
- Amostra: tensores sintéticos de distribuição normal, gerados deterministicamente; não há dataset ou pré-processamento externo.
- Seed: 42; lote(s): [1]; sequências: [64]; dimensões: [128].
- Dados/pesos: entrada `standard_normal`, pesos `xavier_normal` e bias zeros.
- Medição: 20 warm-ups, 50 medições e 2 execuções independentes.
- Dispositivo observado: `cpu`
- Inclusão: todos os casos válidos da grade e os três cenários registrados; exclusão: qualquer artefato com checksum, shape, dtype ou valor numérico inválido.

15.2 Reprodutibilidade

- Hashes de entrada iguais entre execuções: `True`.
- Hashes de saída iguais entre execuções: `True`.
- As latências não precisam ser idênticas: elas medem ruído e estado do sistema; entradas e saídas determinísticas precisam coincidir.

15.3 Resultados por operação e cenário

Operação	Cenário	Latência média (ms)	DP entre execuções	MSE	R²	Cosseno
dense_projection	baseline	0.118959	0.0408976	0	1	1
dense_projection	pruning_magnitude	0.052224	0.00514774	0.0727207	0.926527	0.962572
dense_projection	quantization_int8	0.063593	0.00422426	7.7789e-05	0.999921	0.999961
self_attention	baseline	0.706044	0.502764	0	1	1
self_attention	pruning_magnitude	0.363446	0.0394877	0.0110127	0.762797	0.873495
self_attention	quantization_int8	0.412401	0.171529	1.63925e-05	0.999647	0.999824

15.4 Interpreta33o preliminar

- dense_projection: MSE da quantiza33o 7.7789e-05 e do pruning 0.0727207; evid3ncia compat3vel com H1 nesta amostra.
- self_attention: MSE da quantiza33o 1.63925e-05 e do pruning 0.0110127; evid3ncia compat3vel com H1 nesta amostra.
- Pruning usa matriz densa e n3o usa kernel esparsa; quantiza33o 3 dequantizada para float32 e n3o usa kernel INT8.
- Raz3es de lat3ncia e mem3ria est3o nos CSVs de an3lise, mas s3o podem sustentar alega33o de hardware se o experimento for principal e o caminho executado tiver validade hardware.
- Resultados de smoke ou diagn3stico CPU n3o s3o estimativas finais de desempenho no hardware-alvo.

15.5 Cadeia de evid3ncia

pergunta → H1 → configura33o versionada → CSVs por execu33o → checksums/hashes → resumo com m3dia e DP → interpreta33o limitada

15.6 Reprodu33o

powershell

```
$env:PYTHONPATH = "fases\01_validacao_conceitual"  
.\.venv\Scripts\python.exe -m validacao.benchmark_operacoes --config <config.json>  
↪ --output-dir <diretorio-novo>  
.\.venv\Scripts\python.exe -m validacao.analise_benchmark_operacoes --evidence-dir  
↪ <diretorio-novo>
```

Relatório preliminar - operacoes_principais_gpu_v1

Esta coleta foi configurada como benchmark principal em CUDA; as conclusões ainda devem respeitar o nível de validade de cada cenário.

16.1 Pergunta, hipótese e desenho

- Pergunta principal: quantização INT8 simulada preserva melhor a saída que pruning por magnitude de 50% nas mesmas operações?
- Hipótese: a quantização terá MSE/MAE menores e R^2 /cosseno maiores que o pruning.
- Amostra: tensores sintéticos de distribuição normal, gerados deterministicamente; não há dataset ou pré-processamento externo.
- Seed: 42; lotes: [1]; sequências: [64, 128, 256]; dimensões: [128, 256, 512].
- Dados/pesos: entrada `standard_normal`, pesos `xavier_normal` e bias zeros.
- Medição: 20 warm-ups, 50 medições e 2 execuções independentes.
- Dispositivo observado: `cuda:0 NVIDIA GeForce RTX 4070 Ti SUPER`.
- Inclusão: todos os casos válidos da grade e os três cenários registrados; exclusão: qualquer artefato com checksum, shape, dtype ou valor numérico inválido.

16.2 Reprodutibilidade

- Hashes de entrada iguais entre execuções: `True`.
- Hashes de saída iguais entre execuções: `True`.
- As latências não precisam ser idênticas: elas medem ruído e estado do sistema; entradas e saídas determinísticas precisam coincidir.

16.3 Resultados por operação e cenário

Operação	Cenário	Latência (ms)		DP	MSE	R^2	Cosseno
dense_projection	baseline	0.154976	0.0021694		0	1	1
dense_projection	baseline	0.136247	1.55563e-05		0	1	1
dense_projection	baseline	0.183354	0.0199857		0	1	1
dense_projection	baseline	0.127999	0.0133346		0	1	1
dense_projection	baseline	0.158239	0.00131663		0	1	1
dense_projection	baseline	0.128803	0.00148351		0	1	1
dense_projection	baseline	0.121921	0.00641629		0	1	1
dense_projection	baseline	0.157987	0.0111341		0	1	1
dense_projection	baseline	0.118018	0.0074925		0	1	1
dense_projection	pruning_magnitude	0.142595	0.0107212	0.0727207	0.926527	0.962572	
dense_projection	pruning_magnitude	0.171191	0.000748119	0.0715476	0.927112	0.962866	
dense_projection	pruning_magnitude	0.144272	0.0080497	0.0716447	0.928389	0.963533	
dense_projection	pruning_magnitude	0.124756	0.00698056	0.0707332	0.929816	0.96427	
dense_projection	pruning_magnitude	0.181001	0.030919	0.0720674	0.926982	0.962805	
dense_projection	pruning_magnitude	0.152848	0.0108244	0.0724666	0.927698	0.963173	
dense_projection	pruning_magnitude	0.150323	0.0410108	0.0693999	0.928408	0.96354	
dense_projection	pruning_magnitude	0.146113	0.0168192	0.0722279	0.928179	0.963421	
dense_projection	pruning_magnitude	0.12311	0.0356976	0.0713366	0.928748	0.963716	
dense_projection	quantization_int8	0.192197	0.00854326	7.77889e-05	0.999921	0.999961	
dense_projection	quantization_int8	0.152302	0.0176579	8.68791e-05	0.999911	0.999956	
dense_projection	quantization_int8	0.146561	0.00268842	0.000102175	0.999898	0.999949	

Operação	Cenário	Latência (ms)	DP	MSE	R^2	Cosseno
dense_projection	quantization_int8	0.17891	0.0275489	8.77829e-05	0.999913	0.999956
dense_projection	quantization_int8	0.149771	0.000719835	9.82458e-05	0.9999	0.99995
dense_projection	quantization_int8	0.163131	0.0637655	0.000103987	0.999896	0.999948
dense_projection	quantization_int8	0.123397	0.0159085	7.93685e-05	0.999918	0.999959
dense_projection	quantization_int8	0.143514	0.00309713	9.10519e-05	0.999909	0.999955
dense_projection	quantization_int8	0.149146	0.0336555	0.000141154	0.999859	0.99993
self_attention	baseline	0.547976	0.0418522	0	1	1
self_attention	baseline	0.493358	0.05727	0	1	1
self_attention	baseline	0.551014	0.0355335	0	1	1
self_attention	baseline	0.415846	0.065744	0	1	1
self_attention	baseline	0.494226	0.00640356	0	1	1
self_attention	baseline	0.492053	0.025115	0	1	1
self_attention	baseline	0.443417	0.00706117	0	1	1
self_attention	baseline	0.529221	0.0601649	0	1	1
self_attention	baseline	0.446569	0.0624814	0	1	1
self_attention	pruning_magnitude	0.524125	0.0438816	0.0110127	0.762797	0.873495
self_attention	pruning_magnitude	0.533186	0.089466	0.00952993	0.77801	0.882438
self_attention	pruning_magnitude	0.521855	0.00305329	0.00885257	0.766722	0.875633
self_attention	pruning_magnitude	0.431755	0.0535718	0.00617402	0.755223	0.869392
self_attention	pruning_magnitude	0.517076	0.0459959	0.00548234	0.744202	0.862855
self_attention	pruning_magnitude	0.506012	0.0226585	0.00544488	0.763186	0.873714

Operação	Cenário	Latência (ms)	DP	MSE	R^2	Cosseno
self_attention	pruning_magnitude	0.445271	0.0402414	0.00495851	0.745114	0.863213
self_attention	pruning_magnitude	0.530754	0.117824	0.00346378	0.747677	0.864843
self_attention	pruning_magnitude	0.441184	0.0700403	0.00296347	0.765105	0.874789
self_attention	quantization_int8	0.545552	0.0479814	1.63923e-05	0.999647	0.999824
self_attention	quantization_int8	0.493135	0.0711562	1.7481e-05	0.999593	0.999797
self_attention	quantization_int8	0.539973	0.00226416	1.6899e-05	0.999555	0.999778
self_attention	quantization_int8	0.447587	0.0816298	1.01318e-05	0.999598	0.9998
self_attention	quantization_int8	0.556039	0.00549422	8.9275e-06	0.999583	0.999792
self_attention	quantization_int8	0.493984	0.00935927	1.0123e-05	0.99956	0.999781
self_attention	quantization_int8	0.472683	0.00516047	7.66996e-06	0.999606	0.999803
self_attention	quantization_int8	0.506035	0.057673	5.4124e-06	0.999606	0.999803
self_attention	quantization_int8	0.42709	0.0189024	5.53518e-06	0.999561	0.999781

16.4 Interpretação preliminar

- dense_projection: MSE da quantização 9.64926e-05 e do pruning 0.0715716; evidência compatível com H1 nesta amostra.
- self_attention: MSE da quantização 1.09525e-05 e do pruning 0.00643136; evidência compatível com H1 nesta amostra.
- Pruning usa matriz densa e não usa kernel esparsa; quantização é dequantizada para float32 e não usa kernel INT8.
- Razões de latência e memória estão nos CSVs de análise, mas só podem sustentar alegação de hardware se o experimento for principal e o caminho executado tiver validade hardware.
- Resultados de smoke ou diagnóstico CPU não são estimativas finais de desempenho no hardware-alvo.

16.5 Cadeia de evidência

Pergunta → H1 → configuração versionada → CSVs por execução → checksums/hashes → resumo com média e DP → interpretação limitada.

16.6 Reprodução

```
$env:PYTHONPATH = "fases\01_validacao_conceitual"
.\.venv\Scripts\python.exe -m validacao.benchmark_operacoes --config <config.json>
↪ --output-dir <diretorio-novo>
.\.venv\Scripts\python.exe -m validacao.analise_benchmark_operacoes --evidence-dir
↪ <diretorio-novo>
```

Auditoria do estado da IC - 25/08/2026

17.1 Objetivo e fontes examinadas

Esta auditoria foi feita antes de ampliar a infraestrutura experimental. Foram examinados o histórico Git completo, o código ativo, os testes, a documentação metodológica, o protótipo legado, os materiais-fonte da apresentação, o plano formal de trabalho e a reunião de 31/07/2026. Arquivos binários de apresentação foram confrontados por seus fontes textuais e previews; o .docx do plano foi extraído estruturalmente. A renderização visual do .docx não pôde ser feita porque o ambiente não possui LibreOffice.

As fontes não têm o mesmo papel:

1. o plano formal define objetivo, operações, grade e protocolo submetidos;
2. a fala do professor delimita a sequência de validação e o foco prático;
3. a apresentação registra publicamente o desenho pretendido;
4. o código e os artefatos mostram o que existe de fato;
5. decisões deste diagnóstico são interpretações metodológicas e são marcadas como tais.

17.2 Estado comprovado

Área	Evidência observada	Avaliação
Operações matemáticas	Projeção densa, atenção, máscaras, multi-head, FLOPs, métricas e KV cache possuem testes determinísticos.	Base conceitual sólida no recorte testado.
Arquitetura auxiliar	SimplifiedTransformer Block contém Q/K/V, atenção, posição, residual, normalização e FFN.	É bloco simplificado; não é arquitetura Transformer completa.

Área	Evidência observada	Avaliação
Validação independente	NumPy manual, PyTorch manual, PyTorch SDPA e Keras/TensorFlow usam os mesmos tensores em três casos.	9/9 comparações passaram; erro absoluto máximo 2. 38418579e-07.
Suíte	Comando canônico executado em 25/08/2026.	32 passed, 1 skipped; O skip exige CUDA.
Pipeline piloto	Runner de bloco, schema CSV e análise existem e possuem teste curto em CPU.	Útil como protótipo, mas não atende sozinho ao protocolo principal.
Evidências versionadas	A comparação entre frameworks possui CSV, Markdown e metadados.	Adequada para correção numérica, não para desempenho.
Hardware atual	Apenas Intel Iris Xe foi detectada; nvidia-smi ausente e PyTorch reporta zero GPUs CUDA.	A coleta principal na RTX 4070 Ti Super não pode ser feita nesta máquina.

17.3 Correspondência com a orientação

Atendidos:

- validação antes da otimização;
- comparação do mesmo núcleo com cálculo independente, PyTorch e TensorFlow/Keras;
- produção de uma tabela auditável;
- delimitação explícita de que a implementação não é uma Transformer completa;
- documentação da reunião e início da redação metodológica.

Parcialmente atendidos:

- o professor indicou self-attention como foco, mas o runner ativo mede um bloco inteiro;
- existe automação de tabelas e gráficos, mas ainda não há coleta canônica de desempenho;
- a escrita técnica é extensa, porém faltavam diário, checklist operacional e ligação explícita

entre auditoria e decisões.

Ainda não atendidos:

- benchmark principal das operações no hardware-alvo;
- duas execuções independentes com consolidação de média e desvio-padrão;
- relatório de resultados de desempenho produzido durante a coleta.

17.4 Inconsistências e riscos encontrados

17.4.1 Escopo experimental

O runner atual mede `SimplifiedTransformerBlock`, enquanto o plano formal define projeção densa e scaled dot-product attention e a reunião favorece a self-attention isolada. Medir o bloco mistura FFN, normalizações e residuais com o objeto de interesse.

Decisão interpretativa: o benchmark principal terá duas operações isoladas: projeção densa, preservada por estar no plano formal, e self-attention de cabeça única com projeções Q/K/V/saída, tratada como foco principal da orientação. O bloco completo continuará apenas como evidência arquitetural e piloto.

17.4.2 Parâmetros divergentes

O plano e os materiais de apresentação registram seed 42, lote 1, $L=\{64,128,256\}$, $D=\{128,256,512\}$, 20 warm-ups, 50 medições e duas execuções. O runner ativo usa por padrão seed 2026, $L=\{4,8\}$, $D=8$, 5 warm-ups, 20 medições e não identifica duas execuções independentes. Esses defaults são de teste/piloto e não podem ser confundidos com o protocolo principal.

17.4.3 Cenários e alegações de hardware

O plano principal contém baseline, pruning por magnitude de 50% e quantização linear INT8. O runner adiciona `torchao_int8` por padrão, embora esse cenário não esteja no primeiro desenho formal. Além disso:

- a poda atual mantém máscara e armazenamento denso, portanto não prova ganho físico;
- a quantização manual dequantiza os pesos para `float32` antes da operação, portanto mede fidelidade numérica, não kernel INT8;
- latência desses caminhos pode ser registrada como diagnóstico, mas não como evidência de aceleração por hardware;
- `max_memory_bytes` em CPU representa apenas uma estimativa incompleta e não deve ser chamado de pico real.

17.4.4 Reprodutibilidade e rastreabilidade

- as dependências comuns usam limites mínimos, não versões fixas;
- o runner registra poucas informações de Python, sistema e bibliotecas;
- resultados padrão vão para resultados/, pasta ignorada pelo Git;
- não há configuração versionada para smoke e experimento principal;
- não há manifesto de execuções, hashes de entradas/saídas ou log persistente;
- o schema é verificado por nomes de colunas, mas ainda não rejeita todos os campos numéricos não finitos ou semanticamente inválidos;
- a análise atual escolhe um único baseline por configuração e não calcula incerteza entre execuções independentes.

17.4.5 Viés de medição

Os cenários são medidos sempre na mesma ordem. Aquecimento, frequência, temperatura e carga do sistema podem introduzir viés de ordem. A infraestrutura principal deve registrar e variar deterministicamente essa ordem entre as duas execuções.

17.4.6 Cronograma

O plano declara vigência de maio a setembro de 2026, mas uma célula do cronograma traz '2º semestre de 2027' para agosto/setembro. A reunião menciona novembro de maneira incerta. Isso parece uma inconsistência documental, mas a data institucional precisa ser confirmada externamente; o arquivo formal não foi alterado.

17.5 Conclusão da auditoria

A base matemática e a validação cruzada permitem sair da validação conceitual. Ainda não permitem iniciar a coleta principal com o runner atual. Antes disso é obrigatório alinhar o objeto medido, versionar configurações e evidências, representar as duas execuções, fortalecer validações e provar a pipeline em um smoke test. A coleta de GPU permanece condicionada à disponibilidade do hardware-alvo.

Checklist pré-benchmark

Atualizado em 14/09/2026. 'Benchmark principal' significa a coleta destinada a sustentar conclusões de desempenho no hardware-alvo. Smoke tests em CPU são permitidos para validar a infraestrutura, desde que sejam rotulados como tal. A validação conceitual, o benchmark GPU v1, a robustez sintética v3 e a bateria física v3 já foram concluídos; este checklist agora controla a coleta final dos modelos OPT.

18.1 Obrigatório antes do benchmark principal

- **[OBRIGATÓRIO ANTES DO BENCHMARK]** Arquivar a reunião e separar pedidos explícitos, decisões, dúvidas e interpretações.
- **[OBRIGATÓRIO ANTES DO BENCHMARK]** Validar a atenção manual contra referência matemática, PyTorch e TensorFlow/Keras com as mesmas entradas.
- **[OBRIGATÓRIO ANTES DO BENCHMARK]** Registrar tabela, tolerâncias, versões e limites da validação cruzada.
- **[OBRIGATÓRIO ANTES DO BENCHMARK]** Fixar o recorte: projeção densa como operação prevista no plano e self-attention como foco principal; bloco Transformer completo fora da coleta principal.
- **[OBRIGATÓRIO ANTES DO BENCHMARK]** Fixar cenários principais: baseline, pruning por magnitude de 50% e quantização INT8 simulada.
- **[OBRIGATÓRIO ANTES DO BENCHMARK]** Implementar runner das operações isoladas sem alterar o schema CSV existente.
- **[OBRIGATÓRIO ANTES DO BENCHMARK]** Versionar configurações distintas de smoke e coleta principal.
- **[OBRIGATÓRIO ANTES DO BENCHMARK]** Codificar e testar o contrato formal: seed 42, lote 1, grade $L=\{64,128,256\}$, $D=\{128,256,512\}$, 20 warm-ups, 50 medições e duas execuções independentes. A execução da grade continua condicionada à GPU.
- **[OBRIGATÓRIO ANTES DO BENCHMARK]** Registrar Python, bibliotecas, sistema, dispositivo, parâmetros, ordem dos cenários e método de percentil.
- **[OBRIGATÓRIO ANTES DO BENCHMARK]** Persistir CSV, metadados, manifesto, log e hashes de entradas/saídas em pasta versionável.
- **[OBRIGATÓRIO ANTES DO BENCHMARK]** Rejeitar shapes, dtypes, métricas ou resultados inválidos, NaN e infinitos.

- **[OBRIGATÓRIO ANTES DO BENCHMARK]** Corrigir e documentar a inicialização dos pesos-base: o diagnóstico v1 mostrou que $N(0,1)$ faz a escala da saída crescer com a dimensão e prejudica a comparação de MSE/MAE.
- **[OBRIGATÓRIO ANTES DO BENCHMARK]** Fazer smoke test ponta a ponta, incluindo análise, gráficos e reexecução determinística.
- **[OBRIGATÓRIO ANTES DO BENCHMARK]** Confirmar que o dispositivo da coleta principal é CUDA e registrar a RTX 4070 Ti Super. A GPU e o runtime NVIDIA no Docker WSL2 foram confirmados pelo probe; smoke e bateria física foram executados com a GPU ociosa.

18.2 Recomendado antes do benchmark principal

- **[RECOMENDADO ANTES DO BENCHMARK]** Variar deterministicamente a ordem dos cenários entre execuções para reduzir viés de ordem.
- **[RECOMENDADO ANTES DO BENCHMARK]** Registrar snapshot exato do ambiente instalado, sem transformar o arquivo de requisitos mínimos em um lock específico desta máquina.
- **[RECOMENDADO ANTES DO BENCHMARK]** Fazer sanity checks de crescimento de FLOPs e shapes antes da grade completa.
- **[RECOMENDADO ANTES DO BENCHMARK]** Consolidar as duas execuções com média, desvio-padrão e resultados individuais preservados.
- **[RECOMENDADO ANTES DO BENCHMARK]** Confirmar datas institucionais e corrigir, no canal apropriado, a inconsistência 2026/2027 do cronograma.

18.3 Pode ser feito depois dos primeiros benchmarks

- **[AMPLIAÇÃO V3 EM COLETA]** Inferência com OPT-125M, OPT-350M e OPT-1.3B pré-treinados, com pesos e dataset armazenados e smoke funcional aprovado.
- **[PODE SER FEITO DEPOIS]** Benchmark de bloco Transformer completo.
- **[AMPLIAÇÃO V3 CONCLUÍDA NAS OPERAÇÕES]** Cenários TorchAO INT8 e pruning 2:4 confirmados pelo profiler na bateria física de 1.650 registros.
- **[AMPLIAÇÃO V3 CONCLUÍDA]** Cinco seeds, três repetições, múltiplos lotes, cabeças, larguras, sequências e distribuições foram avaliados na bateria sintética de 2.340 registros.
- **[PODE SER FEITO DEPOIS]** Comparação direta de desempenho entre TensorFlow e PyTorch; TensorFlow é apenas referência de correção nesta etapa.

18.4 Regra de liberação

O benchmark físico de operações foi liberado e concluído após a aprovação da imagem fixada, do probe e do smoke. A coleta OPT final mantém o mesmo portão: jogo fechado, uso médio de cinco amostras abaixo de 10%, até 2.048 MiB de VRAM ocupada e temperatura abaixo de 65 graus Celsius. Em WDDM, leitura residual só é aceita com processos abaixo de 10%, potência até 35 W e clock até 300 MHz, mantendo os limites de memória e temperatura. O pico também é registrado no diagnóstico. Falhas e técnicas incompatíveis devem ser preservadas, sem alegação de desempenho de hardware quando o kernel correspondente não for confirmado.

Relatório da execução autônoma - 25/08/2026

19.1 Resumo executivo

A validação numérica da atenção está concluída, a metodologia pré-benchmark foi auditada e o pipeline de operações isoladas foi implementado, testado e usado em três coletas CPU versionadas. Todos os portões controláveis pelo repositório estão fechados. A coleta principal está corretamente bloqueada porque esta máquina não possui CUDA nem a RTX 4070 Ti Super prevista no plano.

O diagnóstico maior revelou uma ameaça à validade - pesos $N(0,1)$ faziam a escala da saída crescer com D . O resultado original foi preservado, o gerador foi corrigido para Xavier/bias zero e a coleta foi repetida. A hipótese H1 (quantização preserva melhor a saída que pruning de 50%) foi compatível nos casos executados, mas ainda é preliminar e não equivale a ganho de hardware.

19.2 O que foi feito

1. Auditoria do repositório, cinco commits preexistentes, código ativo/legado, documentação, plano formal, apresentação, reunião e ambiente.
2. Arquivamento da transcrição integral e reconstrução classificada da orientação, separando pedidos, decisões, dúvidas e interpretações.
3. Validação NumPy/PyTorch/TensorFlow-Keras do núcleo da atenção: 9/9 aprovada.
4. Criação do diário, checklist pré-benchmark e diagnóstico metodológico.
5. Correção da semântica de hardware do runner piloto.
6. Novo runner de projeção densa/self-attention com configuração versionada, lotes independentes, logs, hashes, manifesto e recusa de GPU incorreta.
7. Análise com checksums, baseline por execução, média, desvio-padrão, relatório e gráficos.
8. Smoke CPU ponta a ponta.
9. Diagnóstico CPU $L=64$, $D=128$ v1, investigação da escala dos pesos, correção metodológica e repetição v2.
10. Suíte final observada: 45 passed, 1 skipped; pip check sem problemas.

19.3 Commits criados nesta execução

Commit	Etapa lógica
6e10da0	Reunião, transcrição e direcionamentos do professor.
2a42a5e	Validação da atenção entre PyTorch e TensorFlow/Keras.
49f081f	Auditoria, diário e portões pré-benchmark.
cedbeb2	Correção das alegações de hardware do piloto.
7113fa5	Runner reproduzível das operações isoladas.
bcea919	Captura do ambiente antes de escrever artefatos.
a0aebe5	Análise entre execuções e validação de integridade.
1474fb5	Smoke CPU versionado.
df061d8	Configuração do diagnóstico CPU representativo.
cc03231	Diagnóstico v1 e registro do risco de inicialização.
bf7b4bd	Pesos Xavier e schema de configuração v2.
4cd1ebf	Diagnóstico v2 e confronto metodológico v1/v2.

Este relatório constitui a etapa final de handoff e será versionado em commit separado após a verificação final.

19.4 Orientações recuperadas do professor

Pedidos explícitos:

- validar a implementação contra TensorFlow/Keras usando a mesma entrada e semente antes de otimizar;

- comparar equação/manual, PyTorch e TensorFlow em tabela ou gráfico;
- só depois escolher uma ou duas técnicas e comparar os resultados;
- concentrar o trabalho na self-attention e nas matrizes de maior custo, não em uma Transformer completa;
- escrever texto corrido e fechar avaliações com resultados/tabelas desde já.

Decisões ou sugestões condicionais:

- uma operação interna equivalente basta se a camada completa ocultar detalhes;
- inferência com modelo do Hugging Face é extensão dependente de tempo;
- novembro foi citado de forma incerta e não substitui a data institucional.

Interpretações posteriores, não atribuídas ao professor:

- manter projeção densa junto da self-attention por constar no plano formal;
- usar NumPy como referência matemática;
- escolher Keras Ops, tolerâncias, schema de artefatos e níveis de validade;
- usar Xavier/bias zero para controlar a escala entre dimensões.

19.5 Pré-benchmark

Portão	Status	Evidência
Reunião reconstruída sem inventar requisitos	Concluído	docs/reunioes/2026-07-31/.
Atenção validada entre referências	Concluído	9/9 em evidencias/comparacao_frameworks/.
Recorte e cenários fixados	Concluído	Projeção densa, self-attention, baseline/pruning/quantização.
Configuração formal codificada	Concluído	Seed 42, grade, 20/50, duas execuções.
Rastreabilidade e integridade	Concluído	Config, ambiente, log, manifesto, SHA-256 e fingerprints.
Análise entre execuções	Concluído	Baseline da mesma execução, média e DP.

Port6o	Status	Evid6ncia
Inicializaao contro-lada	Conclu6do	Schema v2, Xavier e confronto v1/v2.
Smoke ponta a ponta	Conclu6do	12 registros e reprodu-tibilidade aprovada.
GPU-alvo dispon6vel	Bloqueado externa-mente	Intel Iris Xe; nvidia-smi ausente; CUDA count 0.

19.6 Benchmarks e valida6es executados

19.6.1 Comparaa6o da atena6o entre frameworks

- Objetivo: provar correaa6o do n6cleo antes de otimizar.
- Configuraaa6o: tr6s casos, CPU/float32, seed 2026, atol=1e-6, rtol=1e-5.
- Resultado: 9/9; erro absoluto m6ximo 2.38418579e-07.
- Interpretaaa6o: equival6ncia num6rica nos casos testados; nenhuma comparaaa6o de desem-penho.

19.6.2 Smoke CPU

- Objetivo: validar pipeline completa.
- Configuraaa6o: B=1, L=4, D=8, 2 warm-ups, 5 medi6es, 2 execu6es.
- Resultado: 12 registros, checksums e hashes aprovados; gr6ficos leg6veis.
- Interpretaaa6o: infraestrutura aprovada; tempos muito vari6veis e sem valor de hardware. Quantizaao teve MSE menor que pruning nas duas opera6es.

19.6.3 Diagn6stico CPU v1

- Objetivo: executar um ponto formal com 20/50 e duas execu6es.
- Configuraaa6o: B=1, L=64, D=128, pesos $N(0, 1)$.
- Resultado: H1 compat6vel, mas MSE da atena6o quantizada 732.65.
- Interpretaaa6o: resultado leg6timo que revelou escala inadequada do baseline; n6o foi des-cartado nem 'melhorado'.

19.6.4 Diagn6stico CPU v2

- Objetivo: repetir o mesmo ponto ap6s controlar a inicializaao.

- Configuração: mesmos parâmetros, pesos Xavier e bias zero.
- Resultado: na projeção, MSE $7.7789e-05$ (quantização) contra 0.0727207 (pruning); na atenção, $1.63925e-05$ contra 0.0110127 . R^2 /cosseno da atenção quantizada: $0.999647/0.999824$.
- Interpretação: H1 compatível nesse ponto; escala absoluta controlada. Latência CPU abaixo do baseline em algumas barras não autoriza aceleração devido ao ruído e à ausência de kernels especializados.

19.7 Problemas e riscos encontrados

- GPU-alvo indisponível nesta máquina.
- Runner anterior media bloco inteiro, divergindo do recorte de operações.
- Defaults anteriores divergiam da seed/grade/20/50/duas execuções do plano.
- Quantização manual executa pesos dequantizados float32; pruning permanece denso. Nenhum dos dois sustenta hoje alegação de kernel INT8/esparso.
- Memória em CPU é estimativa de armazenamento dos tensores, não pico real.
- Latências pequenas apresentaram alta variabilidade; duas execuções são o mínimo formal, não uma amostra estatística ampla.
- Requisitos comuns têm versões mínimas; cada coleta registra o snapshot exato.
- O plano declara maio-setembro de 2026, mas o cronograma contém uma célula com '2º semestre de 2027'; a reunião menciona novembro sem confirmação.
- O .docx formal foi extraído estruturalmente, mas não renderizado visualmente porque LibreOffice não está disponível.

19.8 Pendências

19.8.1 Críticas

- executar a grade principal na RTX 4070 Ti Super com CUDA;
- antes de alegar ganho físico, adotar e validar formato/kernel realmente INT8 e/ou esparso; os cenários simulados atuais só sustentam validade numérica ou algorítmica.

19.8.2 Importantes

- confirmar o calendário institucional e a inconsistência 2026/2027;
- incorporar tabelas e discussão v1/v2 ao texto acadêmico;
- após a coleta GPU, revisar ruído, outliers e suficiência de duas execuções;
- decidir, com o orientador, se a entrega formal manterá quantização/pruning simulados ou incluirá uma extensão de kernel real.

19.8.3 Melhorias futuras

- modelo pré-treinado do Hugging Face;
- TorchAO/kernel INT8 e esparsidade estruturada em etapa própria;
- mais seeds/lotos/heads e intervalos de confiança;
- benchmark do bloco completo somente como extensão;
- comparação de desempenho entre frameworks, separada da validação numérica.

19.9 Próximo passo recomendado

Mover o repositório para a máquina com RTX 4070 Ti Super, instalar o mesmo ambiente, rodar primeiro a suíte e o smoke v2, confirmar o nome da GPU e então executar experimentos/benchmark_principal_gpu.json em pasta nova. A coleta deve ser analisada imediatamente e não pode ser chamada de ganho INT8/esparsa sem uma segunda etapa que comprove armazenamento e kernel compatíveis.

Diário da iniciação científica

Este diário registra a evolução da pesquisa, as razões das decisões e as evidências produzidas. Entradas anteriores à criação do arquivo são marcadas como reconstruídas a partir de fontes versionadas.

20.1 31/07/2026 - Reunião de orientação (entrada reconstruída)

Objetivo e contexto. Revisar como validar o código já desenvolvido e definir o caminho até os experimentos.

Análises e decisões. O orientador pediu comparação com TensorFlow/Keras e PyTorch usando entradas e sementes equivalentes antes de qualquer otimização. Ficou definido que resultados pequenos deveriam virar tabelas/gráficos e que a self-attention seria o foco, evitando o custo de otimizar a arquitetura inteira. Uma ou duas técnicas seriam escolhidas apenas após esse portão. A escrita do relatório deveria começar imediatamente.

Hipótese metodológica. Se o mesmo núcleo produzir saídas equivalentes na implementação manual e em bibliotecas consolidadas, diferenças posteriores podem ser atribuídas com mais segurança às técnicas, e não a um erro básico da atenção.

Pendências deixadas. Comparação independente, definição detalhada dos benchmarks, tabelas de resultados, redação e confirmação das datas oficiais.

Fonte. Ata e transcrição em docs/reunioes/2026-07-31/.

20.2 24/08/2026 - Validação cruzada da atenção (entrada reconstruída)

Objetivo. Fechar o portão numérico solicitado na reunião.

Implementação. Foi criado `validacao.comparacao_frameworks` com três casos determinísticos e os mesmos tensores NumPy compartilhados entre NumPy manual, PyTorch manual, PyTorch SDPA e Keras/TensorFlow. O adaptador trata os layouts $[B, H, T, D]$ e $[B, T, H, D]$. TensorFlow é importado tardiamente e executado em CPU, `float32`, sem dropout ou flash attention.

Testes e resultados. As nove comparações passaram com `atol=1e-6` e `rtol=1e-5`; erro

absoluto máximo $2.38418579e-07$. A suíte completa retornou 32 passed, 1 skipped; o teste ignorado exige CUDA. CSV, tabela Markdown e JSON de metadados foram versionados em `evidencias/comparacao_frameworks/`.

Interpretação. A evidência sustenta equivalência numérica do núcleo nos casos testados. Não sustenta velocidade, economia de memória ou superioridade entre frameworks.

Próximo passo. Auditar e alinhar o runner de desempenho ao plano formal e ao recorte da reunião antes de executar benchmarks principais.

20.3 25/08/2026 - Auditoria integral e portões pré-benchmark

Objetivo. Revisar criticamente repositório, histórico, código, documentação, plano formal, apresentação, reunião e ambiente antes de implementar nova coleta.

O que foi analisado. Cinco commits preexistentes, código ativo e legado, testes, requisitos, schemas, análise de resultados, materiais textuais da apresentação, plano de trabalho e transcrição. O plano foi extraído estruturalmente; a verificação visual do .docx ficou indisponível pela ausência de LibreOffice.

Problemas encontrados. O runner ativo mede bloco inteiro; usa parâmetros de piloto diferentes do plano; não representa duas execuções; grava por padrão em pasta ignorada; registra ambiente insuficiente; usa ordem fixa de cenários; tem semântica ambígua de memória; e inclui TorchAO por padrão fora do primeiro desenho formal. A análise atual não estima variabilidade entre execuções.

Decisões metodológicas. O benchmark principal medirá projeção densa e self-attention isoladas. A primeira preserva o plano formal; a segunda é o foco explicitado pelo professor. O bloco simplificado continuará como validação arquitetural/piloto. O schema CSV atual será preservado, e informações adicionais irão para configuração, manifesto, log e metadados.

Ambiente. Python 3.11.9, PyTorch 2.11.0+cu128 e build CUDA 12.8, mas `torch.cuda.is_available()` retornou falso, com zero dispositivos. O Windows só expôs Intel Iris Xe; `nvidia-smi` não está instalado/disponível.

Testes. Suíte completa: 32 passed, 1 skipped. Comparação canônica: 9/9 aprovada. `pip check`: nenhuma dependência quebrada.

Correção do runner piloto. A evidência de armazenamento de baixa precisão da quantização manual foi removida: o `int8` é intermediário e o peso executado permanece `float32`. TorchAO também saiu dos cenários padrão, permanecendo como extensão explícita. O metadado agora diferencia pico CUDA de estimativa de armazenamento em CPU. Teste direcionado: 7 passed, 1 skipped (CUDA).

Resultados e interpretação. A validação conceitual está madura, mas o benchmark

principal ainda não está liberado. A indisponibilidade da GPU é externa; os demais portões controláveis serão tratados antes do smoke test.

Arquivos criados/alterados nesta etapa. docs/diario_ic.md, docs/auditoria_estado_2026-08-25.md, docs/checklist_pre_benchmark.md, índice de documentação e reconstrução classificada da reunião.

Próximos passos. Implementar configuração e runner das operações isoladas, fortalecer validações, persistir evidências, executar smoke reproduzível e então reavaliar a liberação da coleta principal.

20.3.1 Continuação - infraestrutura das operações isoladas

Implementação. Foram adicionadas configurações JSON separadas para smoke em CPU e experimento principal em CUDA. `validacao.benchmark_operacoes` carrega e valida o contrato, gera dados sintéticos determinísticos, mede projeção densa e self-attention de cabeça única e executa baseline, poda de 50% e quantização INT8 simulada. Cada execução independente grava seu próprio CSV e metadados; um manifesto com checksums e um log permitem preservar resultados parciais.

Controles metodológicos. O dispositivo não usa auto; o benchmark principal falha se CUDA ou a GPU nominal estiverem ausentes. As duas execuções reutilizam os mesmos dados/pesos e rotacionam deterministicamente a ordem dos cenários. Os hashes confirmam entradas e saídas determinísticas. A poda continua densa e a quantização executa pesos dequantizados em float32, fato registrado nos metadados para impedir alegações de kernel otimizado.

Teste. `test_benchmark_operacoes.py`: 6 passed. O teste verifica o contrato formal, o portão de GPU, a poda exata, a semântica da quantização, a reprodutibilidade entre execuções e todos os artefatos persistidos.

Pendência imediata. Consolidar execuções com média/desvio-padrão, integrar a análise à pipeline e só então produzir a evidência canônica do smoke test.

20.3.2 Continuação - análise entre execuções

Implementação. `validacao.analise_benchmark_operacoes` verifica os SHA-256 do manifesto, valida a completude da grade, confere hashes determinísticos e associa cada candidato ao baseline da mesma execução. A análise preserva os registros individuais, calcula média e desvio-padrão entre execuções e gera comparações, relatório e gráficos.

Proteção contra interpretação indevida. O relatório identifica o propósito do experimento. Para smoke em CPU, tempos e memória são rotulados como diagnóstico da pipeline. A cadeia pergunta → hipótese → configuração → execução → resultado → interpretação fica explícita, e caminhos densos/dequantizados não são promovidos a hardware.

Testes. Dois testes direcionados passaram: geração completa dos oito artefatos de

análise e rejeição de CSV adulterado por divergência de checksum.

Próximo passo. Fazer commit desta infraestrutura, executar o smoke canônico em diretório versionável novo, inspecionar seus artefatos e repetir a análise.

20.3.3 Continuação - smoke canônico e portão da GPU

Objetivo. Validar ponta a ponta leitura da configuração, geração dos dados, execução, métricas, persistência, logs, integridade, análise e visualização antes de qualquer grade maior.

Configuração. CPU, seed 42, B=1, L=4, D=8, projeção densa e self-attention, três cenários, 2 warm-ups, 5 medições e 2 execuções independentes. Evidência em `evidencias/benchmarks/smoke_cpu_2026-08-25/`.

Resultados da pipeline. Manifesto completo; 6 registros por execução e 12 no total; CSVs e metadados passaram nos checksums; hashes de entrada e saída coincidiram entre execuções; oito artefatos de análise foram gerados; os dois gráficos foram inspecionados e estão legíveis.

Qualidade da saída. Na projeção densa, MSE de $7.28672e-05$ para quantização contra 0.352201 para pruning. Na self-attention, MSE de 0.00503124 contra 25.8574 . O sinal é compatível com H1 nesta amostra sintética: a quantização simulada preservou melhor a saída que a poda de 50%.

Latência e limitação. Os casos minúsculos apresentaram grande variabilidade entre as duas execuções. A quantização simulada também executa dequantização em `float32`, e a poda usa matriz densa. Portanto, razões de latência do smoke não são evidência de aceleração nem de hardware.

Portão negativo da GPU. A tentativa controlada com a configuração principal terminou com código 2 e mensagem de CUDA indisponível, antes de criar qualquer diretório de saída. Isso confirma que o runner não fará coleta principal acidental em CPU.

Decisão. O smoke está aprovado. A grade principal continua bloqueada apenas pela ausência da RTX/CUDA neste ambiente. Como trabalho independente, será executado um diagnóstico CPU em um ponto representativo da grade formal, sem promovê-lo a resultado de hardware.

20.3.4 Continuação - diagnóstico CPU v1 em L=64, D=128

Objetivo e configuração. Exercitar o primeiro ponto da grade formal com o protocolo completo de 20 warm-ups, 50 medições e duas execuções, preservando seed 42 e os mesmos cenários. A coleta terminou em aproximadamente dois segundos de tempo registrado pelo log e gerou 12 registros analisáveis.

Integridade e reprodutibilidade. Manifesto completo, checksums válidos e hashes de entradas/saídas idênticos entre execuções. Os gráficos foram inspecionados e estão legíveis.

Resultado de qualidade. Na projeção densa, MSE 0.00995697 da quantização contra 9.30825 do pruning. Na self-attention, MSE 732.65 contra 12144.35; R^2 da quantização 0.95428 e cosseno 0.97714. H1 continua compatível, mas o erro absoluto chamou atenção.

Investigação do resultado inesperado. O gerador v1 usa pesos independentes $N(0, 1)$. Em matrizes maiores, essa variância faz projeções e saída da atenção crescerem com D. Assim, MSE/MAE aumentam também pela escala do baseline e não podem ser comparados diretamente entre dimensões. Não se trata de adulterar um resultado ruim: a coleta v1 foi preservada e identificou uma ameaça à validade.

Decisão metodológica. Antes do benchmark principal, tornar a distribuição dos dados explícita na configuração e usar inicialização de pesos escalada pela dimensão (Xavier para matrizes quadradas), mantendo entradas $N(0, 1)$ e bias zero. O diagnóstico será repetido como v2; v1 continuará versionado como evidência da correção.

Latência. As razões em CPU sugeriram valores abaixo do baseline em alguns cenários, mas com desvio alto e sem kernel esparsos/INT8. Nenhuma aceleração foi concluída.

20.3.5 Continuação - correção metodológica da inicialização

Mudança. O schema de configuração passou à versão 2 e agora registra explicitamente distribuição das entradas, inicialização dos pesos e bias. As entradas continuam $N(0, 1)$; matrizes quadradas usam Xavier normal com desvio $1/\sqrt{D}$; o bias da projeção densa é zero. Evidências de schema v1 continuam legíveis para preservar o histórico.

Justificativa. A mudança não foi escolhida para melhorar uma métrica observada, mas para controlar a variância das projeções ao comparar diferentes dimensões. A coleta v1 permanece intacta e será confrontada com v2.

Testes. A suíte passou com 45 passed, 1 skipped. Um teste novo verifica a escala empírica dos pesos e o bias zero; as evidências v1 foram reabertas e mantiveram checksums e reprodutibilidade válidos.

Próximo passo. Commit da correção e repetição do diagnóstico em nova pasta v2, sem sobrescrever o resultado anterior.

20.3.6 Continuação - diagnóstico CPU v2 e confronto com v1

Execução. A configuração v2 repetiu B=1, L=64, D=128, 20 warm-ups, 50 medições e duas execuções com pesos Xavier e bias zero. Manifesto, checksums, hashes determinísticos, relatório e gráficos passaram. O ambiente foi capturado com worktree limpo no commit da correção.

Resultado. Na projeção densa, MSE da quantização 7.7789e-05 contra 0.0727207 do pruning. Na self-attention, 1.63925e-05 contra 0.0110127; R^2 /cosseno da quantização ficaram em 0.999647/0.999824. H1 foi novamente compatível neste ponto da grade.

Confronto. A queda do MSE da self-attention quantizada de 732.65 (v1) para 1.63925e-05 (v2) confirma que a escala anterior dominava o erro absoluto. O resultado v1 foi pre-

servado. A decisão, alternativas e limites foram registrados em docs/decisao_inicializacao_benchmark.tex.

Latência. As barras ficaram abaixo do baseline em CPU, mas com grande variabilidade e sem caminhos físicos otimizados. Esses números permanecem sem interpretação de aceleração.

Estado do portão. Todas as pendências obrigatórias controláveis pelo código estão concluídas. Resta a condição externa: executar a grade principal na RTX 4070 Ti Super/CUDA e confirmar o calendário institucional.

20.4 29/08/2026 - Migração integral da documentação para LaTeX

Objetivo. Tornar LaTeX a fonte canônica de toda a documentação acadêmica, personalizar o volume final e preparar o repositório para ser clonado na máquina com a GPU alvo.

Reorganização. Fundamentação, método, matriz, auditoria, checklist, diário, reunião, guia de estudo, roteiro e relatórios de evidência foram migrados para arquivos .tex. RE-ADMEs permaneceram em Markdown apenas como índices do GitHub. CSV, JSON, logs e imagens continuam como evidência bruta; a transcrição original e o formulário Word foram preservados como fontes.

Volume compilado. Foi criado um documento mestre com capa, identidade visual, sumário, relatório executivo, plano consolidado, estado atual, roteiro da coleta CUDA, documentação técnica, evidências e anexos. O build usa XeLaTeX em três passagens e publica o PDF em output/pdf/relatorio_ic_transformers.pdf.

Geradores. A comparação entre frameworks, a análise das operações isoladas e a análise do benchmark controlado passaram a emitir relatórios LaTeX em vez de Markdown. Os testes de contrato foram atualizados para verificar a extensão e a estrutura dos novos artefatos.

Próximo passo. Clonar a versão publicada na máquina com a RTX 4070 Ti Super, instalar o ambiente fixado, confirmar CUDA, executar toda a suíte, revalidar a atenção e coletar a grade de experimentos/benchmark_principal_gpu.json em uma pasta nova.

20.5 13/09/2026 - Validação multi-head e robustez sintética v3

Validação conceitual. A comparação entre frameworks foi ampliada de nove para dezoito casos. Além da scaled dot-product attention, passaram a ser comparadas as projeções Q/K/V/O, a separação e recombinação de cabeças e a saída completa da multi-head attention para uma, quatro e oito cabeças. NumPy manual, PyTorch manual, PyTorch SDPA e Keras/TensorFlow produziram resultados equivalentes dentro das tolerâncias pré-registradas. As 18 comparações

foram aprovadas.

Ampliação das entradas. O benchmark sintético v3 variou cinco seeds, 11 perfis arquiteturais, três condições de distribuição e três repetições. Os perfis alteraram batch, comprimento da sequência, dimensão do modelo e número de cabeças. O pruning por magnitude foi medido em 10%, 25%, 50% e 75%, ao lado do baseline FP32 e da quantização INT8 fake por linha.

Coleta. Foram preservados 2.340 registros agregados, 1.950 pares e 117.000 amostras individuais de latência. Em todos os pares, o INT8 fake teve menor MSE que o pruning, e a degradação do pruning cresceu com a sparsity. Como esses cenários ainda usam tensores densos após a transformação, os tempos foram tratados como evidência algorítmica e não como prova de aceleração física.

20.6 14/09/2026 - Docker, kernels físicos e bateria de hardware

Infraestrutura. O Docker Desktop com backend WSL2 foi recuperado. O disco de dados foi realocado para D:\DockerDesktopData e o cache Hugging Face para D:\Caches\scientific-initiation\huggingface. A imagem científica foi construída com PyTorch 2.11/CUDA 12.8, TorchAO 0.17, cuSPARSELt 0.7.1, Transformers 5.17 e dependências fixadas. O cache offline contém OPT-125M, OPT-350M, OPT-1.3B e a revisão escolhida do WikiText-2.

Portões físicos. O probe confirmou a RTX 4070 Ti SUPER, compute capability 8.9, CUDA, Triton, TorchAO INT8 e conversão 2:4. O profiler registrou os operadores INT8 e cuSPARSELt esperados. O INT8 weight-only foi mantido como evidência física de armazenamento, sem ser apresentado automaticamente como computação inteira.

Coleta física. O smoke e a bateria completa terminaram com 1.650 registros, 990 pares e 82.500 medições individuais. Os cinco cenários foram executados nas duas operações e nos 11 perfis, com cinco seeds e três repetições. Manifestos, hashes, pareamentos e artefatos de análise foram verificados antes da promoção para o diretório de evidências.

Resultado. INT8 dinâmico, INT8 weight-only e pruning semiestruturado 2:4 reduziram armazenamento e/ou pico de memória. Entretanto, nenhum dos seis grupos formados por técnica e operação atingiu o critério de speedup: todos os intervalos de 95% ficaram abaixo de 1,0x em relação ao baseline compilado. O resultado negativo foi preservado e limita a conclusão ao hardware, shapes e composição de operações avaliados.

Modelos pré-treinados. O runner OPT foi validado em uma execução diagnóstica com qualidade, perplexidade, logits, geração, prefill, TTFT, decode, memória, armazenamento e profiler. A falha conhecida do decode INT8 dinâmico para batch unitário foi registrada como unsupported, sem eliminar as medições compatíveis. As latências desse diagnóstico não foram promovidas, porque havia concorrência de aplicativos na GPU.

Automação e testes. O script PowerShell ganhou ações Status e Remaining, identificador estável, retomada por hashes, cache explícito em D: e portão de GPU que nunca encerra processos. A suíte no contêiner terminou com 71 passed, 1 skipped; o skip é a integração que exige GPU/download explícito.

Pendência. Restam o smoke OPT limpo e a avaliação final dos três modelos. A execução foi corretamente adiada quando o portão observou 47% de uso e 3.171 MiB de VRAM, acima do limite de 2.048 MiB. A retomada deverá usar o mesmo RunId, sem repetir as baterias sintética e física já concluídas.

20.7 15/09/2026 - Execução final dos modelos OPT

Condições de execução. O comando Remaining retomou o RunId final-20260913. A imagem Docker, o probe de hardware e a bateria física foram reconhecidos como íntegros e não foram repetidos. Antes do smoke OPT, o portão registrou jogo fechado, uso médio de 1,0%, pico de 3%, 1.337 MiB de VRAM, 46 graus Celsius, 9,4 W e clock de 210 MHz. Antes da bateria completa, registrou uso médio de 0,4%, pico de 2%, 1.337 MiB de VRAM, 47 graus Celsius, 10,3 W e clock de 210 MHz.

Cobertura. O smoke OPT final foi aprovado e a bateria completa avaliou facebook/opt-125m, facebook/opt-350m e facebook/opt-1.3b, todos em revisões fixadas e sem treinamento. Foram gerados 45 registros de qualidade, 2.880 janelas de avaliação, 900 métricas de prompts, 1.296 registros de desempenho, 10.080 timings individuais, subconjunto versionado de avaliação, traces de operadores e manifestos de análise. As evidências foram promovidas para fases/01_validacao_conceitual/evidencias/benchmarks/modelos_opt_v1_2026-09-15/ e o smoke para fases/01_validacao_conceitual/evidencias/benchmarks/modelos_opt_smoke_2026-09-15/.

Qualidade. Vinte e cinco das 45 variantes ficaram dentro dos limites operacionais. As quantizações INT8 fake e weight-only permaneceram estáveis nos três modelos. O INT8 dinâmico permaneceu aceitável em OPT-125M e OPT-350M, mas excedeu o limite no OPT-1.3B. O pruning por magnitude em atenção foi aceitável em 10%, ficou acima do limite já em 25% e degradou fortemente em 50% e 75%. O pruning 2:4 degradou qualidade nos três modelos.

Desempenho. Prefill e TTFT foram coletados com registros completos. Nenhuma comparação cumpriu todos os critérios de ganho físico. O caso mais favorável apareceu em blocks_int8_dynamic no OPT-1.3B, com speedup mediano acima de 1 em prefill e TTFT, mas os intervalos de 95% cruzaram o efeito nulo; portanto a tendência foi classificada como mista, não sustentada.

Limitação do decode. Os 288 casos planejados de model_decode foram preservados: 282 como failed e 6 como unsupported. O erro predominante foi a interação entre torch.compile em modo reduce-overhead, CUDA Graphs e atualização do KV cache, com

sobrescrita de saídas capturadas durante a geração token a token. Por isso, a pesquisa passa a ter evidência final para qualidade, prefill, TTFT, memória e armazenamento dos modelos OPT, mas não para latência por token em decode.

Conclusão parcial. A avaliação em modelos pré-treinados confirma a tendência qualitativa principal: INT8 preserva melhor a perplexidade que pruning, e a degradação do pruning cresce com a sparsity. Ela também reforça o resultado físico negativo: reduzir representação e memória não garantiu speedup sustentado. A próxima decisão metodológica é escolher entre corrigir e reexecutar apenas o decode, ou registrar essa limitação como escopo da versão atual do trabalho.

20.8 16/09/2026 - Extensão de escala OPT-2.7B e OPT-6.7B

Motivação. A extensão foi criada para testar a hipótese levantada na discussão: caminhos físicos de otimização poderiam se tornar mais vantajosos em modelos maiores. Foram avaliados facebook/opt-2.7b e facebook/opt-6.7b, sem treinamento ou fine-tuning, mantendo o mesmo protocolo de qualidade, prefill, TTFT, armazenamento, memória e hashes.

Guarda operacional. O OPT-6.7B foi executado com limite de VRAM em 15.800 MiB, retomada por cenário e execução isolada por container. Essa decisão permitiu coletar o modelo maior sem travar a máquina e preservou checkpoints válidos entre tentativas.

Resultado. Cada modelo adicional terminou 15 cenários, 216 registros de desempenho e 1.440 timings. No OPT-6.7B, blocks_int8_dynamic atingiu speedup mediano de 1,756x em prefill e 1,744x em TTFT, com intervalos acima de 1,0. Entretanto, a perplexidade aumentou 165,05%, tornando o ganho físico insuficiente como recomendação final. O blocks_2:4 também acelerou no modelo maior, mas degradou ainda mais a perplexidade.

Interpretação. A escala passou a favorecer alguns kernels físicos, mas não resolveu o conflito entre latência e qualidade. A conclusão correta é que a hipótese de escala ganhou evidência parcial para desempenho, não para utilidade final.

20.9 17/09/2026 - Benchmarks complementares de lacunas

Objetivo. A rodada complementar foi criada para preencher células vazias reais da planilha: desempenho e VRAM dos prunings percentuais e INT8 fake nos modelos OPT, além dos perfis físicos de stress nas operações isoladas. As coletas antigas foram preservadas, e novas evidências foram promovidas com RunId complementar-20260917.

Execução. O script Docker ganhou a ação Complementary, com status/probe, smoke complementar, hardware stress, OPT complementar até 2.7B e OPT-6.7B guardado. O hard-

ware stress produziu 300 registros e 15.000 timings. A bateria OPT de lacunas produziu 40 registros de qualidade, 1.080 registros de desempenho e 7.200 timings, somando 125M, 350M, 1.3B, 2.7B e 6.7B.

Correção técnica. Durante o OPT-6.7B, a coleta chegou a 8/8 cenários, mas o manifesto permaneceu como running porque o runner encerrava após cada checkpoint parcial. O wrapper foi corrigido para fazer uma passagem final sem limite de cenários, fechar status=complete, recalcular checksums e só então executar a análise. Após a correção, todos os manifestos complementares ficaram completos e íntegros.

Resultado científico. O complemento mostrou que pruning denso percentual e INT8 fake nos OPT não reduzem VRAM, pois continuam em caminhos densos. No OPT-6.7B, pruning de 10% em atenção preservou qualidade e ficou praticamente neutro em latência, com 0,9997x em prefill. Pruning de 25% ainda ficou dentro do limite operacional de perplexidade, mas caiu para 0,924x. Os níveis de 50% e 75% degradaram fortemente a perplexidade. Assim, zeros densos servem para estudar degradação numérica, mas não sustentam aceleração física.

Artefatos. A planilha output/spreadsheets/benchmarks_transformers_resumo.xlsx foi regenerada com abas separadas de speedup, VRAM, erro, qualidade, mapa experimental, dicionário e fontes. As células agora usam status explícitos: medido, não medido, não aplicável, unsupported e failed.

Parte IV

Registros e materiais de apoio

Reunião de 31 de julho de 2026

A.1 Identificação

- **Tema:** validação da implementação, definição do recorte experimental e preparação dos benchmarks;
- **Participantes identificados na transcrição:** Lucas Galvão e Walter Silva Oliveira;
- **Horário registrado:** início às 16:01 (GMT-03:00);
- **Duração da transcrição:** 30 min e 7 s;
- **Registro integral compilável:** [transcricao-completa.tex](#);
- **Fonte no Google Docs:** [anotações e transcrição](#);
- **Gravação indicada na ata:** [Google Drive](#).

A transcrição foi gerada automaticamente e pode conter erros de reconhecimento. A cópia integral foi preservada sem correções; esta página concentra apenas a leitura útil para o projeto.

A.2 Leitura executiva

A reunião definiu uma sequência de trabalho: primeiro demonstrar que a implementação local da atenção reproduz referências conhecidas usando as mesmas entradas, pesos e sementes; depois aplicar uma ou duas técnicas de otimização e compará-las por benchmarks controlados. O recorte recomendado foi o mecanismo de autoatenção, sobretudo suas operações matriciais mais custosas, e não uma Transformer completa. Tabelas, gráficos e texto acadêmico devem ser produzidos junto com os experimentos, em vez de ficarem para o final.

A.3 Decisões e direcionamentos

1. **Validar antes de otimizar.** Confrontar a implementação local com cálculo manual e bibliotecas consolidadas, mantendo entrada, pesos, semente e tolerância comparáveis.
2. **Isolar o objeto de estudo.** Priorizar a autoatenção e explicitar que o projeto não pretende implementar nem otimizar toda a arquitetura Transformer.
3. **Transformar testes em evidência.** Registrar resultados em tabelas e gráficos, acompanhados do passo a passo e da fundamentação teórica.

4. **Comparar poucas técnicas de forma controlada.** Depois do portão de validação, selecionar uma ou duas otimizações e medi-las contra o mesmo baseline.
5. **Começar a redação imediatamente.** Conectar conceitos, método, resultados e limitações enquanto os experimentos são executados.
6. **Manter modelos pré-treinados como extensão.** Uma inferência com modelo do Hugging Face pode enriquecer o trabalho, mas somente se houver tempo depois do núcleo experimental.
7. **Confirmar as datas oficiais.** A conversa mencionou novembro como referência provável para entrega/apresentação, mas sem confirmação institucional.

A.4 Reconstrução classificada da orientação

Esta seção distingue o que aparece na fala do orientador das decisões e interpretações feitas depois. Os intervalos remetem à transcrição integral.

A.4.1 Pedidos explícitos do professor

- comparar a implementação local com uma biblioteca consolidada, citando TensorFlow ou Keras, com a mesma entrada e a mesma semente (00:00:00-04:12);
- validar passo a passo antes de aplicar técnicas de otimização (00:02:53-05:35);
- materializar a validação em tabela ou gráfico, comparando cálculo/equação, implementação manual, PyTorch e TensorFlow (00:05:35-08:01);
- depois da validação, escolher uma ou duas técnicas da metodologia e comparar os resultados (00:08:01-09:18);
- concentrar o estudo no mecanismo de self-attention e em suas operações de maior custo, em vez de otimizar uma Transformer inteira (00:09:18-14:09);
- começar a escrever texto corrido imediatamente e fechar as avaliações com resultados e tabelas (00:23:13-24:34).

A.4.2 Decisões tomadas durante a reunião

- a ordem de trabalho ficou como validação da implementação, seleção das técnicas, aplicação controlada e comparação;
- o recorte principal ficou na self-attention, com a limitação de escopo e de tempo usada como justificativa;
- se uma camada completa de TensorFlow ocultasse o núcleo, bastaria comparar uma operação/interação equivalente da atenção (00:14:09-15:21);
- uma demonstração com modelo pré-treinado poderia ser feita somente se houvesse tempo (00:15:21 em diante).

A.4.3 Dúvidas levantadas na reunião

- como garantir que a implementação própria está correta;
- como definir formalmente o que pode ser chamado de Transformer;
- até onde é necessário validar a arquitetura, além das operações matemáticas;
- se TensorFlow expõe o núcleo interno de atenção de forma comparável;
- se haverá tempo para uma inferência com modelo do Hugging Face;
- quais são as datas institucionais efetivas de entrega e apresentação.

A.4.4 Sugestões metodológicas, sem caráter de requisito adicional

- usar resultados pequenos e palpáveis para sustentar cada avanço;
- justificar o recorte pela relevância computacional do mecanismo escolhido e pelo tempo disponível;
- usar literatura para selecionar técnicas já conhecidas, sem exigir uma nova técnica de otimização;
- escrever durante o desenvolvimento para explicitar como os conceitos se encadeiam.

A.4.5 Implementações e validações necessárias derivadas diretamente da reunião

- adaptador de layout e execução do mesmo núcleo de atenção em NumPy/manual, PyTorch e TensorFlow/Keras;
- casos determinísticos com entradas compartilhadas e tolerância explícita;
- artefato tabular com resultados e erros;
- benchmark controlado da self-attention com baseline comum e uma ou duas técnicas, depois da validação numérica;
- documentação em prosa de método, resultados e limites.

A.4.6 Tarefas que podem ficar para depois dos primeiros benchmarks

- demonstração de inferência com modelo pré-treinado;
- ampliação para outros componentes ou para uma arquitetura Transformer completa;
- técnicas adicionais além do primeiro conjunto controlado.

A.4.7 Interpretações posteriores - não são pedidos literais do professor

- usar NumPy como referência matemática independente;
- escolher `keras.ops.nn.dot_product_attention` como API TensorFlow/Keras;
- fixar `float32`, CPU, `atol=1e-6`, `rtol=1e-5` e seed 2026 na validação cruzada;
- manter CSV e JSON de metadados versionados;
- classificar evidências como conceituais, numéricas, algorítmicas ou de hardware;
- tratar a RTX 4070 Ti Super, seed 42, grade, aquecimento e repetições como requisitos do plano formal de trabalho, não desta reunião.

A.5 Situação no repositório em 24/08/2026

Encaminhamento da reunião	Situação atual	Evidência ou lacuna
Validar a matemática da atenção	Implementado no código e nos testes	Há comparação da atenção local com NumPy independente, valores controlados e <code>torch.nn.functional.scaled_dot_product_attention</code> em test_validacao_conceitual.py .
Comparar com bibliotecas consolidadas	Concluído para o núcleo da atenção	NumPy manual, PyTorch manual, PyTorch SDPA e Keras SDPA com backend TensorFlow foram comparados nos mesmos casos. As 9 comparações passaram; veja a evidência versionada .

Encaminhamento da reunião	Situação atual	Evidência ou lacuna
Restringir o escopo à autoatenção	Parcialmente alinhado	A validação isola a atenção, mas o executor principal mede um <code>SimplifiedTransformerBlock</code> ; é preciso decidir se o benchmark final será da atenção isolada ou do bloco simplificado. Ver RE-ADME da fase .
Preparar benchmarks controlados	Infraestrutura pronta; execução canônica pendente	O executor, o esquema do CSV e os testes de contrato existem em benchmark_controlado.py , mas não há resultados experimentais versionados.
Produzir tabelas, gráficos e conclusões	Automação pronta; artefatos pendentes	analise_resultados.py processa os CSVs, mas depende da coleta real.
Formalizar a metodologia	Em andamento avançado	A fase já possui protocolo, matriz de ferramentas, perguntas de pesquisa e documentação de validação em docs da fase .
Usar modelo pré-treinado	Não iniciado e não obrigatório	Deve continuar fora do caminho crítico até que validação, benchmarks e relatório estejam fechados.

Essa tabela descreve o que está implementado e documentado; ela não afirma que a bateria final de benchmarks já foi executada no ambiente-alvo.

A.6 Próximos passos priorizados

A.6.1 P0 - Fechar o contrato experimental

1. Escolher explicitamente o objeto medido no resultado principal: atenção isolada, como favorecido pela reunião, ou bloco Transformer simplificado.
2. Fixar baseline, técnicas de otimização, shapes, lotes, tamanhos de sequência, dtype, dispositivo, sementes, aquecimento, repetições e tolerâncias.
3. Preservar TensorFlow/Keras como portão de correção já concluído e reexecutar a comparação somente quando mudar a atenção, as versões dos frameworks ou as tolerâncias.
4. Confirmar com a orientação as datas institucionais de submissão, entrega e apresentação.

A.6.2 P1 - Concluir a validação e coletar evidências

1. Reexecutar a suíte no ambiente fixado e guardar versão das dependências, hardware e resultado dos testes.
2. Fazer um piloto curto em CPU para validar o fluxo de ponta a ponta.
3. Executar a grade final no dispositivo-alvo, salvando CSV e metadados. Como resultados/ está ignorado pelo Git, definir antes quais artefatos finais serão versionados ou publicados em outro local reproduzível.
4. Distinguir claramente simulação numérica de ganho efetivo de hardware; só usar esse último rótulo quando armazenamento, tipo numérico e kernel executado sustentarem a afirmação.

A.6.3 P1 - Escrever o relatório junto com a análise

1. Gerar uma tabela de validação com entrada, saída esperada, saída local, saída da referência, erro máximo, tolerância e status.
2. Gerar tabelas e gráficos de latência, throughput, memória, erro e redução de operações para cada cenário contra o mesmo baseline.
3. Responder às perguntas de pesquisa com os resultados observados e registrar ameaças à validade, limitações e casos em que uma técnica não trouxe ganho.
4. Incorporar método, resultados e discussão ao texto acadêmico sem esperar o encerramento de toda a coleta.

A.6.4 P2 - Extensão opcional

Depois de concluir o núcleo, avaliar uma demonstração de inferência com um modelo pré-treinado do Hugging Face. A extensão não deve alterar o baseline nem atrasar as entregas principais.

A.7 Critério de conclusão desta etapa

A etapa pode ser considerada fechada quando houver:

- validação reproduzível da atenção contra as referências escolhidas;
- decisão documentada sobre o escopo e sobre TensorFlow/Keras;
- benchmarks comparáveis, com configuração e ambiente registrados;
- CSVs, tabelas e gráficos rastreáveis até a execução;
- conclusões compatíveis com o nível real de evidência, sem extrapolação para hardware ou Transformer completa;
- seção correspondente já integrada ao relatório do projeto.

A.8 Trechos da reunião que sustentam os encaminhamentos

- **00:00-04:12:** necessidade de validar a implementação contra bibliotecas consolidadas antes de avançar;
- **05:35-08:01:** sugestão de tabelas e gráficos comparando cálculo manual, PyTorch e TensorFlow, além de iniciar o relatório;
- **08:01-14:09:** sequência validação → otimização → comparação e delimitação do escopo na autoatenção;
- **15:21-16:15:** modelo pré-treinado como possibilidade condicionada ao tempo e incerteza sobre o calendário de novembro;
- **23:13 em diante:** recomendação de redigir em prosa desde já e fechar cada avaliação com resultados e tabelas;
- **29:30-30:07:** compromisso de avançar nos benchmarks após a reunião.

Guia de Estudo do Tema

B.1 1. O que é um Transformer

Transformer é uma arquitetura de rede neural introduzida por Vaswani et al. em 2017. Ela se tornou central em IA porque substituiu estruturas recorrentes por operações matriciais paralelizáveis. Isso permitiu treinar e executar modelos muito maiores com melhor uso de hardware moderno.

Em termos simples, o Transformer recebe uma sequência de vetores e aprende quais partes dessa sequência devem influenciar outras partes. Ele faz isso principalmente por meio do mecanismo de atenção.

B.2 2. Por que a atenção custa caro

Na autoatenção, cada token pode ser comparado com muitos outros tokens da mesma sequência. Isso significa que, quando a sequência cresce, o número de comparações cresce bastante. Além disso, antes da atenção, o modelo ainda faz projeções lineares densas para gerar Q, K e V.

Então, o custo da inferência não vem de um único ponto, mas da combinação de:

- multiplicações matriciais densas
- comparação entre tokens
- armazenamento e leitura de memória intermediária

B.3 3. Diferença entre treino e inferência

Treino é a fase em que o modelo aprende ajustando seus pesos. Inferência é a fase em que o modelo já treinado é usado para gerar respostas.

No seu trabalho, o foco está na inferência. Isso é importante porque muitas otimizações relevantes para uso prático buscam tornar a inferência mais barata, rápida e escalável.

B.4 4. O que é quantização

Quantização é a redução da precisão numérica usada para representar dados. Por exemplo, em vez de armazenar valores em formatos de maior precisão, é possível usar 8 bits ou menos.

Benefícios esperados:

- menos memória
- menos tráfego de dados
- potencial aumento de velocidade

Risco:

- introdução de erro numérico

Por isso, quantização sempre envolve um trade-off entre eficiência e fidelidade.

B.5 5. O que é pruning

Pruning é a remoção ou desativação de pesos considerados menos relevantes. Em tese, isso reduz o tamanho efetivo do modelo e o número de operações.

Mas existe uma diferença importante em relação à quantização:

- quantização muda a representação dos valores
- pruning muda a estrutura efetiva do cálculo

Na prática, pruning pode exigir mais cuidado para realmente virar ganho de execução, porque hardware e bibliotecas nem sempre aproveitam bem esparsidade.

B.6 6. O que é KV cache

KV cache significa key-value cache. Durante a geração de tokens em modelos autoregressivos, o modelo guarda chaves e valores já calculados para não recomputar tudo a cada passo.

Isso acelera a inferência, mas tem um custo:

- conforme o contexto cresce, o KV cache ocupa cada vez mais memória

Por isso, técnicas que comprimem KV cache são muito relevantes para inferência eficiente.

B.7 7. O que o TurboQuant propõe

TurboQuant é um método recente de quantização online divulgado pelo Google Research em 24 de março de 2026. O paper correspondente está no arXiv desde 28 de abril de 2025.

Em alto nível, ele busca quantizar vetores com baixa distorção, incluindo aplicações em KV cache e busca vetorial. Na narrativa da sua apresentação, o ponto principal não é reproduzir toda a matemática do método, mas usar o trabalho como evidência de que quantização está no centro das soluções mais fortes para eficiência em inferência.

B.8 8. Por que quantização parece mais promissora aqui

No contexto do seu trabalho, quantização parece uma primeira hipótese mais forte por três motivos:

1. Ataca diretamente memória e largura de banda.
2. Tem apoio forte em literatura recente.
3. Pode ser comparada de forma relativamente controlada com baseline e pruning.

Isso não significa que pruning é irrelevante. Significa apenas que, para um primeiro ciclo experimental, quantização parece oferecer melhor custo-benefício científico.

B.9 9. O que significam as métricas

B.9.1 Latência

Tempo gasto para executar uma operação ou cenário.

B.9.2 Memória

Quanto espaço é necessário durante a execução.

B.9.3 FLOPs

Estimativa do número de operações aritméticas de ponto flutuante.

B.9.4 MSE

Erro quadrático médio. Penaliza mais fortemente erros grandes.

B.9.5 MAE

Erro absoluto médio. Mede, em média, o quanto a saída se distancia da referência.

B.9.6 R^2

Coeficiente de determinação. Indica o quanto a saída candidata acompanha a referência.

B.9.7 Similaridade de cosseno

Mede o alinhamento entre dois vetores. É útil quando interessa preservar direção e estrutura relativa da saída.

B.10 10. Como explicar seu experimento de forma clara

Uma forma simples de explicar é:

‘Eu fixo os mesmos dados e os mesmos pesos-base, executo operações centrais do Transformer em três cenários diferentes e comparo custo computacional e qualidade de saída.’

Essa frase já resume:

- controle experimental
- comparação entre cenários
- foco em eficiência e fidelidade

B.11 11. Perguntas prováveis da banca

B.11.1 ‘Por que você escolheu Transformer?’

Porque é a arquitetura dominante em IA atual e concentra desafios reais de inferência em tempo e memória.

B.11.2 ‘Por que comparar quantização com pruning?’

Porque são duas estratégias clássicas e relevantes de otimização, mas atuam de formas diferentes: uma na representação numérica e outra na estrutura efetiva do modelo.

B.11.3 ‘Por que dizer que quantização é melhor se você ainda não rodou o experimento?’

Eu não afirmo como conclusão do meu experimento. Eu apresento como hipótese inicial motivada por literatura recente e por características do problema.

B.11.4 'Por que usar cenário controlado e modelo simplificado?'

Porque isso ajuda a isolar o efeito da técnica de otimização antes de ir para modelos completos mais complexos.

B.11.5 'O que você espera encontrar?'

Espero observar redução de custo computacional nos cenários otimizados e depois comparar o quanto isso preserva ou degrada a qualidade das saídas em relação ao baseline.

B.12 12. Frase de segurança para a apresentação

Se em algum momento você travar, pode usar esta frase:

'Meu trabalho ainda está na fase inicial de execução, então hoje eu apresento principalmente a relevância do problema, a hipótese técnica e o desenho experimental que vai permitir validar essa hipótese com dados próprios.'

Roteiro de 10 Minutos

Tempo total planejado: aproximadamente 9min40s.

C.1 Slide 1 - Capa (20s)

Objetivo: abrir a fala e posicionar o tema.

Fala: Este trabalho discute eficiência computacional em operações de Transformers. O foco está em quantização, pruning e em como comparar essas estratégias de forma experimental. Hoje eu apresento o problema, a hipótese e o desenho do estudo.

Transição: Primeiro, vale situar por que esse tema é relevante.

C.2 Slide 2 - Roteiro (20s)

Objetivo: dar previsibilidade para a banca.

Fala: Vou dividir a apresentação em cinco partes. Começo pelo contexto e pelo problema. Depois passo pelas operações centrais e pelas estratégias de otimização. Em seguida mostro a metodologia, o estado atual do projeto e as conclusões.

Transição: Começando pelo contexto geral.

C.3 Slide 3 - Transformers e a IA atual (35s)

Objetivo: justificar relevância.

Fala: Transformers se tornaram a arquitetura dominante em linguagem, visão e multimodalidade. Isso acontece porque eles combinam paralelização com boa modelagem de dependências de longo alcance. Com essa adoção ampla, eficiência deixou de ser detalhe técnico e passou a ser requisito prático.

Transição: Mas o problema aparece de forma mais clara quando olhamos a inferência.

C.4 Slide 4 - Custo da inferência (30s)

Objetivo: mostrar por que otimizar é necessário.

Fala: O custo da inferência cresce com o comprimento da sequência, com a dimensão interna do modelo e com o número de camadas. Em produção, isso aparece como latência, uso de memória e limite de throughput. Então, tornar a inferência mais eficiente é essencial para viabilizar esses modelos em hardware real.

Transição: Esse custo se concentra em alguns gargalos bem definidos.

C.5 Slide 5 - Gargalos principais (40s)

Objetivo: localizar os gargalos.

Fala: Os gargalos centrais são a autoatenção, as projeções densas e o KV cache. A atenção cresce com o contexto, as projeções densas repetem multiplicações matriciais e o KV cache pressiona a memória. Esses pontos são justamente os mais promissores para otimização.

Transição: Isso leva ao problema de pesquisa do trabalho.

C.6 Slide 6 - Problema e hipótese (30s)

Objetivo: explicitar o recorte científico.

Fala: O problema de pesquisa é reduzir custo computacional sem degradar excessivamente a saída. O recorte escolhido foi trabalhar com projeções densas e self-attention em cenários controlados de inferência. A hipótese inicial é que quantização tende a oferecer um trade-off melhor do que pruning.

Transição: Com esse recorte, os objetivos ficam mais claros.

C.7 Slide 7 - Objetivos (25s)

Objetivo: resumir o que será feito.

Fala: O trabalho implementa um protótipo simplificado das operações centrais do Transformer. Depois compara baseline, pruning por magnitude e quantização INT8 simulada. Por fim, mede custo computacional e qualidade de saída.

Transição: Agora eu mostro as operações estudadas.

C.8 Slide 8 - Projeções densas (30s)

Objetivo: explicar a primeira operação.

Fala: As projeções densas transformam vetores por multiplicações com matrizes de pesos. Elas aparecem na construção de Q, K e V e também em camadas lineares do bloco Transformer. Por isso, já concentram custo importante mesmo antes de analisar a atenção completa.

Transição: A segunda operação é a self-attention com KV cache.

C.9 Slide 9 - Self-attention e KV cache (40s)

Objetivo: explicar a segunda operação.

Fala: Na self-attention, o modelo gera Q, K e V, calcula escores, aplica softmax e monta a saída contextualizada. Para inferência autoregressiva, o KV cache evita recomputação, mas cresce com o tamanho do contexto. Então essa operação impacta tanto latência quanto memória.

Transição: Com essas operações definidas, entram as estratégias de otimização.

C.10 Slide 10 - Quantização (30s)

Objetivo: apresentar a primeira estratégia.

Fala: Quantização reduz a precisão numérica usada para representar pesos, ativações ou caches. Os ganhos esperados são menor uso de memória e maior throughput. O ponto de atenção é o erro numérico introduzido na saída.

Transição: A outra estratégia relevante é o pruning.

C.11 Slide 11 - Pruning (30s)

Objetivo: contrastar com quantização.

Fala: Pruning remove pesos considerados menos relevantes e introduz esparsidade. A economia teórica pode ser alta, mas o ganho real depende muito do suporte do hardware. Além disso, o efeito na saída tende a ser menos previsível.

Transição: A literatura recente reforça por que a hipótese inicial favorece quantização.

C.12 Slide 12 - TurboQuant (45s)

Objetivo: sustentar a hipótese com literatura.

Fala: Um trabalho importante aqui é o TurboQuant. Ele foi divulgado pelo Google Research em 24 de março de 2026, com paper disponível no arXiv desde 28 de abril de 2025. O foco está em compressão de vetores e do KV cache para inferência. Para o meu projeto, isso não é prova final, mas reforça a decisão de priorizar quantização no primeiro ciclo experimental.

Transição: Com essa base teórica, eu passo para a metodologia.

C.13 Slide 13 - Metodologia: desenho (35s)

Objetivo: mostrar o desenho do experimento.

Fala: O estudo compara três cenários: baseline, pruning em 50% e quantização INT8 simulada. As operações analisadas são projeção densa e self-attention de cabeça única. A grade principal varia comprimento de sequência e dimensão para observar o comportamento do custo.

Transição: Além do desenho, também defini métricas e controles.

C.14 Slide 14 - Metodologia: métricas (35s)

Objetivo: mostrar rigor experimental.

Fala: O ambiente previsto usa Python 3.11, bibliotecas numéricas e GPU RTX 4070 Ti Super. Cada cenário terá aquecimento, repetições e sincronização para medir latência. As métricas cobrem latência, memória, FLOPs e qualidade de saída com MSE, MAE, R^2 e coseno.

Transição: Com isso, eu mostro o que já está pronto no projeto.

C.15 Slide 15 - Resultados parciais (30s)

Objetivo: mostrar andamento real.

Fala: O projeto ainda não tem benchmarks completos executados. Mas o código-base já implementa operações, métricas e cenários. Ou seja, o trabalho já tem estrutura técnica pronta para iniciar a coleta comparativa.

Transição: Mesmo antes dos dados próprios, a literatura já aponta uma direção inicial.

C.16 Slide 16 - Evidências da literatura (40s)

Objetivo: usar o gráfico como motivação.

Fala: Esse gráfico resume evidências publicadas sobre TurboQuant. De um lado aparece a redução relativa de memória para KV cache; do outro, o ganho relativo de velocidade reportado. Esses números não são resultados do meu projeto, mas ajudam a justificar por que quantização é a hipótese inicial mais forte.

Transição: Com isso, eu posso fechar as conclusões.

C.17 Slide 17 - Conclusões (35s)

Objetivo: fechar com direção clara.

Fala: O trabalho ataca um problema real de inferência em Transformers. Hoje, a quantização aparece como a estratégia inicial mais promissora para esse recorte. O próximo passo é validar essa hipótese com benchmarks próprios e análise comparativa dos cenários.

Transição: Por fim, deixo as referências principais.

C.18 Slide 18 - Referências (20s)

Objetivo: mostrar base bibliográfica.

Fala: Aqui estão as referências principais que sustentam a apresentação. Elas incluem o trabalho original de Transformer, revisões recentes de compressão e as fontes usadas para o slide de TurboQuant.

Transição: Encerrando, fico à disposição para perguntas.

C.19 Slide 19 - Obrigado (10s)

Objetivo: fechamento.

Fala: Obrigado pela atenção. Fico aberto para perguntas e discussão.

C.20 Onde acelerar se faltar tempo

- Resumir o Slide 4 em uma frase.
- Juntar verbalmente os Slides 8 e 9.
- Resumir o Slide 18 em uma única frase.

Observações

jul. 31, 2026

D.1 Reunião em 31 de jul. de 2026 às 16:01 GMT-03:00

Registros da reunião [Transcrição](#) [Gravação](#)

D.1.1 Resumo

Discussão sobre validação do modelo Transformer, escopo de otimização no mecanismo de autoatenção e uso de modelos.

Validação do modelo Transformer A comparação com bibliotecas como o TensorFlow assegura a correção das implementações customizadas. Operações matemáticas fundamentais já validaram resultados específicos com sucesso.

Escopo da otimização técnica A otimização concentrada no mecanismo de autoatenção limita a complexidade do projeto ao custo computacional relevante. A arquitetura completa foi descartada por excesso de trabalho.

Estratégia de testes externos A utilização de modelos pré-treinados do Hugging Face para testes de inferência constitui uma abordagem válida. O cronograma de entrega permanece mantido para novembro.

D.1.2 Próximas etapas

- \[Lucas Galvão\] Comparar Implementação: Comparar a implementacao local com funcoes padrao do TensorFlow e PyTorch para validar a integridade das operacoes.
- \[Lucas Galvão\] Criar Relatório: Elaborar um relatorio contendo tabelas e graficos comparativos dos resultados obtidos. Registrar o passo a passo das atividades e as referencias teoricas consultadas.
- \[Lucas Galvão\] Realizar Testes: Conduzir testes de benchmark para avaliar o desempenho do modelo desenvolvido.

D.1.3 Detalhes

- **Validação do Modelo Transformer:** Lucas Galvão e Walter Silva Oliveira discutiram a validação de sua implementação do modelo Transformer. Walter Silva Oliveira sugeriu utilizar as camadas prontas do TensorFlow como base de comparação para validar o código desenvolvido. O objetivo é garantir que as implementações customizadas estejam alinhadas com as bibliotecas consolidadas antes de avançar para a aplicação de técnicas de otimização (00:00:00).
- **Estratégia de Validação e Comparação:** Walter Silva Oliveira enfatizou a necessidade de garantir a correção do código comparando os resultados com bibliotecas como o TensorFlow ou Keras, utilizando a mesma entrada e semente de ruído. Lucas Galvão observou que comparações determinísticas já foram realizadas para operações específicas, como o produto interno (dot product) e as operações de Query, Key e Value (QKV), que não exigem validações arquiteturais adicionais por serem definidas por equações matemáticas claras (00:01:28).
- **Documentação e Relatórios:** Walter Silva Oliveira recomendou que Lucas Galvão elabore gráficos e tabelas para documentar os resultados das comparações entre as implementações manuais e os resultados esperados. Esse relatório é considerado fundamental por Walter Silva Oliveira para consolidar o entendimento do trabalho e garantir que a base esteja sólida antes de aplicar técnicas avançadas como poda (pruning) ou quantização (00:05:35).
- **Escopo da Otimização:** Lucas Galvão questionou sobre a definição formal de um Transformer e os limites da otimização. Walter Silva Oliveira esclareceu que o foco do projeto deve ser otimizar o mecanismo de autoatenção (self-attention), que possui o maior custo computacional, em vez de tentar otimizar a arquitetura completa do Transformer, o que seria excessivamente complexo e trabalhoso para o cronograma disponível (00:09:18) (00:11:27).
- **Testes com Modelos de Terceiros:** Lucas Galvão perguntou sobre a possibilidade de utilizar modelos pré-treinados, como os disponíveis no Hugging Face, para testar a inferência. Walter Silva Oliveira confirmou que essa é uma abordagem válida, embora o sucesso da implementação dependa do tempo restante até o final do projeto (00:15:21).
- **Cronograma e Prazos:** Os participantes discutiram o cronograma, notando que o prazo para a entrega/apresentação é por volta de novembro. Walter Silva Oliveira mencionou que as reuniões institucionais e o retorno formal dos professores acontecerão na semana seguinte, o que permitirá definir melhor os próximos passos administrativos (00:15:21).
- **Atividades Acadêmicas e Certificados:** Lucas Galvão expressou dificuldades em obter as 20 horas complementares exigidas pela faculdade, mencionando problemas com a aceitação de certificados de maratonas de programação. Walter Silva Oliveira orientou

sobre a verificação dos limites de horas permitidos por categoria no sistema da instituição (00:17:28).

- **Discussão Pessoal e Equilíbrio:** Lucas Galvão e Walter Silva Oliveira conversaram sobre interesses pessoais, incluindo jogos como League of Legends e Civilization, e sobre suas rotinas de trabalho e estudo. Lucas Galvão detalhou sua configuração de trabalho com desktop remoto, enquanto ambos discutiram o equilíbrio entre vida pessoal e obrigações acadêmicas (00:18:20).
- **Encaminhamentos Finais:** Walter Silva Oliveira reforçou que Lucas Galvão deve começar a escrever o relatório e documentar os processos imediatamente para evitar sobrecarga futura. Lucas Galvão confirmou que iniciará os testes de benchmark e a estruturação da documentação conforme orientado (00:23:13).
- **Reflexões sobre a Vida Universitária e Deslocamento:** A reunião terminou com uma conversa sobre as dificuldades de deslocamento entre Santos e São Paulo, as dinâmicas de amizade durante e após o período acadêmico e as expectativas para o final do semestre (00:25:52).

Revise as anotações do Gemini para checar se estão corretas. Confira dicas e saiba como o Gemini faz anotações

*Como está a qualidade de **destas observações**? Responda a uma breve pesquisa para nos dar seu feedback, incluindo o quanto as observações foram úteis para o que você precisa.*

Transcrição

jul. 31, 2026

E.1 Reunião em 31 de jul. de 2026 às 16:01 GMT-03:00 \\- Transcrição

E.1.1 00:00:00

Lucas Galvão: Pronto. Daí depois eu posso usar o que você comentou. Hum.

Walter Silva Oliveira: Porque tá que você tava usando v que você usa as coisas do pai torche, mas você não lembra seu próprio p seu kerfow que já tem

Lucas Galvão: Uhum.

Walter Silva Oliveira: o sistema de transformer pronto, não tem? Não sei se chegou a ver

Lucas Galvão: Eh,

Walter Silva Oliveira: isso.

Lucas Galvão: não, o transformer completo. Pronto.

Walter Silva Oliveira: Isso é porque você consegue fazer uma layer de transformer, pelo menos no temor flow, ela tem.

Lucas Galvão: Uhum.

Walter Silva Oliveira: Tu acha que dá seria legal para validar também? Você tá usando o p o pche basicamente para criar os objetos.

Lucas Galvão: Yry.

Walter Silva Oliveira: mais criar as layers, talvez pensar como que a gente consegue compare esse cara com o que já é feito direto no nessas bibliotecas que se coloca quero uma lei de transformer e aí ele utiliza porque aí a gente garante, beleza? Então essas implementações que tem aqui estão batendo com a funçãozinha lá do Tens flow. Então a gente conseguindo validar esse ponto, o que que a gente pode fazer? a gente pode começar a brincar um pouco mais de como essas operações são feitas, antes elas com um pouco mais de detalhe e ver o impacto disso, entende?

E.1.2 00:01:28

Walter Silva Oliveira: Só pra gente tem tem essas funções prontas, aparentemente elas estão OK. dei uma olhada por cima, mas só pra gente ter um caso base comparando com a biblioteca pura do densor flow,

Lucas Galvão: Uhum.

Walter Silva Oliveira: por

Lucas Galvão: OK. É o interessante, dá para usar assim o Tflow,

Walter Silva Oliveira: exemplo.

Lucas Galvão: o que o Pyth aqui, eu acho que ele não tem o o não tem eh não tem o transformer completo, mas ele tem aquelas partes de dos tensores, né? Eu acho que tem até aqui.

Walter Silva Oliveira: Sim,

Lucas Galvão: Eh,

Walter Silva Oliveira: sim.

Lucas Galvão: ah, não lembro onde estava. É aqui, exatamente parte dos tensores que eh daí você acha que é interessante pegar tipo isso e comparar também com o desempenho do da própria do tensor flow para ter mais uma

Walter Silva Oliveira: É porque isso que beleza.

Lucas Galvão: validação?

Walter Silva Oliveira: Aparentemente tá funcionando, mas é aquela questão que a gente conversou lá no início,

Lucas Galvão: Uhum.

Walter Silva Oliveira: como que você garante que o que você tem feito? Então beleza,

Lucas Galvão: Sim.

Walter Silva Oliveira: a ideia inicial era vamos entender um pouquinho mais como o transformer funciona,

Lucas Galvão: Yry.

Walter Silva Oliveira: como que a gente consegue otimizar esse transformer considerando que ele é um bloco que vai ser rodado recorrentemente, a gente quer de alguma maneira diminuir o tamanho desse cara.

E.1.3 00:02:53

Walter Silva Oliveira: Aí, beleza,

Lucas Galvão: Yry.

Walter Silva Oliveira: a gente tem um monte de técnica que a gente pode utilizar para

diminuir tamanho, fazer quantização, etc. Show. Questão é como esse, então a hipótese nossa é como que a gente vai avaliar impacto dessas técnicas no Transformer. Aí a ideia então foi, beleza, vamos estudar primeiro como ele funciona. A partir do momento que ele funcionou, a gente começa a aplicar essas técnicas. Aí agora o ponto é como que eu sei que esses códigos que eu desenvolvi eles

Lucas Galvão: Yry.

Walter Silva Oliveira: estão corretos? Então eu preciso de alguma maneira comparar o resultado desse cara com alguma questão da literatura. Então a minha hipóte, eu acho que o que faz sentido é pensar em beleza,

Lucas Galvão: Yry.

Walter Silva Oliveira: eu sei que tensor flow ou que eras são bibliotex consolidad essa área de mach learning, então eu posso d mesma entrada, uma mesma semente de ruído eu tenho que ter a mesma saída nos dois.

Lucas Galvão: Yry.

Walter Silva Oliveira: Feito isso, beleza. Se então que todas essas questões ali de de metas,

Lucas Galvão: Bot.

Walter Silva Oliveira: na minha cash, tá tudo funcionando. Minhas operações são feitas corretamente. Aí feito isso, aí é a parte de brincar.

E.1.4 00:04:12

Walter Silva Oliveira: Então, como que eu posso criar cenários? Aí a gente continua nessa parte de que pensar em cenários que a gente vai testar esse nosso Transformer.

Lucas Galvão: Sim.

Walter Silva Oliveira: Aí só pra gente garantir o que que quando a gente tem esse salto a gente saiba que o problema não tá na nossa implementação

Lucas Galvão: Uhum.

Walter Silva Oliveira: inicial.

Lucas Galvão: Outra coisa que eh foi feita foi fazer a comparação de alguns processos com equações prontas, né? Que daí seria tipo aquelas equações que são tipo eh,

Walter Silva Oliveira: Yrym.

Lucas Galvão: que é tem aqueles QKV, né? aqui KT que daí, tipo, eles são operações prontas, daí eu fiz a comparação delas com as com as implementações que a gente fez aqui. Então, por exemplo, para esses para esse caso, não é necessário fazer uma mais comparações, né? Porque a gente já fez tipo uma comparação determinística com uma equação pronta, que seria a equação da definição lá, por exemplo, do que que é uma que que é um tensoro, o que que é um, qual o nome? Aquele produto pontual lá, dot product. Aí tem essas,

Walter Silva Oliveira: assim.

Lucas Galvão: é, sim, tem essas partes que elas são determinísticas, que elas são definidas por equações, não são por, por exemplo, não é arquitetura.

E.1.5 00:05:35

Lucas Galvão: A parte da arquitetura é mais difícil de validar do que essa parte de multiplicação matricial e tais.

Walter Silva Oliveira: Sim, sim, com certeza. Mas e exatamente,

Lucas Galvão: Uhum.

Walter Silva Oliveira: por isso é mais complicado, porque que é importante a gente vai lhe dando passo a passo e talvez fazer isso,

Lucas Galvão: Uhum. Uhum.

Walter Silva Oliveira: talvez pegar, fazer uns gráficos, nem que seja uma tabelinha de resultados, só comparando. Beleza? Aí aqui está o que eu estou fazendo, digamos,

Lucas Galvão: Tá.

Walter Silva Oliveira: pegando a equação lá, fazendo multiplicação matricial, dado é a mesma entrada, tenho esse resultado. Qual eu falei na mão utilizando p torche, utilizando o tensor flow e aí a gente consegue

Lucas Galvão: Uhum.

Walter Silva Oliveira: validar isso. Aí a gente começa a pensar nessa questão de como que a gente melhora o prone, como a gente melhora a aonar a quantização desses caras e ver como isso impacta.

Lucas Galvão: Uhum. Beleza. Legal. Eh.

Walter Silva Oliveira: Acho você já fez bastante coisa, então eu acho que é importante também começar a Claro está ainda numa etapa inicial,

Lucas Galvão: Uhum.

Walter Silva Oliveira: mas vai pensar o que que é resultado, que que eu fiz, que que posso comparar, fazer uns gráficos, que eu até comentei lá, pegar esses, fazer meio que o relatóriozinho, que é ficar até mais fácil para depois você entender,

E.1.6 00:06:51

Lucas Galvão: Uhum.

Walter Silva Oliveira: tá? Que que eu fiz? Da onde que eu peguei as coisas para fazer um relatório para

Lucas Galvão: Uhum. Sim.

Walter Silva Oliveira: depois?

Lucas Galvão: Que gráficos que você acha que são interessantes? Por exemplo, eh, por exemplo, esse de, por exemplo, como se fosse um eixo é as variáveis e um eixo é o resultado das equações relacionadas às variáveis.

Walter Silva Oliveira: Isso passei o gráfico uma tabela, por exemplo. Digamos que,

Lucas Galvão: Pode

Walter Silva Oliveira: ó, se eu pegar fazer a conta ali novo no Python,

Lucas Galvão: ser?

Walter Silva Oliveira: vai dar X. Izando o método tal,

Lucas Galvão: Uhum.

Walter Silva Oliveira: vai dar uma variância de vai dar o tal valor.

Lucas Galvão: Uhum.

Walter Silva Oliveira: Aí, claro, depende muito do da maneira que você tá testando e do que você tá testando, né? Mas só pra gente ter esses esses estados palpáveis que fica até mais legal pra gente falar falar. Beleza? Ó, então ó,

Lucas Galvão: M.

Walter Silva Oliveira: a gente estudou essa parte teórica, entendemos como funciona, quais são as operações que estão sendo feitas e aí beleza? Então vamos implementar o nosso. Implementamos o nosso, ó, tô vendo aqui, ó.

E.1.7 00:08:01

Walter Silva Oliveira: Se eu pegar a minha conta aqui do produto interno e eu pegar a funçãozinha lá do pai que faz o

Lucas Galvão: Yry.

Walter Silva Oliveira: dot product, deu a mesma coisa. E aí a gente vai escalando isso até, ó, se eu pegar essa mesma entrada, passar com o mesmo transforme, com o mesmo com minhas mesmas chaves, eu vou ter o mesmo resultado na saída. Aí, beleza? Então, eu sei que todo o nosso fluxo tá funcionando. Feito isso, aí a gente pode pensar, beleza, quais são as técnicas melhores? Aí a gente volta na segunda etapa lá da metodologia e pensar em como que a gente cria

Lucas Galvão: Yry.

Walter Silva Oliveira: para testar e otimizar o Transformer em si. Eu cheguei a ver que tinha até uns modelinhos de você rodar meio com uma lá no próprio Transformer para você saber quais são os pesos que você tem da Prun. Tem tem um aí tem uma vida para pesquisar.

Lucas Galvão: Yry.

Walter Silva Oliveira: A gente pode pegar uma ou duas técnicas dessas da metodolo-

gia, aplicar testar o resultado.

Lucas Galvão: É, eh, sim, entendi. Uma coisa que ficou bastante na minha cabeça era, por exemplo, o que que é um, assim, a definição formal de um transformer. Tipo assim, o que a gente tá pegando aqui?

E.1.8 00:09:18

Lucas Galvão: Eh, eu acho que a gente não tá tipo replicando exatamente o que que o Attention isol e o Need fala que que é um transformer, que ele é muito, ele tem muitas etapas lá, mas a gente tá pegando operações que ocorrem dentro do Transformer, né?

Walter Silva Oliveira: É porque basicamente você tá fazendo a parte do selfattention, né?

Lucas Galvão: Uhum.

Walter Silva Oliveira: Então beleza. Então por enquanto a gente tá precisando só trazer a ideia de transform inteiro é beleza. Vai pegar o self, passar um LP, fazer o encoding e tudo mais.

Lucas Galvão: Sim, sim.

Walter Silva Oliveira: Ei,

Lucas Galvão: Daí vai fazer o profet

Walter Silva Oliveira: só que essa própria questão do Attention,

Lucas Galvão: lá.

Walter Silva Oliveira: ele acaba sendo uma matriz grande, né? Então, beleza, que a gente não chega no transform,

Lucas Galvão: Sim.

Walter Silva Oliveira: vamos tentar só como que a gente otimiza essa matriz de autens.

Lucas Galvão: Uhum.

Walter Silva Oliveira: Acho que é isso. A gente tem um mundo para explorar. Só é importante sempre que a gente dá um passinho sabendo que a gente tem que ter firmeza, porque senão se a gente avançar muito rápido, a gente vai chegar lá na frente, putz, tá dando errado e aí onde é que tá errado?

E.1.9 00:10:29

Lucas Galvão: Uhum.

Walter Silva Oliveira: Só essa é a única preocupação. Mas assim, mas você tá já fazendo mais do que eu imaginava.

Lucas Galvão: Entendi.

Walter Silva Oliveira: Você tá indo no

Lucas Galvão: É, então a minha ideia era, na verdade era tipo fazer 90% da aí nesse mês, né? Mas acho que eu fiquei muito muito preguiçoso aí período de

Walter Silva Oliveira: não, mas não tem. É assim,

Lucas Galvão: férias.

Walter Silva Oliveira: claro, nas férias é o momento que você tem mais tempo para fazer as coisas, mas é isso, não tem por ter pressa. O que importa é validando as etapas, o início é a parte mais complicada,

Lucas Galvão: Uhum.

Walter Silva Oliveira: porque você ainda não, pô, o que que é o K kv ali? Como é que funciona, como que eu faço com que tá,

Lucas Galvão: Sim.

Walter Silva Oliveira: é uma matriz, que operação que é essa? Depois que você internaliza isso,

Lucas Galvão: Uhum.

Walter Silva Oliveira: as próximas etapas ficam muito mais fáceis. Então, início é a parte mais

Lucas Galvão: É sim. É sim. E assim,

Walter Silva Oliveira: complicada.

Lucas Galvão: o que eu achei especialmente complicado é essa parte de definição.

E.1.10 00:11:27

Lucas Galvão: Tipo, por exemplo, o que que é transformer? Você vai pegando um artigo lá e ele não fala tipo explicitamente, ah, definição de transformer é tal, não. Ele só, tipo, fala assim, ah, estou implementando isso aqui, isso aqui é transformer, é tipo um monte de attentions lá. Daí, por exemplo, tanto que é o tanto que, tipo, então, por exemplo, como é que eu faço comparações para saber se as minhas operações elas estão corretas diante de dos artigos do que ele definá? Yeah. É fácil fazer paraas operações matemáticas, né, que é tipo fazer a atation lá, que é o QKV, eh, que é mais tranquilo a parte de projeção de produto lá, o dot pro, mas a parte de eh como esses itens se relacionam é muito, acho que é a parte mais complicada.

Walter Silva Oliveira: É bem

Lucas Galvão: E e eu não sei se, por exemplo, se dentro do escopo do nosso título lá, do nosso trabalho inicial, eu até revisei isso, talvez não seja necessário fazer esse tipo de revisão, tipo assim, de como os itens se relacionam, porque o nossa proposta é analisar eh estratégias de otimização dentro de operações específicas que estão dentro da Transformer, não otimização da Transformer em si, mas talvez fique meio É irrelevante fazer também a

otimização

E.1.11 00:12:54

Walter Silva Oliveira: Não,

Lucas Galvão: ou

Walter Silva Oliveira: o que você pode, você pode pensar é o seguinte,

Lucas Galvão: não?

Walter Silva Oliveira: beleza? Se a gente pensar como funciona, né, esse bloco, essa camada inteira, a gente vai ter a parte que mais tem custo computacional é a parte tal.

Lucas Galvão: Uhum.

Walter Silva Oliveira: Então vamos focar só nessa parte que a priori lá semestre passado,

Lucas Galvão: Uhum.

Walter Silva Oliveira: se a gente otimizar essa parte que é a mais relevante,

Lucas Galvão: Mum.

Walter Silva Oliveira: a gente consegue melhorar a performance. Então beleza, a gente pode até colocar isso como justificativa no faz a gente otimizar, otimizar o transforma inteiro, talvez seja algo que dê muito trabalho pro nosso tempo. Então a gente vai tem que focar nosso esforço em entender o mecanismo principal, que é o mecanismo de alta tensão.

Lucas Galvão: Sim,

Walter Silva Oliveira: E aí a partir desse pressuposto,

Lucas Galvão: sim.

Walter Silva Oliveira: tá? Como ele funciona com essas operações que ele faz e como o qual é o espaço que eu tenho para para otimizar. E desde as estratégias que tem na literatura, a gente não precisa inventar nada como elas funcionam e como elas se comparam.

Lucas Galvão: Mhm.

Walter Silva Oliveira: Então acho que é isso. Pensar, beleza, vamos focar só na parte de alta tensão que tem lá multiplicação, é só max para calcular a tensão.

E.1.12 00:14:09

Walter Silva Oliveira: Beleza? Como que isso acontece?

Lucas Galvão: Uhum.

Walter Silva Oliveira: Qual quais as técnicas que eu posso utilizar para melhorar isso? Aí a gente aplica essas técnicas e mexe formas.

Lucas Galvão: Sim. Perfeito, perfeito. É, mas você acha que antes de fazer isso tem

uma etapa anterior, né, que era fazer a comparação lá com a outra biblioteca,

Walter Silva Oliveira: É,

Lucas Galvão: né,

Walter Silva Oliveira: e antes isso é talvez é que eu é agora

Lucas Galvão: outra que

Walter Silva Oliveira: parando para pensar o do tensor flowing pegar faz tanto tempo que ouvi

Lucas Galvão: Ehm

Walter Silva Oliveira: isso eh porque eu acho que não sensor flow. Eu só coloco o layer transforma ele inteiro. Não sei se a gente vai conseguir ver a parte de alta tensão interna ali da operação sendo

Lucas Galvão: M.

Walter Silva Oliveira: feita.

Lucas Galvão: Não deve ter sim.

Walter Silva Oliveira: É, mas é isso. É tipo pegar pega só vai uma um cálculo desses,

Lucas Galvão: Eh,

Walter Silva Oliveira: né? Uma interação e aí a gente compara, vê se o resultado que dá na saída dele é o resultado que a gente tem. É só isso que precisa fazer. para falar, beleza, a minha implementação ela

Lucas Galvão: certo.

E.1.13 00:15:21

Lucas Galvão: Legal. Acho que dá fazer.

Walter Silva Oliveira: funciona.

Lucas Galvão: Você acha que vale a pena pegar eh modelos prontos? Tipo, quando a gente for testar, eh, vale a pena pegar, por exemplo, alguma coisa, um modelo já treinado para gente testar inferência em cima dele, tipo,

Walter Silva Oliveira: Pode pegar no Face.

Lucas Galvão: pegar um alguma coisa lá no hugging face e isso.

Walter Silva Oliveira: Pode sim, sim, com certeza.

Lucas Galvão: Uhum.

Walter Silva Oliveira: Se sobrar tempo, né? Mas acho que não é

Lucas Galvão: É, eu tô É,

Walter Silva Oliveira: assim.

Lucas Galvão: já tá meio apertado, né? É para setembro.

Walter Silva Oliveira: Eu tenho que ver a data exata, mas deve ser por aí,

Lucas Galvão: É.

Walter Silva Oliveira: porque aí vai a apresentação deve ser ou nacional do ano ou em dezembro.

Lucas Galvão: É,

Walter Silva Oliveira: Aí durante uns um, dois meses antes. Essa para aí setembro,

Lucas Galvão: é iss até que é quando?

Walter Silva Oliveira: outubro. aca em novembro,

Lucas Galvão: Em novembro. Ah, tá. Então tá tranquilo.

Walter Silva Oliveira: mas aí tem que mandar antes para ser aprovado. Não sei. Eh,

E.1.14 00:16:15

Lucas Galvão: Não,

Walter Silva Oliveira: provavelmente isso vai ser discutido na semana que vem, né, que os professores a priori voltaram hoje. Hoje é o primeiro dia que de férias,

Lucas Galvão: não.

Walter Silva Oliveira: mas todas as reuniões com a reitoria e com o colegiado s semana que vem.

Lucas Galvão: Uhum.

Walter Silva Oliveira: A gente deve discutir isso no próprio colegiado, como é que vai ficar essas atas?

Lucas Galvão: Uhum. Vocês chegam muitos alunos. nessas reuniões.

Walter Silva Oliveira: Só um só que merece.

Lucas Galvão: Beleza. Chegar lá o Felipe.

Walter Silva Oliveira: Não, nem fala,

Lucas Galvão: Lembra do Felipe?

Walter Silva Oliveira: nem fala esse nome. Pelo amor de Deus. Graças. Eu tô feliz quando eu vou dar lá para vocês só

Lucas Galvão: Que isso para você vai dar?

Walter Silva Oliveira: alegria.

Lucas Galvão: Eh, para aquele pessoal lá do primeiro ano de

Walter Silva Oliveira: Eu sou Não, não, só tô com quarto e sexto.

Lucas Galvão: novo.

Walter Silva Oliveira: Eu vou eu vou est aqui à noite, quer dizer, eu vou estar aqui terça, quarta, quer dizer, segunda, terça e quarta, mas segunda e quarta eu só dou aula de manhã. Aí vai ficar amanhã à tarde e terça ou à noite.

E.1.15 00:17:28

Lucas Galvão: Uhum. Cara, não aguento mais a faculdade. Tem que acabar logo.

Walter Silva Oliveira: Tá acabando,

Lucas Galvão: Acabando.

Walter Silva Oliveira: tá acabando.

Lucas Galvão: Tá acabando, realmente.

Walter Silva Oliveira: Eu acho que o pior já passou. Só fechar essas horinhas aí. Você vai cheirar de letra e a

Lucas Galvão: É,

Walter Silva Oliveira: gente aí,

Lucas Galvão: eu tenho ainda as 20 horas complementares aí. Acho que eu vou doar sangue conta.

Walter Silva Oliveira: sei lá, vai doar sangue. Ai, sei lá,

Lucas Galvão: Acho que não.

Walter Silva Oliveira: pô. Na minha época, contava

Lucas Galvão: Acho que não. Tem que vou ter que pagar a lura lá e emitir um monte de certificado

Walter Silva Oliveira: Isso para loura tem um tem uns limites de horas tem as categorias lá e você

Lucas Galvão: lá.

Walter Silva Oliveira: tem é limitado a quantidade de horas por categoria. Então por mais que você faça isso de 200 horas você só consegue emitir tipo 15, tipo 10\.

Lucas Galvão: Sim, mas eu preciso só de 20\.

Walter Silva Oliveira: Tem que ver.

Lucas Galvão: Cara, você acredita que eu fiz eu fiz a maratona de programação todos os semestres

E.1.16 00:18:20

Walter Silva Oliveira: Mas vai que eu acredito que você

Lucas Galvão: e eu não subi e eu não subi nenhuma

Walter Silva Oliveira: não mandou nenhuma. É, eu acredito.

Lucas Galvão: delas,

Walter Silva Oliveira: Você é mais comum do que deveria, Lucas.

Lucas Galvão: mano. É assim,

Walter Silva Oliveira: É mais comum do que

Lucas Galvão: e não,

Walter Silva Oliveira: deveria.

Lucas Galvão: o que é chato é eles não permitirem eu mandar o certificado

Walter Silva Oliveira: Você tem que mandar 15 dias depois.

Lucas Galvão: com absurdo. Deixa eu ver quantas horas eu preciso, mano. Você jogar LOL, professor. Sua

Walter Silva Oliveira: jogo, né? Jogo assim,

Lucas Galvão: cara,

Walter Silva Oliveira: fazia muito tempo quando eu jogava, eu vi, aí eu saí o LOL Classic lá, eu joguei umas três partidinhas com os meus amigos para relembrar os velhos tempos.

Lucas Galvão: não vou te adicionar pra gente jogar lá depois, pô.

Walter Silva Oliveira: Eu só jogar.

Lucas Galvão: Não, bora. Sou erra.

Walter Silva Oliveira: Perfeito.

Lucas Galvão: Qual o seu Qual seu nickname?

Walter Silva Oliveira: Eu vou, cadê? Vou mandar minha tag. Voltinho. V hasinho.

E.1.17 00:19:20

Walter Silva Oliveira: Não, faltou um H. Pior que seas férias.

Lucas Galvão: Bom nick. Bom nick.

Walter Silva Oliveira: Eh, eu joguei Civilization com o Raul e com o Henrique. Tur de reserva.

Lucas Galvão: Quem é R Henrique?

Walter Silva Oliveira: é da outra turma

Lucas Galvão: Ah, tá, entendi. Pô, mas o o CV online é

Walter Silva Oliveira: aí. Ah,

Lucas Galvão: complicado.

Walter Silva Oliveira: é assim, mas é bom, é legal, é legal. 100% de

Lucas Galvão: Nossa,

Walter Silva Oliveira: rate.

Lucas Galvão: pô. Mas você assim, acho que eu nunca joguei uma partida do Civilization até o final. Meu limite,

Walter Silva Oliveira: É assim.

Lucas Galvão: quando eu jogava, meu limite era sempre quando eu descobria o satélite, que daí desbloqueava o mapa todo e meu PC não rodava mais ali, ó.

Walter Silva Oliveira: Uhum.

Lucas Galvão: Era o limite, dera o fim.

Walter Silva Oliveira: Ah, mas agora você tem uma Não, você tem uma placa de boa, pô.

Lucas Galvão: É, mas agora,

Walter Silva Oliveira: Agora vai lá agora.

Lucas Galvão: mas agora todos os meus amigos trabalham, fazem mestrado, namoram e ninguém quer ficar 36 horas numa parceira de

E.1.18 00:20:29

Walter Silva Oliveira: É, não,

Lucas Galvão: C.

Walter Silva Oliveira: 36 não, mas a primeira partida foi foram 8 horas.

Lucas Galvão: É,

Walter Silva Oliveira: Aí a segunda foi mais rapidinho,

Lucas Galvão: então

Walter Silva Oliveira: acho que foi três. Não é tão ruim.

Lucas Galvão: não, não, nunca falei que é ruim, falei que é longo.

Walter Silva Oliveira: Pronto, na durante as férias dá para jogar,

Lucas Galvão: Te adicionei aqui, ó.

Walter Silva Oliveira: tá? Não,

Lucas Galvão: Deixa eu ver seu perfil.

Walter Silva Oliveira: não consigo abrir aqui porque, né, a não deixa,

Lucas Galvão: Ah. Não acredito.

Walter Silva Oliveira: Mas agora chegar em casa.

Lucas Galvão: Não

Walter Silva Oliveira: É sério?

Lucas Galvão: acredito.

Walter Silva Oliveira: Você nunca tentou abrir? Se eu tent, ó, se eu tentar abrir aqui. Acho que não tem nesse celular não. Se eu tentar abrir aqui. Tem logo aqui? Não, não tem nesse computador. Graças a Deus.

Lucas Galvão: Nossa, você é muito mentiroso, professor. Olha isso. Você jogou três, seis.

Walter Silva Oliveira: Eu joguei ontem é umas três horinhas.

Lucas Galvão: Jogou, mas jogou 10 partidas.

E.1.19 00:21:24

Walter Silva Oliveira: Jogando aranha. É rapidinho. Vê depois disso. A última vez que eu joguei, tinha jogado há uns três meses

Lucas Galvão: Não, daí Ah, não, o mês passado, pô.

Walter Silva Oliveira: atrás.

Lucas Galvão: É, realmente, teve um, é que foi dia 29, daí em cima tava dia 30, só que um é 29/06, outro é 30/07. Olha que coincidência,

Walter Silva Oliveira: Não, não é coincidência.

Lucas Galvão: não é coincidência.

Walter Silva Oliveira: Não é coincidência. Final do mês sempre mais tranquilo pro pessoal. Mas assim, eu jogo bem casualmente, não jogo

Lucas Galvão: que eu não jogava ranked desde 2022,

Walter Silva Oliveira: ranked.

Lucas Galvão: que foi o ano que eu parei de eh que eu comecei a faculdade, né? 2023 comecei a faculdade. Daí eu joguei bastante 2022\ Da. Tô jogando um pouquinho. Divertido. É divertido até não ficar mais assim, até você ficar no elo que você pertence e daí o jogo fica ruim de

Walter Silva Oliveira: Não é assim, eu eu tô com e você fica muito tempo para você jogar,

E.1.20 00:22:20

Lucas Galvão: novo.

Walter Silva Oliveira: você joga, fala: 'Pô, muito legal jogar lô'.

Lucas Galvão: Até

Walter Silva Oliveira: Mas aí depois você joga dois dias,

Lucas Galvão: mesmo.

Walter Silva Oliveira: acabou, acabou o amor, esquece. Tem que ser assim.

Lucas Galvão: É, então é que tipo assim, quando você tá muito tempo sem jogar, toda partida que você joga você ganha mais de 10 kills,

Walter Silva Oliveira: É legal.

Lucas Galvão: né? Você pega tipo 15

Walter Silva Oliveira: É.

Lucas Galvão: kills.

Walter Silva Oliveira: E isso igual no Fortnite você tem que jogar só, tem que ganhar das criancinha. Aí quando vem o pessoal bom,

Lucas Galvão: Aham.

Walter Silva Oliveira: aí você vai

Lucas Galvão: Não, por isso que é bom jogar nas férias,

Walter Silva Oliveira: jogar.

Lucas Galvão: que daí tem as crianças às 10 horas da manhã. Ó,

Walter Silva Oliveira: É

Lucas Galvão: mostrar a tática que eu uso para trabalhar.

Walter Silva Oliveira: verdade.

Lucas Galvão: Tá vendo aqui, ó, que tá vendo esse aplicativo aqui?

Walter Silva Oliveira: Uhum.

Lucas Galvão: É um desk top remoto que eu uso pro meu notebook.

Walter Silva Oliveira: Sim.

Lucas Galvão: O notebook fica tipo atrás de mim aqui, daí eu consigo usar minha minha tela.

E.1.21 00:23:13

Lucas Galvão: Daí fica tipo assim,

Walter Silva Oliveira: E aí? Aí fica só a área do trabalho.

Lucas Galvão: não fica é fica tudo do relacionado ao meu trabalho da

Walter Silva Oliveira: É, eu tenho dois computadores.

Lucas Galvão: empresa.

Walter Silva Oliveira: notebook uso para fazer coisa doutorado da Santos e me PC uso para parte

Lucas Galvão: Uhum.

Walter Silva Oliveira: boa.

Lucas Galvão: Então daí é isso. Deixa eu ver algo mais interessante para ser falado

aí.

Walter Silva Oliveira: Acho que é isso. Acho que isso aí é um caminho bom. Só fazer essa questão de já começar a escrever porque senão você vai esquecer. escrever texto mesmo, não só colocar os tópicos,

Lucas Galvão: Yry.

Walter Silva Oliveira: que é importante até para você começar a a lincar suas ideias, que são muitos conceitos. E aí como que uma coisa se encadeia na outra? Você vai acabar só entendendo isso melhor quando você começar a escrever. É uma parte importante até para não ficar quando chegar lá em setembro não ficar muito apertado nisso. E é isso. Fechar essas avaliações com resultados, fazer umas tabelinhas de resultados. os estados parciais em si. E aí a gente parte para continuar as

Lucas Galvão: Beleza.

E.1.22 00:24:34

Walter Silva Oliveira: coisas. Deixa eu ver se tem mais alguma dizer. Eu acho que é isso, Lucas. Você tem mais alguma pergunta, alguma coisa que você quer conversar?

Lucas Galvão: Deixa eu ver. Não, conversar sempre bom, né? Deixa eu pensar alguma

Walter Silva Oliveira: Bom,

Lucas Galvão: coisa.

Walter Silva Oliveira: agora voltamos vem semana que vem eu tô eu tô aqui quase todo

Lucas Galvão: É. Ah, tem que aparecer lá para ver a

Walter Silva Oliveira: dia.

Lucas Galvão: calorada. É, mas agora você não tá dando mais a para calor, né? Só para quatro

Walter Silva Oliveira: Não, não, só quarto e sexto.

Lucas Galvão: semestre.

Walter Silva Oliveira: Problema é que a aula da noite é ruim agora porque é uma aula sua, é uma aula só. Então é tipo, papum, rapidão. Então nem dá. Quando são quatro aulas seguidas, eu dá para deixar um pouquinho mais livre, mas quando é duas é

Lucas Galvão: Nossa, então eu gosto bem mais quando é aula dupla, porque tipo assim,

Walter Silva Oliveira: corrido.

Lucas Galvão: porque dá umas 9 da noite, o professor fala assim: 'Ah, cansei, já podem ir embora'. Agora, quando é tipo duas aulas seguidas de dois professores diferentes, o

professor sempre dura até às 10\.

E.1.23 00:25:52

Walter Silva Oliveira: né? Que eu tenho ficar mais curto que até vocês chegarem já é 9 horas e tem esse

Lucas Galvão: Ah,

Walter Silva Oliveira: ponto.

Lucas Galvão: nada a ver.

Walter Silva Oliveira: Aí você falou

Lucas Galvão: Eu não chego tarde. Eu não chego. Ah, sim. Eu só chego tarde quando o quando tá chovendo,

Walter Silva Oliveira: não.

Lucas Galvão: né? Daí não tem como. Aí é culpa culpa do trânsito.

Walter Silva Oliveira: Mas você mora onde?

Lucas Galvão: Hã,

Walter Silva Oliveira: Você mora onde?

Lucas Galvão: eu moro em Santos, só que às vezes eu trabalho em São Paulo e quando eu trabalho em São Paulo eu desço de

Walter Silva Oliveira: Ah,

Lucas Galvão: fretado,

Walter Silva Oliveira: tá.

Lucas Galvão: entendeu?

Walter Silva Oliveira: Fado da

Lucas Galvão: Eu pego no shopping é o

Walter Silva Oliveira: onde?

Lucas Galvão: Dourado,

Walter Silva Oliveira: Ah,

Lucas Galvão: que é uns 60 km,

Walter Silva Oliveira: sei, sei, sei. Já

Lucas Galvão: só que é muito trânsito. E a sua aula era bem de quinta,

Walter Silva Oliveira: é.

Lucas Galvão: quinta é o pior dia da semana.

Walter Silva Oliveira: Sei. Tô de olho.

Lucas Galvão: É sério?

E.1.24 00:26:48

Walter Silva Oliveira: Tô de oi em você.

Lucas Galvão: O quê? É sério?

Walter Silva Oliveira: Tô de oi. Você ia na minha aula.

Lucas Galvão: Por quê? O quê?

Walter Silva Oliveira: Pode ser. Mas você ia na minha aula, tá tudo

Lucas Galvão: Sim, exatamente isso que eu tô falando, pô.

Walter Silva Oliveira: certo.

Lucas Galvão: Deixa eu ver. É, então, porque esse semestre, nossa, esse semestre eu quero ir o menos possível pra faculdade,

Walter Silva Oliveira: E você vai sentir falta

Lucas Galvão: cara.

Walter Silva Oliveira: depois.

Lucas Galvão: Vou pior que sabe o que eu fico pensando? Eh, queria ver se consigo manter o contato aí com os meus colegas mais próximos aí, tipo o Lucas e o Felipe e o João. Mas eu acho bem improvável. A vez o Lucas vai, mas os outros acho bem improvável.

Walter Silva Oliveira: Por que você diz isso?

Lucas Galvão: Ah, porque que parece porque eu não converso com eles em outra situação sem ser na faculdade. Certo?

Walter Silva Oliveira: É assim, eu não pessoal que eu fiz a graduação poucos que a gente mantenho toda hora assim, mas pelo menos todo ano a gente se vê. Todo ano chega agosto que tem aniversário de uns quatro, aí a gente marca alguma coisa para comemorar e todo ano a gente se encontra.

E.1.25 00:28:07

Lucas Galvão: Ah, entendi. Então,

Walter Silva Oliveira: Mas ficar todo dia junto assim tem problema.

Lucas Galvão: ah, ah, mas ah,

Walter Silva Oliveira: Não pode perder o vínculo de vez.

Lucas Galvão: mas por exemplo,

Walter Silva Oliveira: É normal. Vida

Lucas Galvão: é sim,

Walter Silva Oliveira: adulta.

Lucas Galvão: cara. Acredita que meus amigos, eles são meus amigos desde, sei lá,

Walter Silva Oliveira: 3

Lucas Galvão: maior parte, não,

Walter Silva Oliveira: anos.

Lucas Galvão: 2012, assim, são os mesmos amigos que eu tenho, as mesmas pessoas que eu saio até hoje assim. Então, tipo assim, daí eu penso, é, mas aí não tenho amigos novos, entendeu? Por exemplo, daí, por exemplo, é muito conveniente ter os mesmos grupos de amigos, porque você continua indo nos mesmos eventos

Walter Silva Oliveira: É,

Lucas Galvão: assim.

Walter Silva Oliveira: aí você tem que se forçar a manter a amizade, que manter a amizade é que dá trabalho,

Lucas Galvão: É muito difícil.

Walter Silva Oliveira: né?

Lucas Galvão: É, então acho que é isso. Tem alguma coisa?

Walter Silva Oliveira: Eu

Lucas Galvão: Tá vendo aqui a cidade que eu tô? Ó, aqui, ó. Tá vendo? Se você fosse paraa indústria, ó, você estaria aqui, ó. Dubai. Olha aqui, ó, a visão bonita.

Walter Silva Oliveira: tô bem aqui de livro para mim tá bom.

Lucas Galvão: Uhum.

Walter Silva Oliveira: Mas eu acho que é isso, Lucas.

Lucas Galvão: Tá bom.

Walter Silva Oliveira: Acho que tá tá indo bem.

Lucas Galvão: Beleza.

Walter Silva Oliveira: Qualquer coisa você me avisa.

Lucas Galvão: Vamos jogar uma aranha lá.

Walter Silva Oliveira: Não prometo, mas podemos jogar um aranzinho.

Lucas Galvão: Bora então. É isso. Valeu,

Walter Silva Oliveira: Cachorro.

Lucas Galvão: professor.

Walter Silva Oliveira: Vai loucas. Qualqu coisa você me avisa, hein?

Lucas Galvão: Falou. Beleza, vou progredir com o que você falou e vou progredir também com a

Walter Silva Oliveira: Valeu.

Lucas Galvão: eh eu acho que eu já vou começar a fazer uns testes de benchmark, já

vou montar pelo menos os negócios.

Walter Silva Oliveira: Uhum. Vou já volta.

Lucas Galvão: Beleza,

Walter Silva Oliveira: Fechou.

Lucas Galvão: valeu,

Walter Silva Oliveira: Valeu,

E.1.26 A transcrição foi encerrada após 00:30:07

Esta transcrição editável foi gerada por computador e pode conter erros. As pessoas também podem alterar o texto depois que ele for criado.